

Interconnections and Communication Protocols

Prof. Claudio Passerone

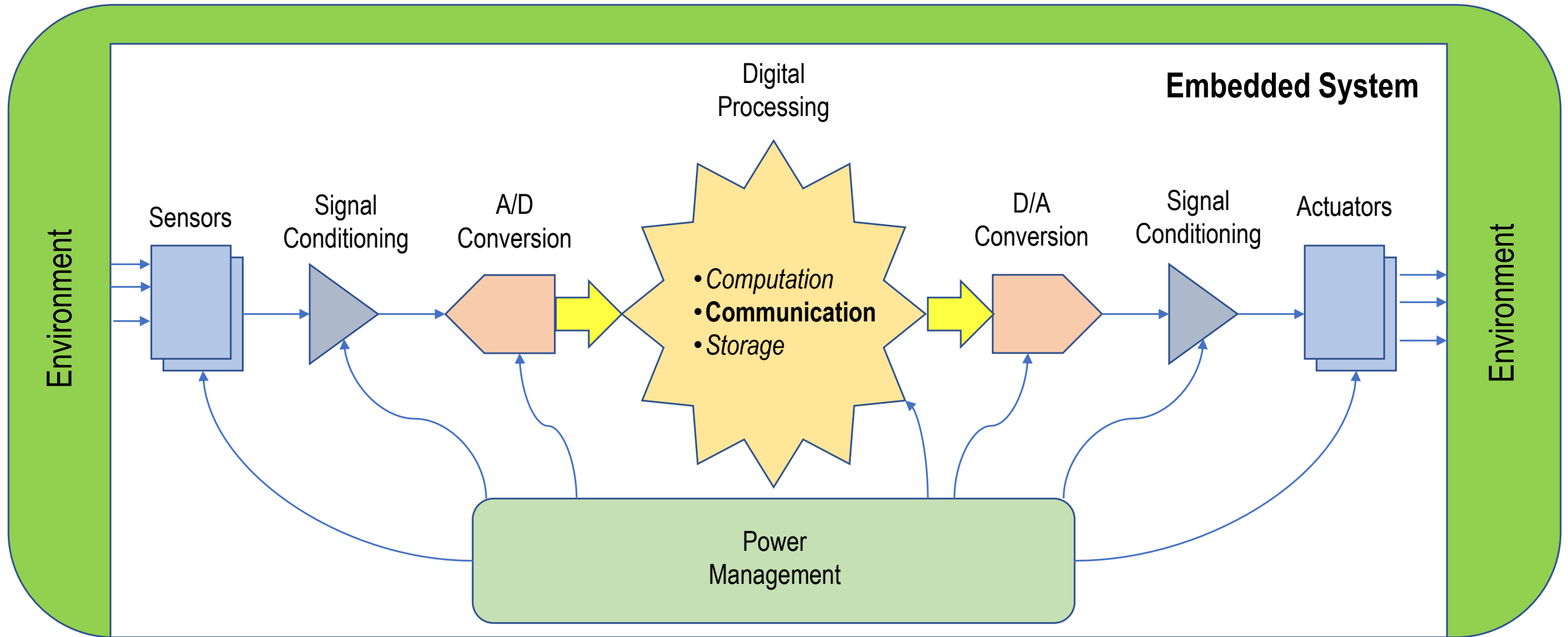
(Electronics for Embedded Systems)

Outline

- Introduction
 - Interconnection delays
 - Serial vs. Parallel Interconnections
- Asynchronous Protocols
 - UART
 - Eye diagram
- Synchronous Protocols
 - Encodings
 - NRZ, RZ, Bipolar, MTL, Manchester, MFM
 - SPI, I²C
 - USB, CAN

Introduction

- Typical architecture of an Embedded System



Introduction

- Interconnections can be at several levels
 - Within an integrated circuit
 - Metal layers, parallel bits
 - Lots of wires, high performance
 - Low parasitics, low noise
 - Delays can be accurately estimated (Elmore)
 - Example: AMBA Bus (see processor peripherals)
 - Between integrated circuits on the board
 - Copper lines on a PCB, parallel or serial
 - Higher noise, higher delays than in an integrated circuit
 - We analyze in detail
 - Example: memory interface, SPI, camera interface, UART

Introduction

- Interconnections can be at several levels
 - Between boards in the same embedded system
 - Cables made of wires, parallel or serial
 - Even higher noise and delays
 - Example: USB, Ethernet, RS232, CAN, ATA, PCI
 - Between embedded systems
 - Cable made of wires or wireless connections
 - Unpredictable delays and extremely high noise
 - Need to use at least error detection in many cases
 - Example (wired): Ethernet, RS485, CAN
 - Example (wireless): WiFi, Bluetooth, NFC
 - We see some of the wired interconnections

Introduction

- Goal of interconnections
 - Transfer information (bits) as quickly as possible
 - While not sacrificing other metrics
 - Correctness
 - Reliability
 - Power consumption
 - Costs
 - ...
- Interconnections are defined at different levels
 - Physical
 - Protocol
 - Application
 - See standard stacks (e.g., ISO)

Introduction

- In this course, we mostly deal with the physical and protocol levels
 - Focus on how to transfer bits from a transmitter to a receiver
 - Achieve bit synchronization
 - In some cases, details on transfers of words (characters)
- At the protocol level:
 - Examine the information needed to successfully complete a transfer
 - No details on the semantic content of the messages that are transferred
 - Semantic depends on the application

Interconnection delays

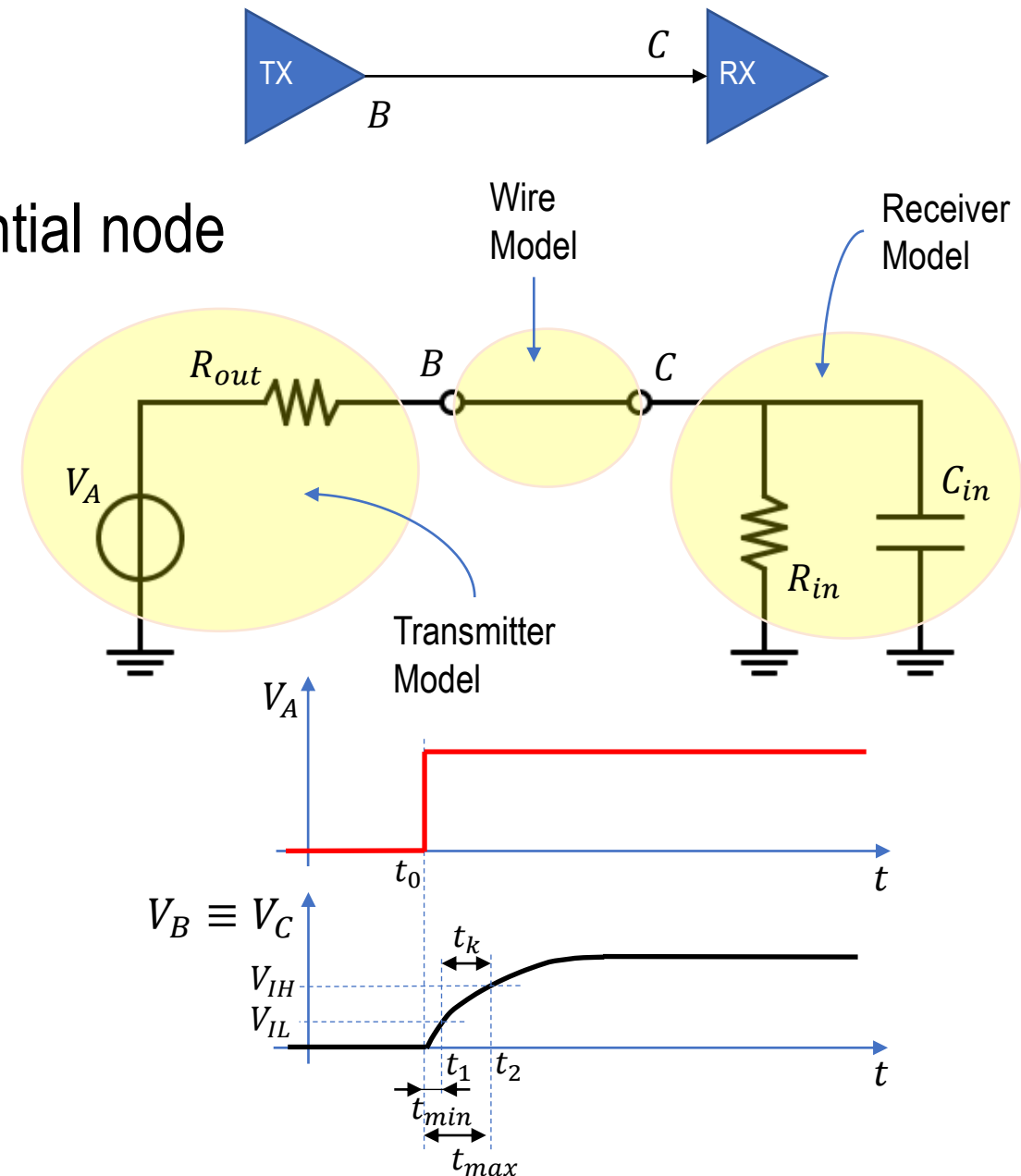
- When connecting the output of a logic gate to the input of another logic gate, delays affect the exchange of data
 - Measure or estimate the delay between a transition of the transmitting logic gate and the instant in which the transition is detected by the receiving logic gate
 - Even if the transmitter generates an ideal step, the waveform at the receiver is generally exponential with a certain time constant
 - This is due to parasitic resistance, capacitance and inductance of transmitter, receiver and wire
 - Difference in switching thresholds of the receiving logic gates causes different delays for nominally identical circuits
 - The delay is affected by an uncertainty
 - The uncertainty is called *skew*

Interconnection delays

- Model an interconnection as an ideal equipotential node

- Assume an ideal low to high transition at TX
 - Voltage V_A makes an ideal step at time t_0
- Waveforms in B and C are identical
- Delay depends on the receiving gate threshold
 - Given a certain logic family, it is between V_{IL} and V_{IH}
- Exponential response

- Time constant $\tau = C_{in} \cdot (R_{in} \parallel R_{out})$
- $t_{min} = t_1 - t_0$ is the minimum delay
 - When the gate threshold is equal to V_{IL}
- $t_{max} = t_2 - t_0$ is the maximum delay
 - When the gate threshold is equal to V_{IH}
- $t_k = t_{max} - t_{min} = t_2 - t_1$ is the skew

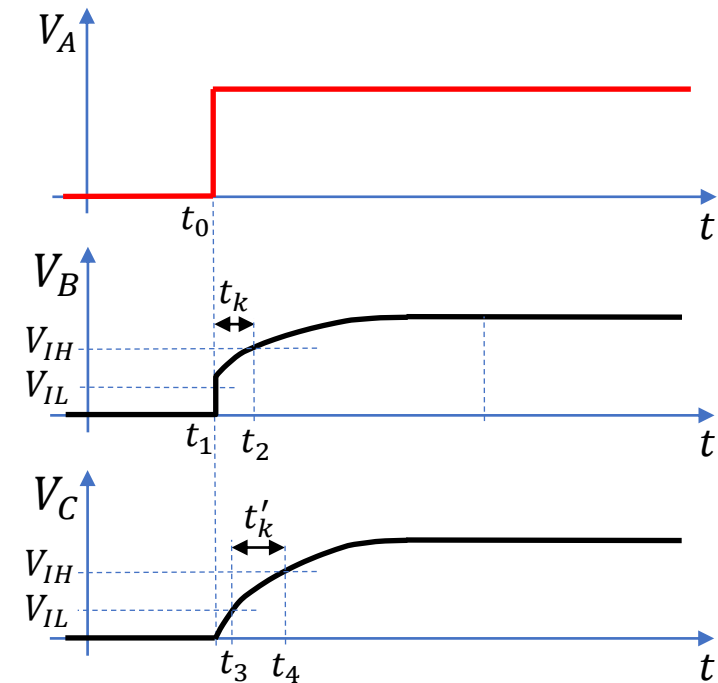
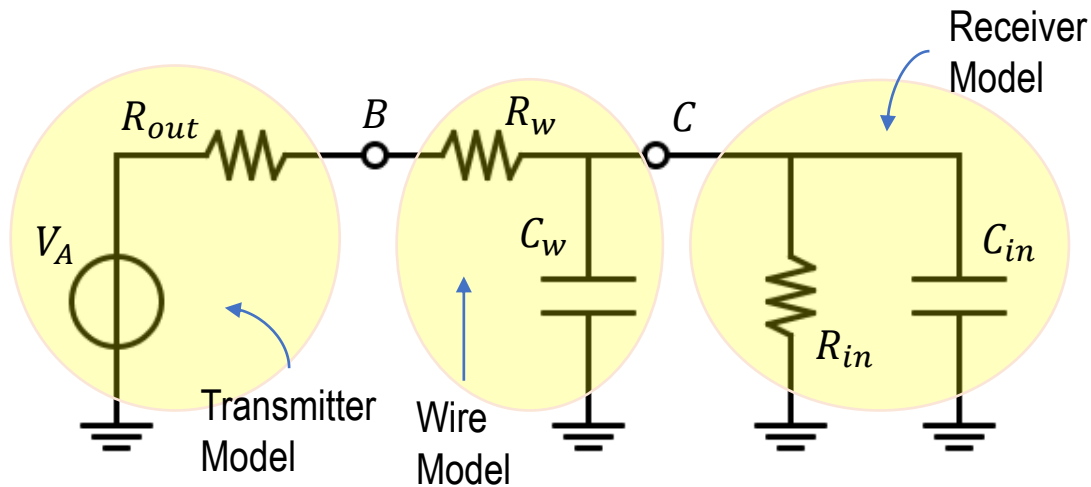


Interconnection delays

- Model an interconnection as an ideal equipotential node
 - Even if the interconnection is ideal...
 - Even if the transmitter transition is ideal...
 - ... the transmission shows a delay
 - Due to the input parasitic capacitance of the receiver
 - ... the delay shows an uncertainty (skew)
 - Due to different thresholds among nominally identical logic gates
 - Delay does not depend on
 - The position of the receiver along the interconnection wire
 - The length of the wire

Interconnection delays

- Model an interconnection as lumped parasitic resistance R_w and capacitance C_w
 - Assume an ideal low to high transition at the transmitter
 - Waveforms in B and C are different
 - To correctly characterize the entire interconnection
 - Find the minimum of all minimum delays
 - Find the maximum of all maximum delays
 - Compute average and skew

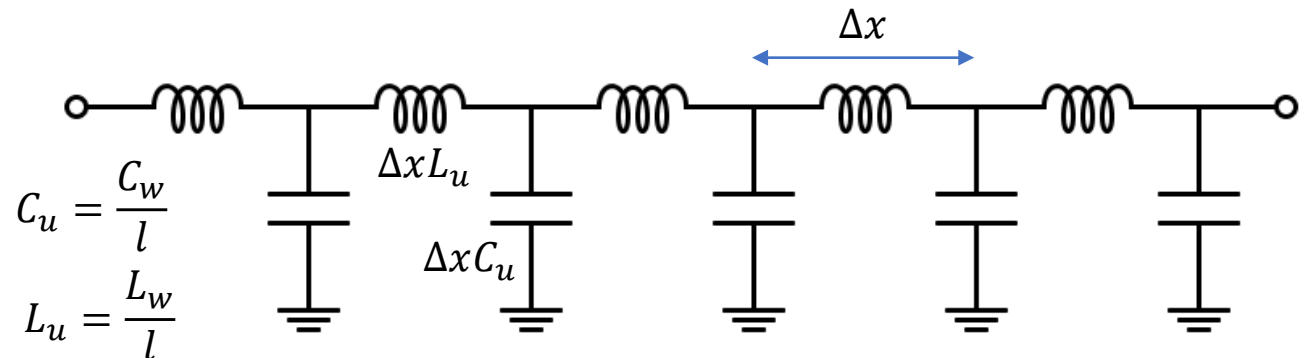


Interconnection delays

- Model an interconnection as lumped parasitic resistance R_w and capacitance C_w
 - The transmission shows a delay
 - Due to the input parasitic capacitance of the receiver
 - Due to the lumped parasitic capacitance of the wire
 - The delay shows an uncertainty (skew)
 - Due to different thresholds among nominally identical logic gates
 - Due to possibly different positions of the receiver along the interconnection wire
 - Wire length
 - It causes different parasitic resistance R_w and capacitance C_w
 - It affects both the delay and its uncertainty

Interconnection delays

- Model an interconnection using distributed parasitics
 - Divide the wire of length l into N segments, each with a lumped model
 - Each segment has a length $\Delta x = l/N$
 - Take the limit for the number of segments going to infinite ($N \rightarrow \infty$)
 - Each segment has length dx , with parasitics per unit of length
 - If considering only inductance and capacitance
 - $\rightarrow$ Ideal transmission line with no losses
 - Signal propagates with a certain speed, with possible reflections
 - Delay depends on the position of the receiver along the line
 - Can compute an average and a skew
 - If resistances are also included
 - $\rightarrow$ Losses are introduced in the model



Interconnection delays

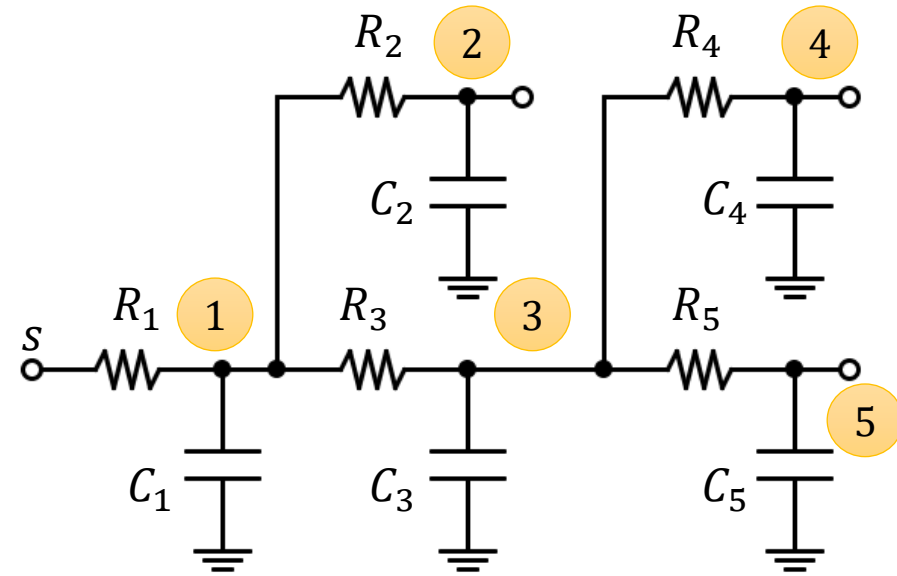
- Model an interconnection using distributed parasitics
 - Divide the wire into N segments, each with a lumped model
 - For complex interconnections, you can determine a tree
 - There is a single root node s
 - There are multiple outputs (leaves of the tree)
 - Each segment is characterized by a resistance R_j and a capacitance C_j
 - Elmore delay
 - Define the shared path resistance between node i and node k
 - $R_{ik} = \sum R_j \Rightarrow (R_j \in [path(s \rightarrow i) \cap path(s \rightarrow k)])$
 - Example: $R_{45} = R_1 + R_3$
 - The time constant τ_i of the delay at a node i is approximately given by

$$\tau_i = \sum_{k=1}^N C_k R_{ik}$$

Examples:

$$\tau_2 = C_1 R_1 + C_2 (R_1 + R_2) + C_3 R_1 + C_4 R_1 + C_5 R_1$$

$$\tau_5 = C_1 R_1 + C_2 R_1 + C_3 (R_1 + R_3) + C_4 (R_1 + R_3) + C_5 (R_1 + R_3 + R_5)$$



Interconnection delays

- Model an interconnection using distributed parasitics

- Elmore delay for an RC chain

- The shared path resistance is always the sum of all resistance between root s and the considered node
 - For the last node i the shared path resistance with node k is always

- $R_{ik} = R_{kk}$

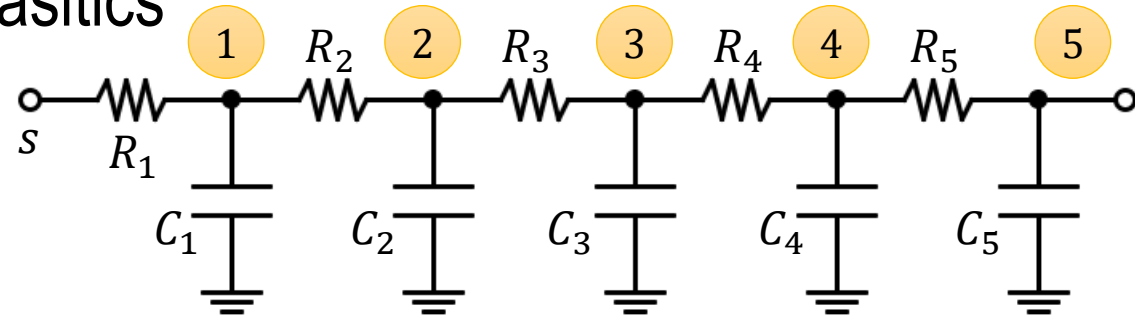
- Example: $R_{53} = R_{33} = R_1 + R_2 + R_3$

- For a wire of length l divided into N identical segments

- All resistances are identical: $R_i = r \frac{l}{N}$ (where r is the resistance per unit of length)
 - All capacitances are identical: $C_i = c \frac{l}{N}$ (where c is the capacitance per unit of length)

$$\tau_i = R_{tot} C_{tot} \frac{N+1}{2N} \quad \text{where } R_{tot} = NR_i \text{ and } C_{tot} = NC_i$$

When N is large:
$$\tau_i = \frac{R_{tot} C_{tot}}{2} = \frac{1}{2} N^2 R_i C_i$$



Interconnection delays

- Whatever model is considered
 - Delays affect the exchange of information between logic gates
 - The value of the delay is not deterministic, but depends on several parameters
 - Technological
 - Thresholds, Voltage supplies, ...
 - Geometrical
 - Lengths, Positions, ...
 - Environmental
 - Temperature, Humidity, Noise, ...
 - Did not show these dependencies in previous slides
 - The exact delay is not known a priori and has random variations within a certain range
 - Skew is a measure of the uncertainty of the delay

Interconnection delays

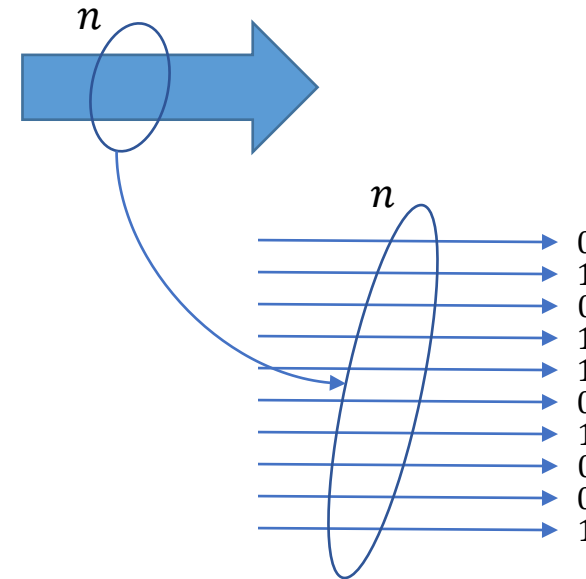
- Interconnection delay expressed as:
 - $t_d = t_{pd} \pm \frac{t_k}{2}$
 - Sum of an average propagation delay t_{pd} , plus or minus half of the skew t_k
 - There are layout techniques for ICs, PCBs and cables to decrease the skew
 - But it is not avoidable
 - Skew is greater for long interconnections
 - Skew is affected (increased) by the presence of noise
 - Circuits shall be designed such that they are insensitive to delay uncertainties
 - Always consider the worst case scenarios

Parallel vs. Serial Interconnections

- Transmit a word of n bits
- Bits transmitted according to a synchronization signal
 - The synchronization signal is a clock
 - Clock frequency f_{ck} , clock period $t_{ck} = \frac{1}{f_{ck}}$
 - Clock shall be transmitted as well
 - There are several ways of communicating the clock
- Compare techniques based on throughput
 - Number of bits per second Thr_{bits}
 - Number of words per second Thr_{words}
- Assume transmitted symbols represent single bits
 - Can increase performance by using multiple bits per symbol
 - Lower noise margins

Parallel vs. Serial Interconnections

- Parallel (word of n bits)
 - Transmit an entire word using n parallel wires
 - An additional wire for the clock
 - Transmit a word at each clock cycle
 - Number of bits per second $Thr_{bits} = n f_{ck}$
 - Number of words per second $Thr_{words} = f_{ck}$
 - The number of bits is usually a power of 2
 - Potentially fast...
 - Exploits parallelism
 - ... but certainly expensive
 - Need a lot of wires for large busses
 - Not good for long interconnections
 - Heavy, bulky, large size



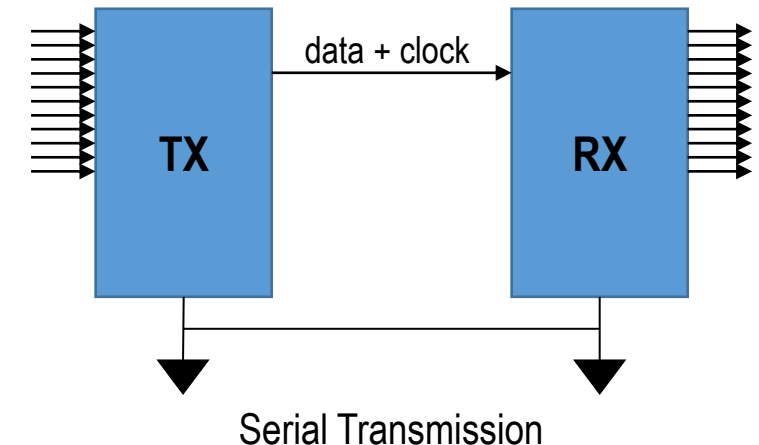
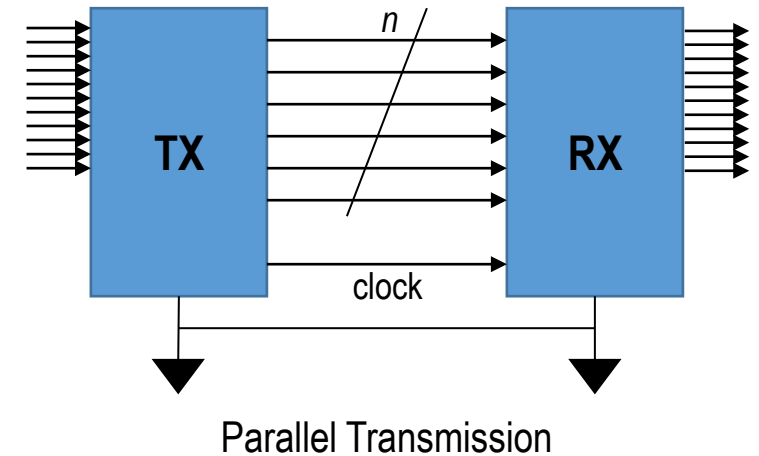
Parallel vs. Serial Interconnections

- Serial (word of n bits)
 - Transmit the word and the clock using a single wire
 - Bits are multiplexed in time
 - One per clock cycle
 - One after the other, sequentially (LSB or MSB first)
 - Introduces latency
 - Clock must be *embedded* in some way with the data
 - Need special encodings for clock recovery
 - Use multiple clock cycles to transmit a word
 - Number of bits per second $Thr_{bits} = f_{ck}$
 - Number of words per second $Thr_{words} = \frac{f_{ck}}{n}$
 - Potentially slow compared to parallel...
 - ... but certainly cheaper
 - A single wire is sufficient, regardless of the value of n

0 1 0 1 1 0 1 0 0 1
→

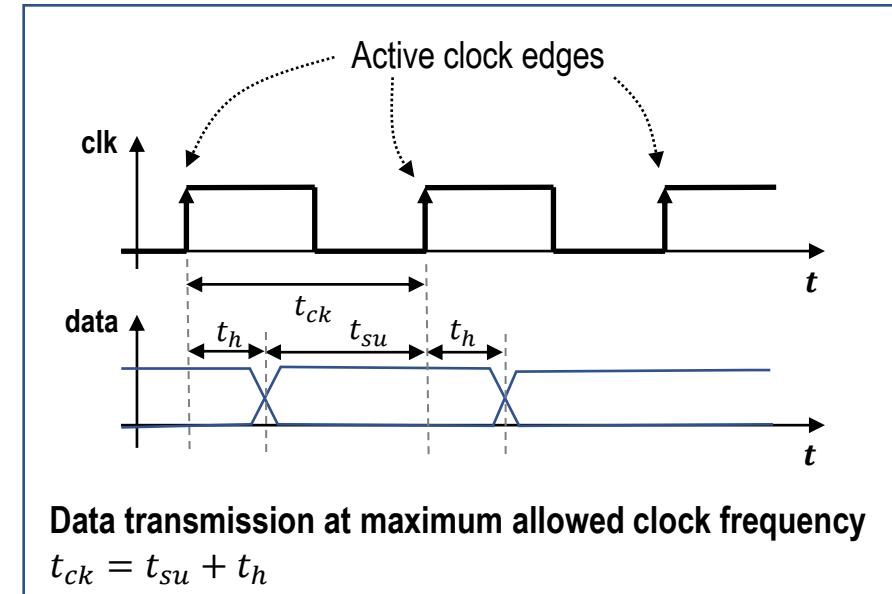
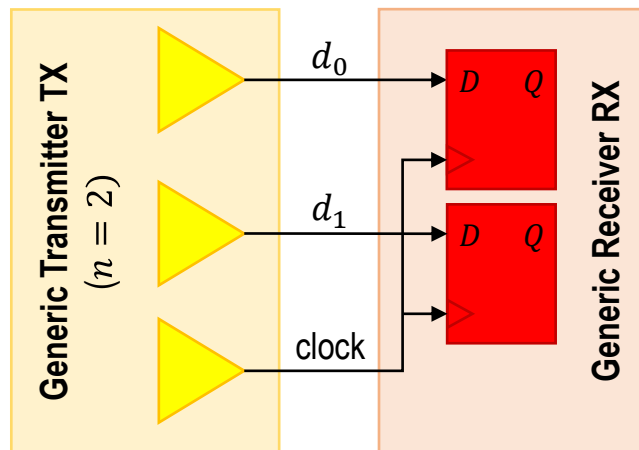
Parallel vs. Serial Interconnections

- TX and RX are transmitter and receiver respectively
 - Assume clock is generated and transmitted by TX
 - This is not always the case
- Ground wire(s) along with data and clock wires
 - Not considered in previous slides
- Signals are digital in most cases
- Voltage levels usually correspond to logic levels
 - High: 3.3V, 5V, custom (within an IC)
 - Low: 0V (GND)
 - Encodings used to represent bits



Parallel vs. Serial Interconnections

- Limitations on the clock frequency
 - The receiving end usually has flip-flops to store the incoming data
 - Setup and hold times t_{su} and t_h must be satisfied at the receiving end
 - With no transmission delays and perfect synchronization
 - Maximum f_{ck} depends on t_{su} and t_h of the receiving flip-flops only
 - $t_{ck} > t_{su} + t_h$, $f_{ck} < \frac{1}{t_{su} + t_h}$

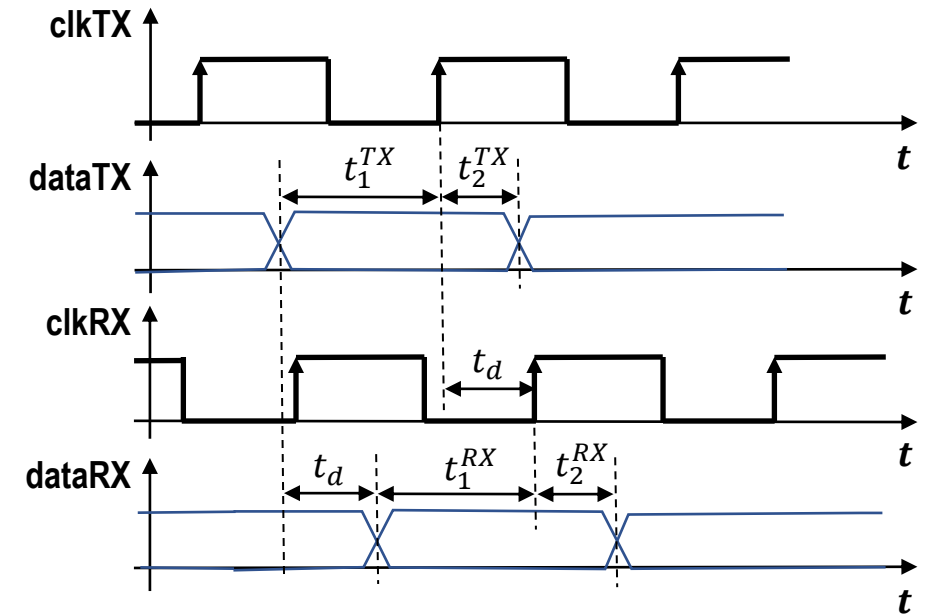


Parallel vs. Serial Interconnections

- Limitations on the clock frequency
 - When TX is driving both data and clock in parallel, signals shall be generated such that setup and hold time are satisfied at RX
 - It is not sufficient that t_{su} and t_h are satisfied at the transmitter
 - Delays between TX and RX may impose further limitations to the clock frequency
 - The transmitter TX must generate data and clock taking into account transmission delays
 - Let's define:
 - t_1^{TX} : time data is stable at the TX side before the active clock edge
 - t_1^{RX} : time data is stable at the RX side before the active clock edge
 - t_2^{TX} : time data is stable at the TX side after the active clock edge
 - t_2^{RX} : time data is stable at the RX side after the active clock edge
 - It must be $t_1^{RX} > t_{su}$ and $t_2^{RX} > t_h$

Parallel vs. Serial Interconnections

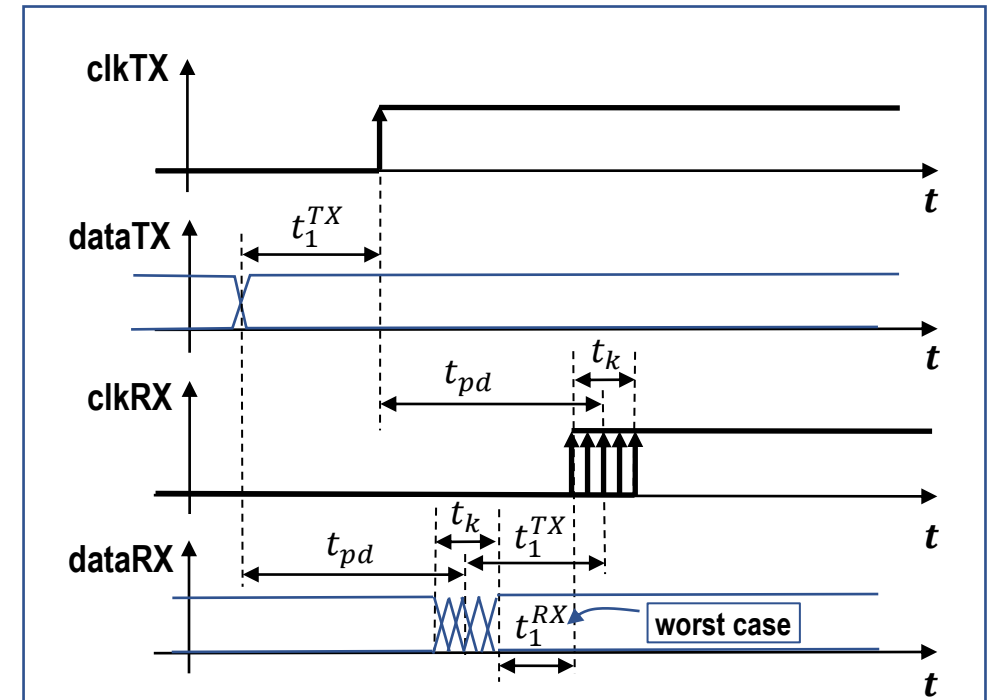
- Assume delays known with 100% accuracy and constant ($t_k = 0$)
 - Transmission delay of data and clock exactly equal to the propagation delay
 - $t_d = t_{pd}$
 - Phase delay between data and clock maintained
 - $t_1^{RX} = t_1^{TX}, t_2^{RX} = t_2^{TX}$
 - TX can generate signals according to the setup and hold requirements
 - Max clock frequency still depending on t_{su} and t_h only
 - t_d can have any value
 - It does not affect the maximum performance (throughput)
 - It causes a latency in the transmission
 - If clock and data have different (but known) delays
 - Transmitter properly produces the clock earlier or later so that setup and hold times are satisfied at the receiver



Parallel vs. Serial Interconnections

- Delays with uncertainties
 - $t_d = t_{pd} \pm t_k/2$
 - t_k is the transmission skew
 - Phase delay between data and clock not maintained
 - Delays are unpredictable and different in clock and data
 - Delays change from one bit transmission to the next
- Transmitter should consider the worst case scenario
 - For the setup time:
 - Clock arrives early at RX after $t_d^{ck} = t_{pd} - t_k/2$
 - Data arrives late at RX after $t_d^{data} = t_{pd} + t_k/2$
 - $t_1^{RX} = t_1^{TX} + t_d^{ck} - t_d^{data} = t_1^{TX} - t_k$
 - Since it should be $t_1^{RX} > t_{su}$, it derives that

$$t_1^{TX} > t_{su} + t_k$$



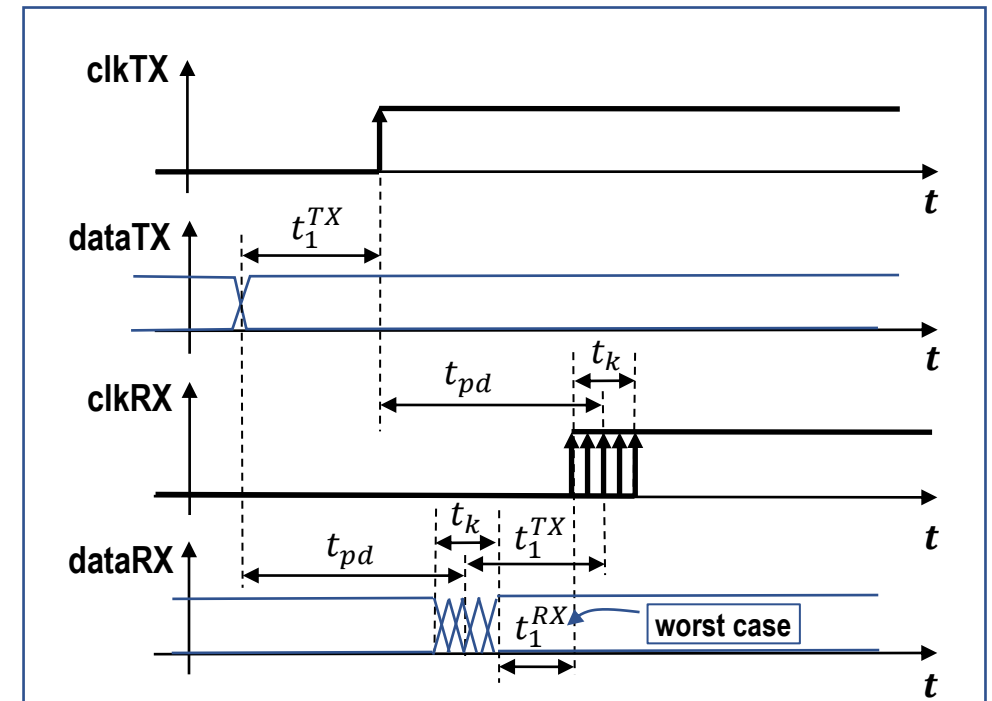
Parallel vs. Serial Interconnections

- Delays with uncertainties

- Considering similar expressions for the hold time in the worst case scenario, the maximum clock frequency at the transmitter is given by:

$$f_{ck} < \frac{1}{t_{su} + t_h + 2t_k}$$

- The maximum clock frequency is severely affected by the skew
- This applies to parallel transmission only
- Note that serially transmitting bits on a single wire and the clock on a separate wire is considered parallel transmission



Parallel vs. Serial Interconnections

- Performance of serial transmission is not affected by skew
 - The transmitted signal actually always shows a delay with some skew...
 - ... but data and clock, being on the same wire, are affected in the same way
 - If data arrives early, clock arrives early
 - If data arrives late, clock arrives late
 - → the phase delay between data and clock is maintained
 - If skew is significantly higher than setup and hold times
 - The clock frequency of a serial transmission can be significantly higher than the clock frequency of a parallel transmission
 - Serial can have higher performance than parallel when
 - $f_{ckser} > n \cdot f_{ckpar}$

Parallel vs. Serial Interconnections

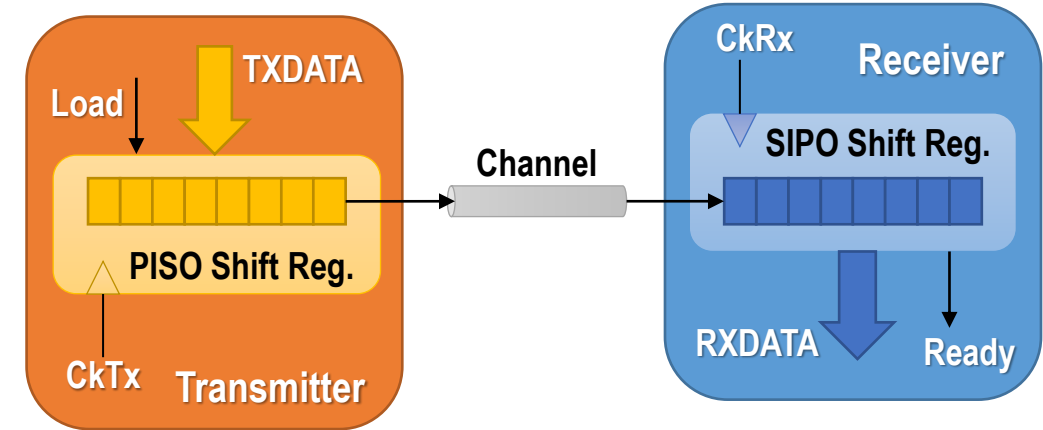
- Most high performance interconnections are serial
 - Especially for long wires (high skew)
 - Serial ATA replaced Parallel ATA
 - Similarly for SAS and SCSI
 - PCI Express (serial) replaced PCI (parallel)
 - USB is ubiquitous
 - Replaced various interconnection protocols, such as Centronics (parallel), but also RS232 (UART, serial)
 - HDMI is used in digital video (6Gb/s)
- Parallel busses survive outside ICs only for
 - Memory interconnections
 - High performance, need extreme care in design
 - Low performance communications with short interconnections

Parallel vs. Serial Interconnections

- Interconnections allow exchanging data between logic gates in the digital domain
- Interconnections introduce a delay with some uncertainty (skew)
- For parallel interconnections:
 - The delay introduces a latency, which does not affect the throughput
 - Latency can be a problem in some cases
 - The skew limits the maximum clock frequency, and hence the throughput
- For serial interconnections (with embedded clock):
 - The delay introduces a latency, which does not affect the throughput
 - The skew, although present, does not (significantly) affect the throughput
- Conclusions:
 - For interconnections with large skew, serial is often faster than parallel
 - For interconnections with small skew, parallel can be faster, but more expensive than serial

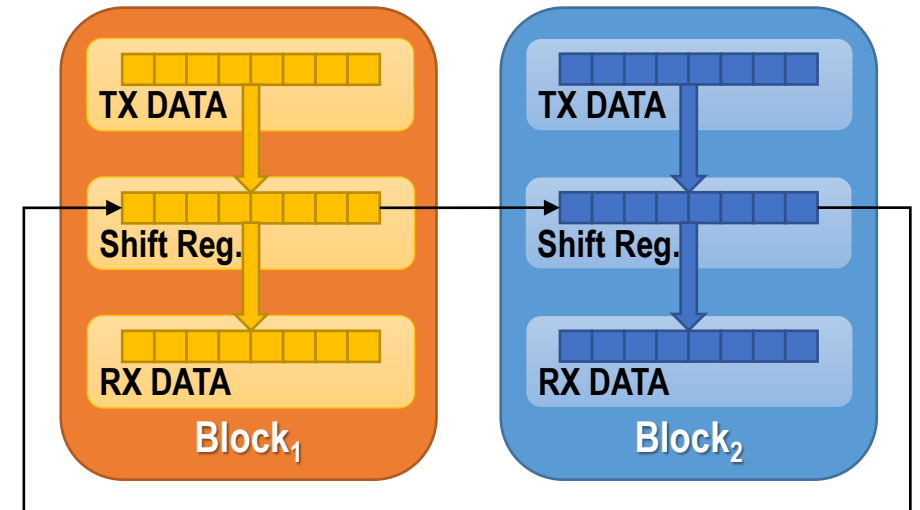
Parallel vs. Serial Interconnections

- Basic serial transmission
 - The signal transmitted is a sequence of symbols
 - Each symbol carries a single bit or multiple bits
 - Data at the transmitter is copied into a PISO SR
 - Load signal is activated
 - Data transferred from the transmitter to the receiver through a communication channel
 - Receiver has a SIPO SR
 - Transmit and receive clocks are synchronized
 - They maintain a known phase relationship
 - Data at the receiver is finally available
 - Ready signal is asserted



Parallel vs. Serial Interconnections

- Basic serial transmission full duplex
 - Data to be transferred is in the TXDATA register of each block, respectively
 - Data from TXDATA is transferred to the Shift Register
 - Blocks make this transfer without synchronizing
 - Shift Registers of both blocks are clocked synchronously
 - Data is shifted bit by bit from one shift register to the other
 - After n clock cycles, data has been transferred
 - The content of each Shift Register is transferred to the RXDATA register
 - Blocks make this last transfer without synchronizing

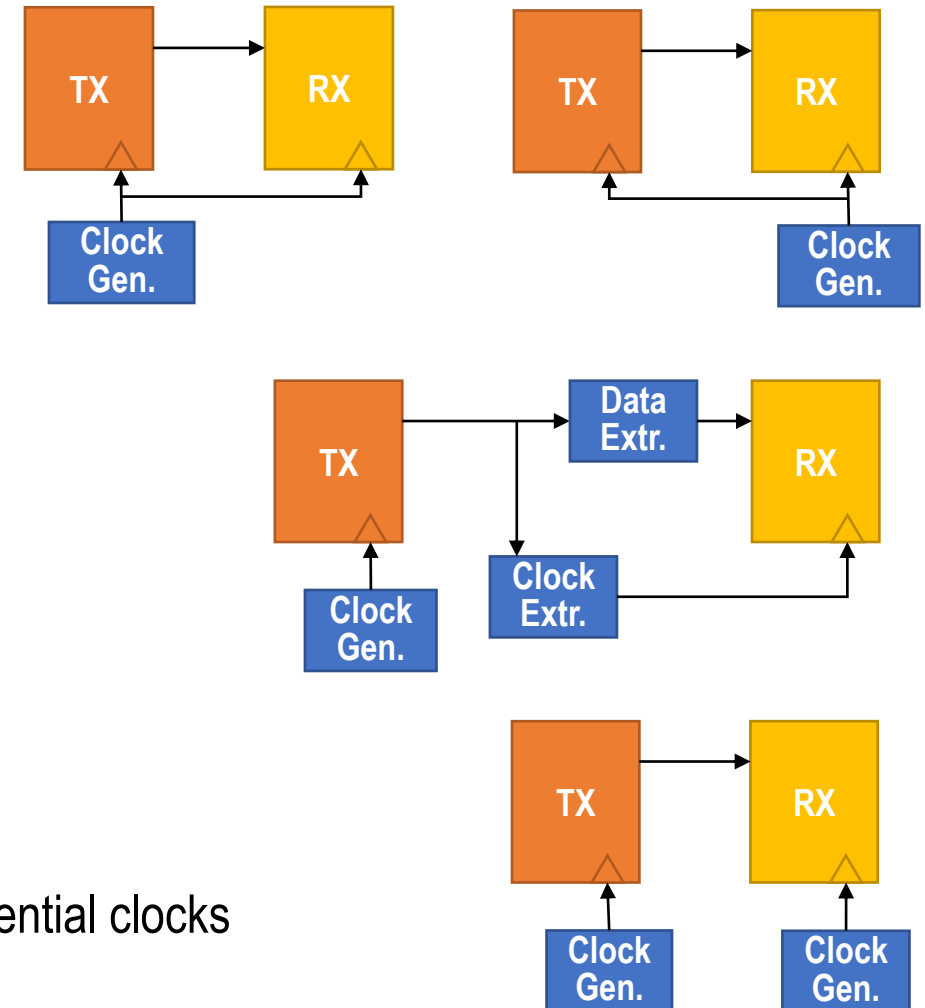


Parallel vs. Serial Interconnections

- Proper clock transmission allows bit synchronization
 - Guarantees correct sampling of single bits
 - Loading in the receiver shift register
- Higher protocol levels allow character and message synchronization
 - Character
 - For serial protocols, define LSB or MSB first
 - For serial protocols, define the length of a character
 - Handshake (using other wires) or message content to define when ready to send or to receive characters
 - Message
 - Define the semantic of characters
 - Header, Length, Address, Priority, Data, ...

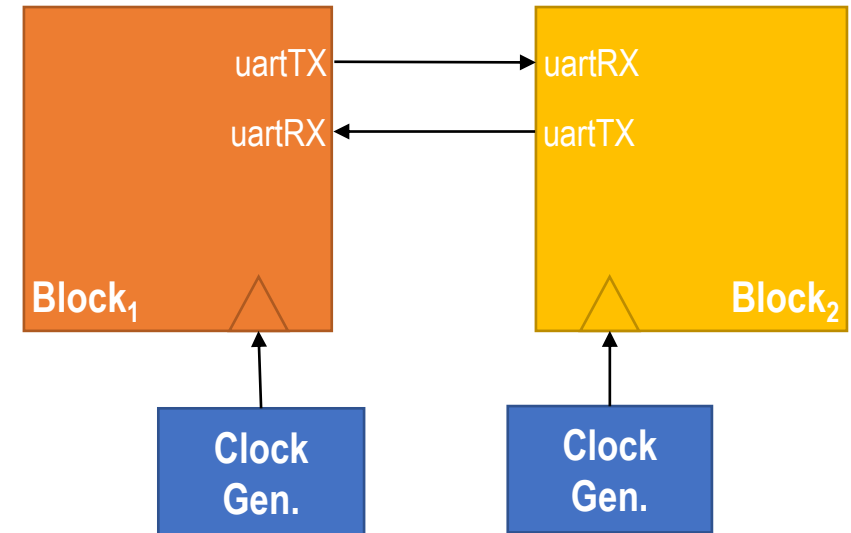
Parallel vs. Serial Interconnections

- Generating the clock
 - At the transmitter side
 - Sent to receiver on a parallel line with respect to data
 - Limited by skew
 - At the receiver side
 - Sent to transmitter on a parallel line with respect to data
 - Limited by skew and transmission delay
- Embedded (Clock Data Recovery, CDR)
 - Generated by transmitter, encoded in the data (presence of transitions), extracted by the receiver
 - Periodically (define maximum interval between transitions)
 - Continuously
- Isofrequential
 - Transmitter and receiver have separate free running isofrequential clocks
 - Periodically synchronized



Universal Asynchronous Receiver Transmitter (UART)

- Universal Asynchronous Receiver Transmitter
 - Point-to-Point serial communication
 - Isofrequent independent clocks
 - Two serial wires
 - One for transmission from Block₁ to Block₂
 - One for transmission from Block₂ to Block₁
 - Plus a third wire for GND, not shown
 - In the following only one wire is considered (plus GND)
 - Transmitter = Block₁, Receiver = Block₂
 - The opposite direction follows the same rules
 - Periodic synchronization of the clocks
 - After synchronization clocks run free

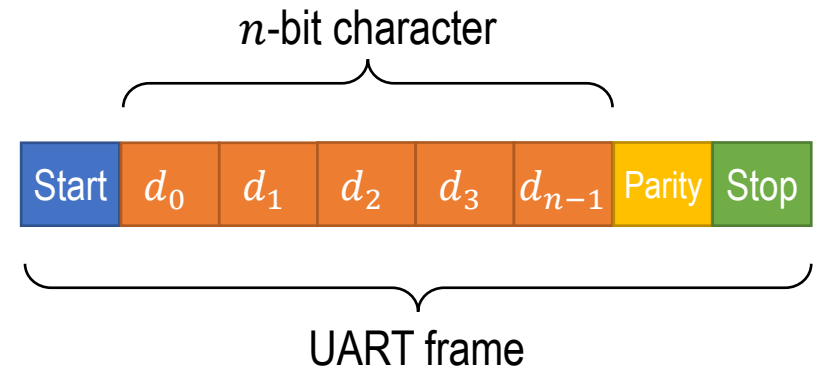


Universal Asynchronous Receiver Transmitter (UART)

- Isofrequent independent clocks
 - Nominally identical, same frequency
 - Need at least to be synchronized once to set the phase shift between active edges
 - But tolerances in fabrication cause drifts
 - Frequencies are not perfectly identical
 - The phase relationship changes over time
 - The change is unpredictable
 - Can only estimate the worst case
 - Need periodic synchronization to re-set the phase shift between active edges
 - Estimate how often synchronization is needed (eye diagram)
- Environmental quantities affect clocks
 - Temperature, atmospheric pressure, humidity, ...

Universal Asynchronous Receiver Transmitter (UART)

- Character based transmission
 - Each transmission consists of a frame
 - A frame always contains one character of n bits
 - A frame has a defined format
 - START BIT (1 bit)
 - Character (n bits, LSB first)
 - PARITY BIT (1 bit, optional, might be absent)
 - STOP BIT (1 or 2 bits)
 - Synchronization between clocks at the START BIT only
 - A complex message requires the transmission of several characters
 - After a frame is completed, a new frame can be transmitted



Universal Asynchronous Receiver Transmitter (UART)

- Transmitter and receiver should agree on the following parameters

- Baud Rate (symbols per second)

- It is the nominal clock frequency f_{ck} of the independent clocks
- It corresponds to bps in this context
 - But symbols in general may represent more than one bit
- Common values: 300, 1200, 2400, 4800, 9600, 19200, 28800, 38400, 57600, 115200

- Number n of bits in a character

- Common values: 6, 7, 8, 9

- Presence and kind of parity

- No parity (N), Even parity (E), Odd parity (O), Mark (M), Space (S)

- Number of STOP BITS

- Either 1 or 2

- Configuration of parameters is outside the UART protocol

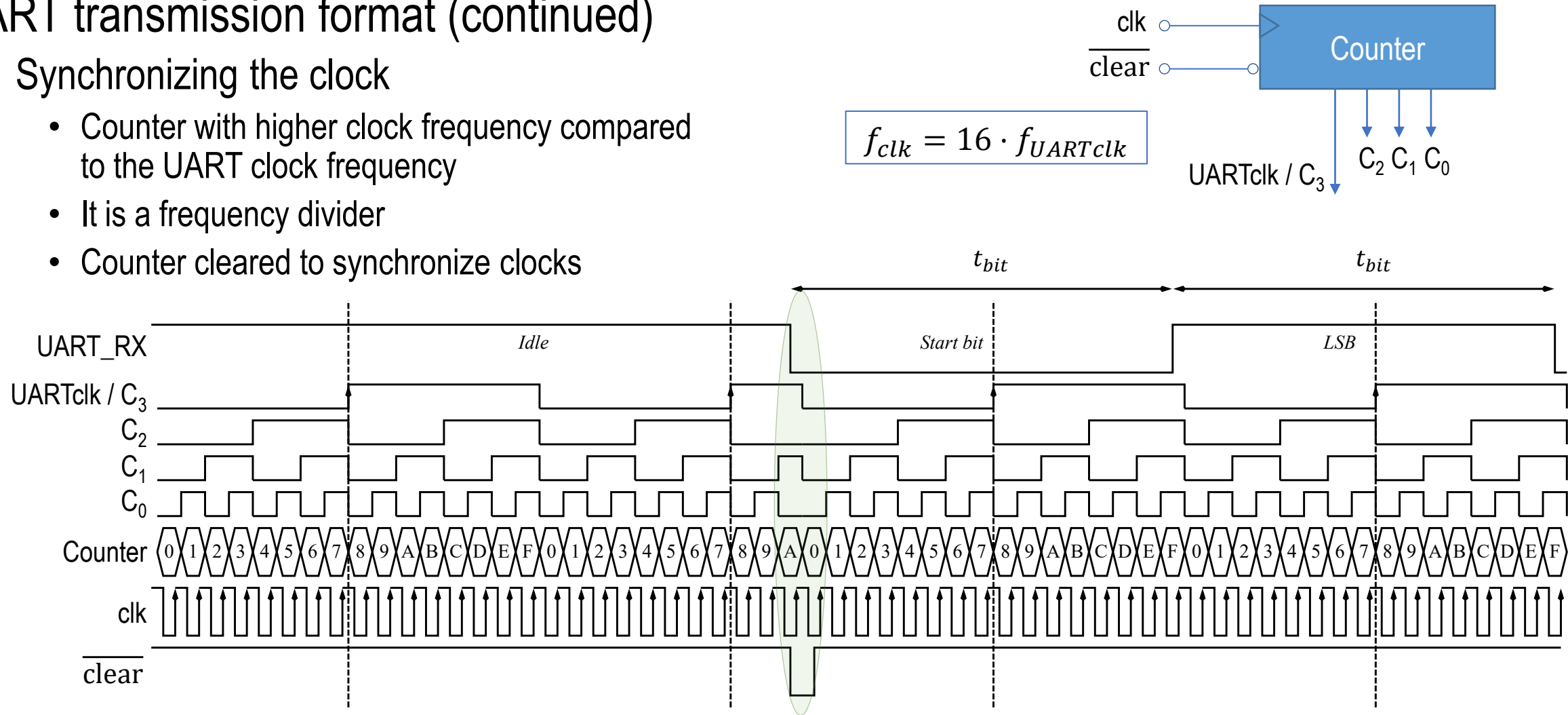
BaudRate	Bit	Parity	Stop
300	6	N	1
1200	7	E	2
2400	8	O	
4800	9	M	
9600		S	
19200			
28800			
38400			
57600			
115200			

Universal Asynchronous Receiver Transmitter (UART)

- UART transmission format
 - An UART frame can start at any time
 - Each bit nominally lasts one clock cycle
 - $t_{bit} = 1/f_{ck}$
 - Character and parity bits transmitted using standard NRZ encoding
 - No guaranteed transitions
 - How to guarantee a transition in each frame?
 - UART line kept at logic 1 when idle
 - STOP BITS are always at logic 1
 - START BIT is always a logic 0
 - Either after an idle phase, or following the previous frame, the START BIT of a new frame always causes a 1 to 0 transition
 - The START BIT transition allows synchronizing clocks at the beginning of a frame
 - There is always at least a 0 to 1 transition, as well, but its position is not known a priori, so it cannot be used for synchronization

Universal Asynchronous Receiver Transmitter (UART)

- UART transmission format (continued)
 - Synchronizing the clock
 - Counter with higher clock frequency compared to the UART clock frequency
 - It is a frequency divider
 - Counter cleared to synchronize clocks

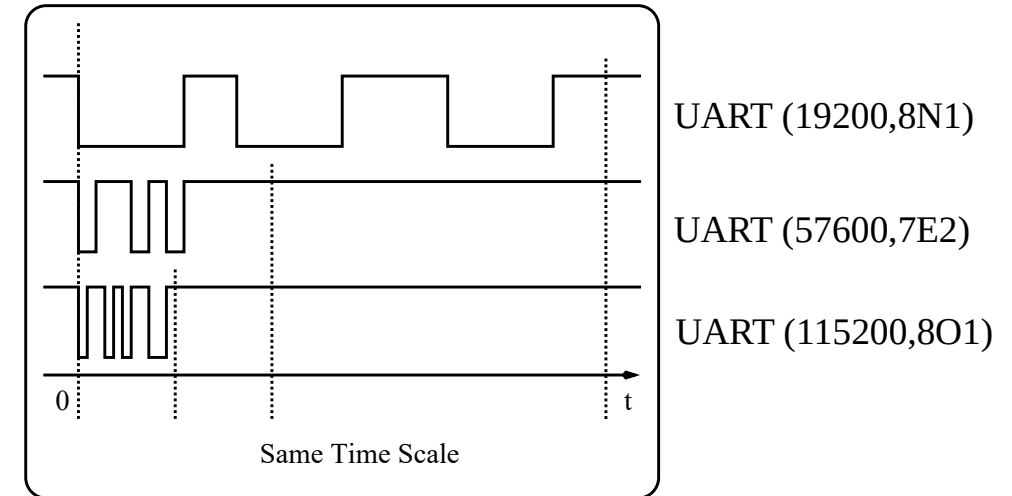
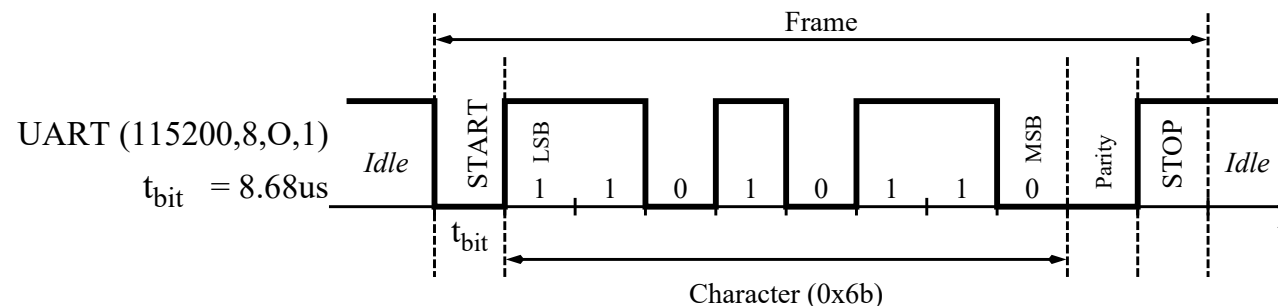
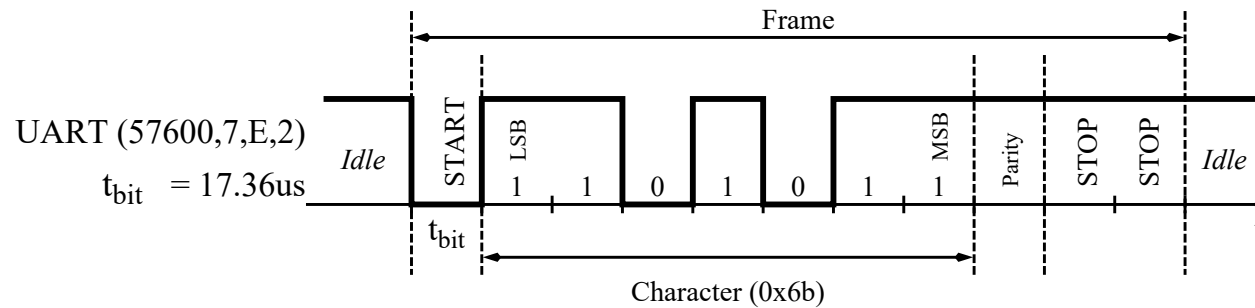
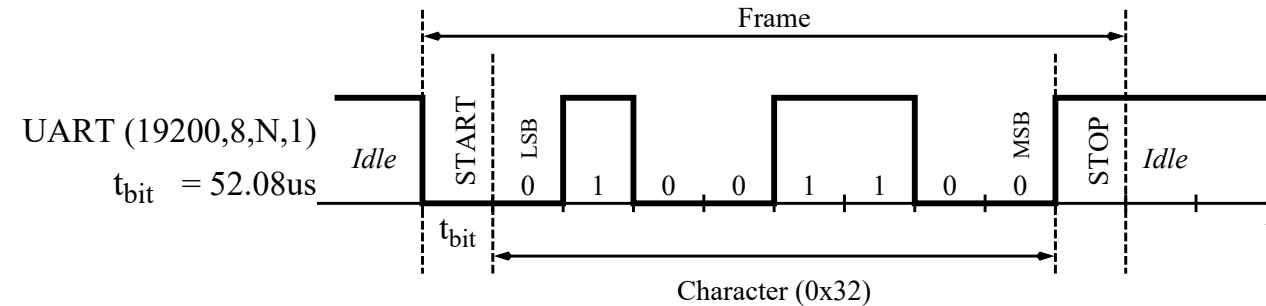


Universal Asynchronous Receiver Transmitter (UART)

- UART transmission format (continued)
 - Character bits are transmitted LSB first
 - The character bits are followed by the optional parity bit, if present
 - Even parity: the total number of bits equal to 1, including the parity bit (but excluding the STOP BITS), should be even
 - Odd parity: the total number of bits equal to 1, including the parity bit (but excluding the STOP BITS), should be odd
 - STOP BITS follow the parity bit, or the MSB of the character if parity is absent
 - A new frame may start immediately after the STOP BITS, or otherwise the line becomes idle (it remains at logic 1)
 - Voltages for logic 1 and logic 0 depend on the actual implementation
 - TTL: logic 0 = 0V, logic 1 = 3.3V
 - RS232: logic 0 between +3V and +25V, logic 1 between -3V and -25V
 - Need conversion circuits (MAX232, for instance)

Universal Asynchronous Receiver Transmitter (UART)

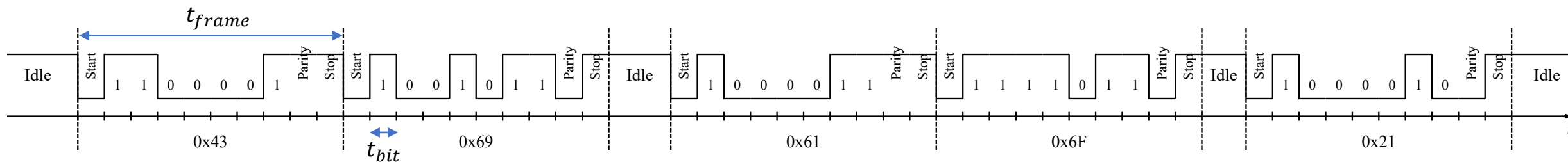
- UART transmission format examples



Universal Asynchronous Receiver Transmitter (UART)

- UART transmission format examples
 - A new character may start immediately after the previous one
 - A START BIT follows a STOP BIT
 - No idle time after the STOP BIT of the previous character
 - An idle time may be present between two consecutive characters
 - The UART line remains at logic high during the idle time
 - The length of the idle time is not fixed
- Sending 5 characters, 9600-7E1
 - $t_{bit} = 1/9600 = 104 \mu s$, $t_{frame} = (7 + 3) \cdot t_{bit} = 1.04 \text{ ms}$
 - Maximum performance: 960 frame/s, Useful bits: $960 \cdot 7 = 6720 \text{ bits/s}$

The number of useful bits per second is always lower than the Baud Rate!



Universal Asynchronous Receiver Transmitter (UART)

- Maximum transmission rate depends on
 - Baud Rate
 - The higher the baud rate, the higher the performance
 - There are f_{ck} bit times in a second
 - Overhead
 - START BIT, Parity and STOP BITS do not carry information
 - Minimum overhead: 2 bits (START BIT, 1 STOP BIT)
 - Maximum overhead: 4 bits (START BIT, Parity, 2 STOP BITS)
- Bit Rate
 - $$\text{BitRate} = \frac{n}{n+oh} f_{ck}$$
 - The higher is n , the lower is the effect of the overhead
 - Example: 115200-8N1 (Baud Rate = 115200, $n = 8$, $oh = 2$)
 - BitRate = 92160 bps

Universal Asynchronous Receiver Transmitter (UART)

- Increasing f_{ck} and n may lead to high performance
- But there are limits that cannot be exceeded
 - Sampling is performed within a certain window where data is stable
 - Window is between two active clock edges, where data potentially changes
 - Window defined by a set of parameters
 - Voltage levels (V_{OL} , V_{OH} , V_{IL} , V_{IH})
 - Noise (NM_L , NM_H)
 - Timing (setup t_{su} , hold t_h , rising t_r and falling t_f , slew rate, bandwidth)
 - Window = EYE DIAGRAM
 - Increasing the clock frequency makes the window smaller
 - Clock drifting moves the sampling point outside the window after a certain number of bits without synchronization
 - Once windows parameters are set, maximum n and f_{ck} can be computed

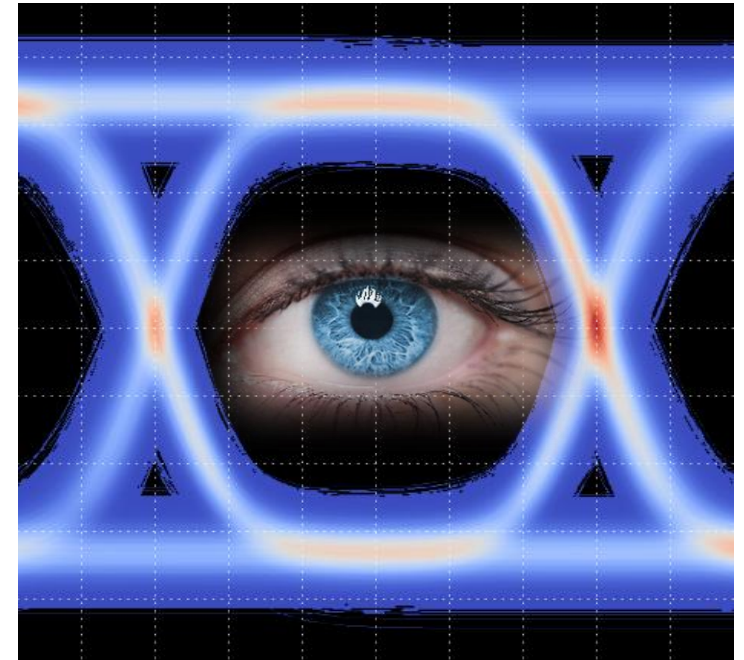
Universal Asynchronous Receiver Transmitter (UART)

- UART Performance
 - Long interconnections
 - Performance limited by attenuation, distortion, noise
 - Affect the vertical axis in the eye diagram (see later)
 - Performance limited by unpredictable delays
 - Affects the horizontal axis in the eye diagram (see later)
 - Improved by large voltage swings
 - Example: RS232
 - Typical values for RS232 voltages in PC are -13V and +13V
 - Short interconnections
 - Performance limited by clock jitter and tolerances
 - Affect the horizontal axis in the eye diagram (see later)
 - Interconnection length and speed are conflicting parameters
 - Short and fast link
 - Long and slow link

Universal Asynchronous Receiver Transmitter (UART)

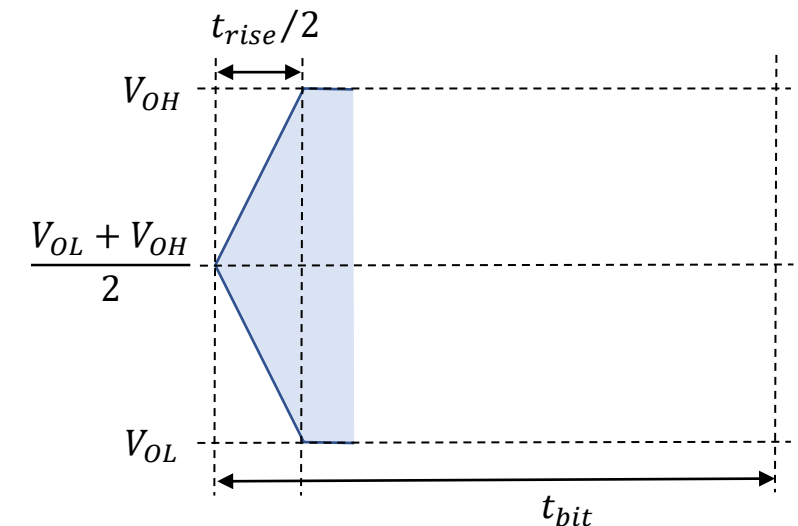
- Eye Diagram

- Defines the sampling window
- Signal waveforms at the receiver
- Parameters
 - Bit time $t_{bit} = 1/f_{ck}$
 - Output voltage levels V_{OL} and V_{OH}
 - Input thresholds V_{IL} and V_{IH}
 - Noise margin NM_L and NM_H
 - Transmitter rising and falling time t_{rise} and t_{fall}
 - Channel bandwidth BW and slew rate SR
 - Receiving Flip Flop setup and hold time t_{su} and t_h
 - Built considering all possible transitions between transmitted bits
 - Worst cases are:
 - Low-to-High followed by High-to-Low (i.e. 000100)
 - High-to-Low followed by Low-to-High (i.e. 111011)
 - Show all transitions superimposed on the same graph



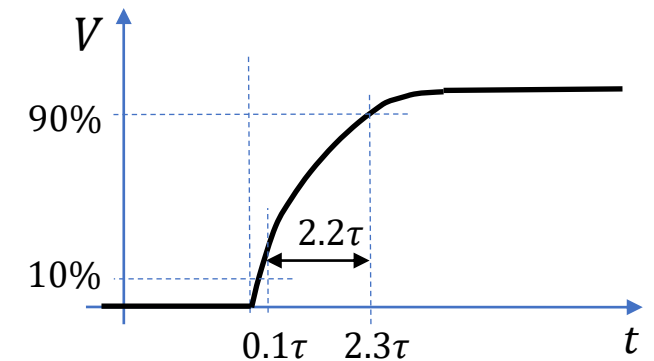
Universal Asynchronous Receiver Transmitter (UART)

- Drawing the eye diagram
 - Consider a time interval equal to t_{bit} and a voltage range between V_{OL} and V_{OH}
 - Out of the typical range 0 V to V_{cc}
 - Assume for simplicity that $t_{rise} = t_{fall}$
 - Draw rising and falling edges at the receiver, at the beginning of the bit
 - Start at $(V_{OL} + V_{OH})/2$ and go towards V_{OL} and V_{OH} respectively
 - Edges last $t_{rise}/2$
 - Channel bandwidth BW may induce higher t_{rise} and t_{fall} compared to what the transmitter produces
 - Rule of thumb
 - $t_{rise} = 0.35/BW$
 - BW in GHz and t_{rise} in ns (same for t_{fall})
 - For large signals the channel slew rate SR is a better parameter
 - $t_{rise} = \frac{V_{OH} - V_{OL}}{SR}$



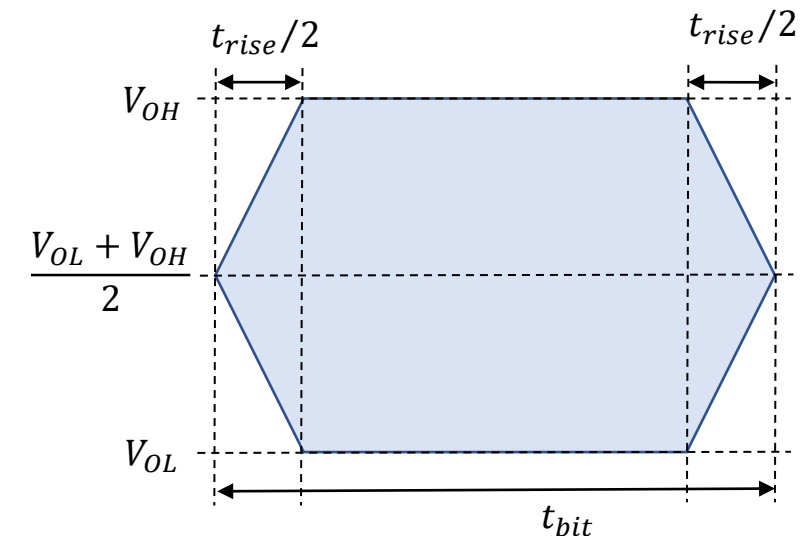
Universal Asynchronous Receiver Transmitter (UART)

- Drawing the eye diagram
 - Rising and falling time from the channel Bandwidth
 - The channel Bandwidth BW defines the cut-off frequency f_c
 - In a single pole filter, the cut-off frequency is given by
 - $BW = f_c = \frac{\omega_c}{2\pi} = \frac{1}{2\pi RC}$ with $\omega_c = \frac{1}{RC}$
 - The time constant in a filter is $\tau = RC = 1/\omega_c$
 - Therefore
 - $\tau = \frac{1}{2\pi BW}$
 - The rising and/or falling times are approximately 2.2 times the time constant
 - This is to go from 10% to 90% of the total voltage dynamic range
 - $1 - e^{-0.1} = 0.1$
 - $1 - e^{-2.3} = 0.9$
 - Therefore
 - $t_{rise} = 2.2\tau = \frac{2.2}{2\pi BW} = \frac{0.35}{BW}$



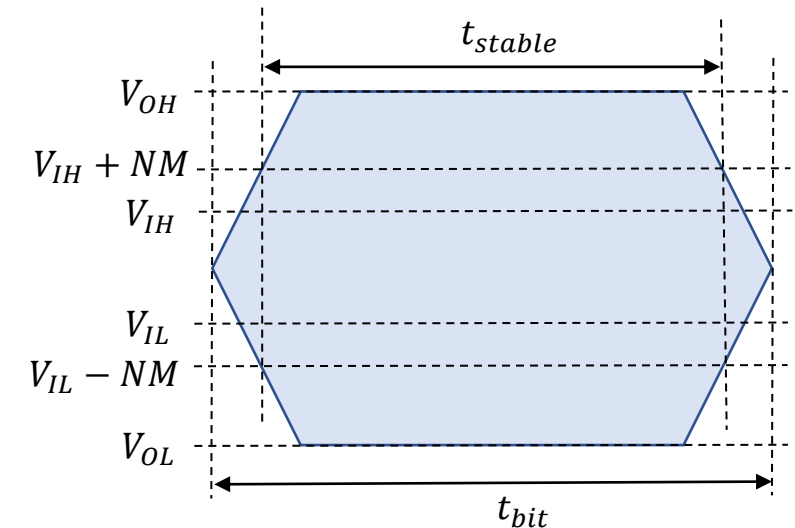
Universal Asynchronous Receiver Transmitter (UART)

- Drawing the eye diagram (continued)
 - Consider a time interval equal to t_{bit} and a voltage range between V_{OL} and V_{OH}
 - Out of the typical range 0 V to V_{cc}
 - Assume for simplicity that $t_{rise} = t_{fall}$
 - Draw rising and falling edges at the receiver, at the end of the bit
 - Symmetrical to the beginning of the bit
 - Start from V_{OL} and V_{OH} and go towards $(V_{OL} + V_{OH})/2$
 - Edges last $t_{rise}/2$
 - Connect edges with straight lines at V_{OL} and V_{OH}
 - You get a hexagon



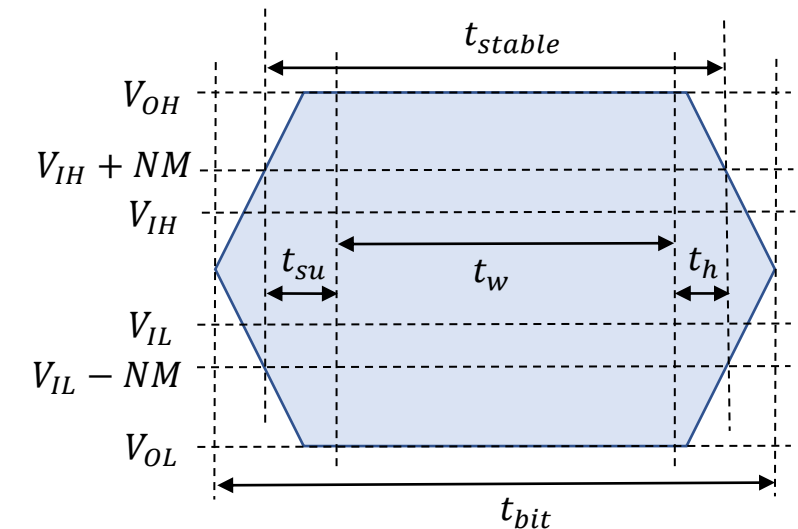
Universal Asynchronous Receiver Transmitter (UART)

- Drawing the eye diagram (continued)
 - Identify the voltage levels V_{IL} and V_{IH} in the graph
 - Subtract the noise margin NM to V_{IL} and add it to V_{IH}
 - For simplicity, same margin at the low and high levels
 - Mark these two new voltage levels on the graph with horizontal lines
 - Identify the time interval t_{stable} (a subset of t_{bit})
 - The graph is higher than $V_{IH} + NM$ and lower than $V_{IL} - NM$
 - This is the time window where the signal is guaranteed to be stable at the receiver
 - Receivers recognize high and low levels
 - Even in the presence of noise
 - Easier to identify when the graph is symmetric
 - Otherwise always consider the worst case at the beginning and at the end of the window



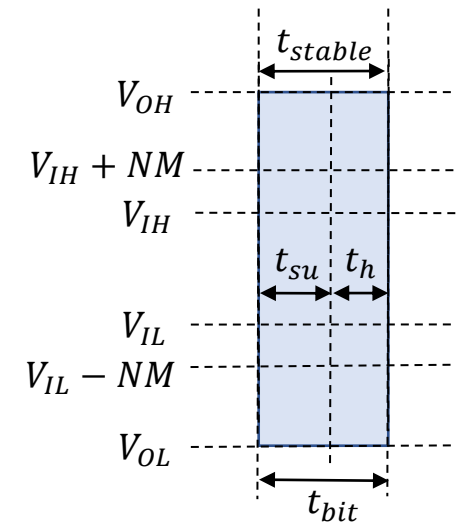
Universal Asynchronous Receiver Transmitter (UART)

- Drawing the eye diagram (continued)
 - Reduce the size of the stable window by the setup time t_{su} at the beginning
 - Reduce the size of the stable window by the hold time t_h at the end
 - The new resulting window is the *sampling window*, with duration t_w
 - Sampling inside t_w is always correct
 - The signal is stable
 - The signal has proper voltage levels
 - Setup and hold time are satisfied
 - It is always $t_w < t_{stable} < t_{bit}$
 - Immediately after synchronization, the receiver determines the best sampling instant
 - We assume it is at the center of the sampling window



Universal Asynchronous Receiver Transmitter (UART)

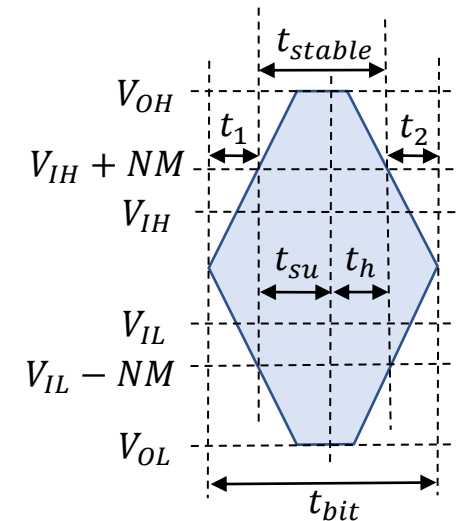
- Eye Diagram
 - Perfect isofrequential clocks
 - Synchronize clocks (START BIT)
 - Sampling happens within the sampling window for all successive bits
 - There is no limit to the number n of consecutive bits
 - There is a limit to the maximum clock frequency
 - Increasing the clock frequency makes t_w smaller
 - When $t_w = 0$, no correct sampling is possible
 - The eye closes
 - With ideal rising and falling edges, it is
 - $f_{ck} < \frac{1}{t_{su} + t_h}$, $t_{bit} > t_{su} + t_h$
 - In real cases the maximum frequency is much less
 - The maximum clock frequency limits the maximum performance



Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram

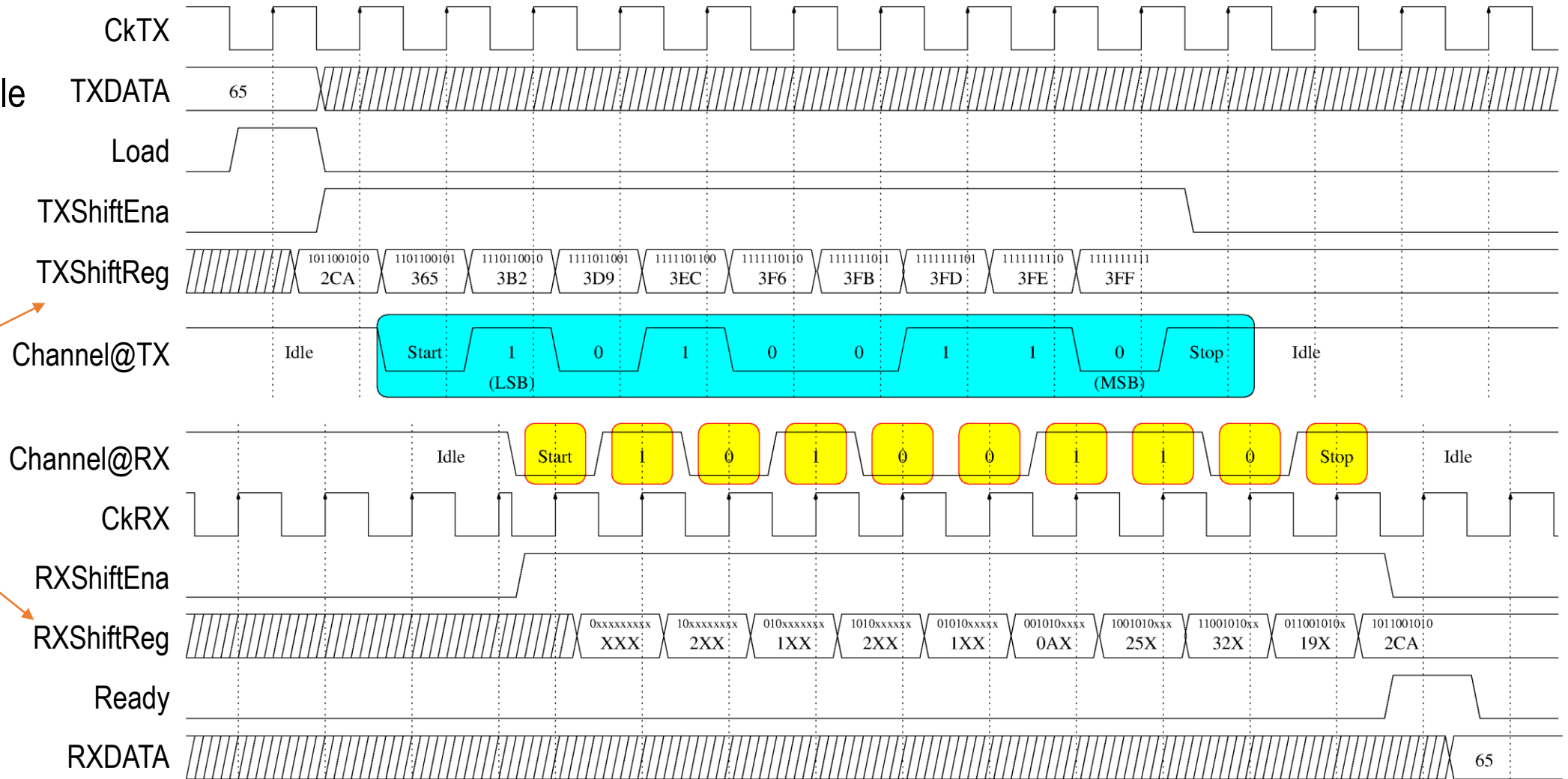
- Perfect isofrequential clocks
 - Synchronize clocks (START BIT)
 - Sampling happens within the sampling window for all successive bits
 - There is no limit to the number n of consecutive bits
- There is a limit to the maximum clock frequency
 - Increasing the clock frequency makes t_w smaller
 - When $t_w = 0$, no correct sampling is possible
 - The eye closes
 - With real rising and falling edges, it is
 - $t_1 = \left(\frac{V_{IH}+NM}{V_{OH}-V_{OL}} - \frac{1}{2} \frac{V_{OH}+V_{OL}}{V_{OH}-V_{OL}} \right) t_{rise}$, $t_2 = t_1$
 - $f_{ck} < \frac{1}{t_{su}+t_h+2t_1}$, $t_{bit} > t_{su} + t_h + 2t_1$
- The maximum clock frequency limits the maximum performance



Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram
- UART Example
- Perfect isofrequential clocks

The ShiftRegs also contain the START BIT and the STOP BIT to be transferred on the UART line; the DATA is in between them



Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram
 - Nominally isofrequentual clocks with tolerances
 - Relative error ε (in percentage)
 - $f_{ck} = f_{nom} \pm \varepsilon\%$, $t_{bit} = t_{nom} \pm \varepsilon\%$, $t_{nom} = 1/f_{nom}$
 - The clock at the transmitter and the clock at the receiver may have different t_{bit}
 - Because of this difference, the sampling point from one bit to the next drifts by an amount
 - $\Delta t = t_{bit}^{RX} - t_{bit}^{TX}$
 - In the worst case approximately $\Delta t = t_{nom} \cdot 2\varepsilon$
 - Considering equal tolerances at transmitter and receiver, but in opposite directions
 - Example: $f_{ck} = 10 \text{ MHz} \pm 2\%$, $t_{nom} = 100 \text{ ns}$, $\Delta t = 4 \text{ ns}$
 - Drift may cause sampling outside the *sampling window* after a certain number of consecutive bits without re-synchronization

Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram

- Nominally isofrequential clocks with tolerances (continued)

- Example:

- $f_{ck} = 57600 \pm 2\%$
- $t_{ck} = \frac{1}{57600} \pm 2\% = 17.36\mu s \pm 2\% = 17.36\mu s \pm 347ns$
- $\delta_t = 347ns, \Delta t = 2\delta_t = 694ns \cong 700ns$
- In the worst case, after $17360/694 = 25$ clock cycles, the two clocks differ by a whole period
 - However, communication fails much earlier than that!

- Detailed derivation of time uncertainty from frequency uncertainty

- Clock frequency and tolerance: $f_{ck} = f_{nom} \pm \delta_f$
- Clock period and tolerance: $t_{ck} = t_{nom} \pm \delta_t$
- Relative uncertainties (can be expressed in percentage): $\varepsilon_f = \frac{\delta_f}{f_{nom}}, \varepsilon_t = \frac{\delta_t}{t_{nom}}$
- $t_{nom} = \frac{1}{f_{nom}} \Rightarrow \varepsilon_t = \varepsilon_f$ in fact: $\varepsilon_t = \frac{\delta_t}{t_{nom}} = \frac{\left|\frac{\partial t_{nom}}{\partial f_{nom}}\right| \delta_f}{t_{nom}} = \frac{1}{t_{nom}} \frac{1}{f_{nom}^2} \delta_f = f_{nom} \frac{1}{f_{nom}^2} \delta_f = \frac{\delta_f}{f_{nom}} = \varepsilon_f$

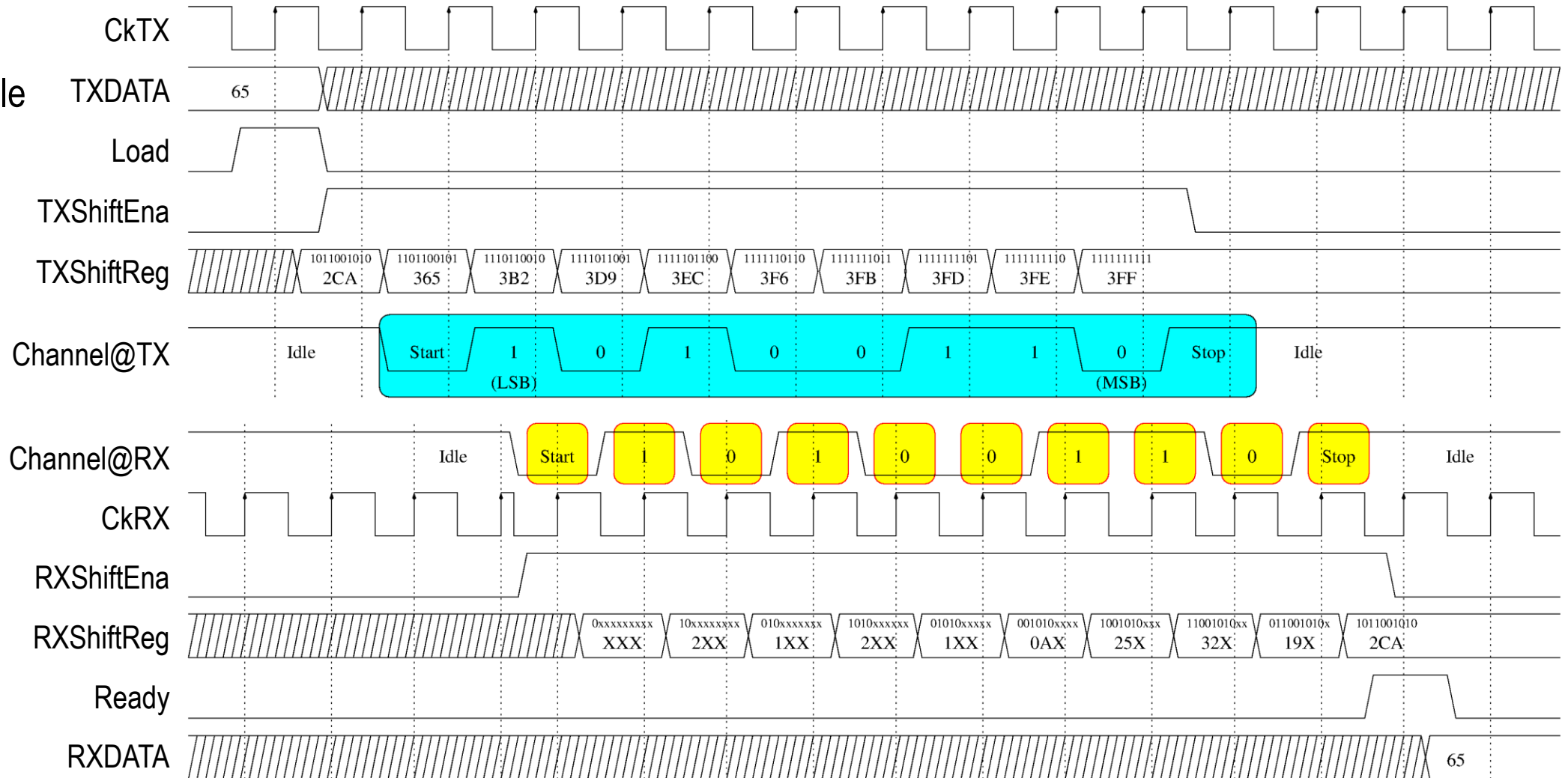
Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram

- Nominally isofrequential clocks with tolerances (continued)
 - Assume sampling immediately after synchronization happens exactly in the middle of the sampling window
 - This may not correspond to the middle of the bit time
 - It is the best case, not always easy to determine
 - Drift may be positive or negative
 - Assume it is negative, i.e. sampling earlier at each new bit
 - Only half of the sampling window is used (the left half)
 - The maximum number of bits that can be correctly sampled is
 - $n_{max} = \frac{t_w/2}{\Delta t}$
 - It must be $n \leq n_{max}$
 - An upper bound on n limits the performance, due to the overhead
- The upper bound on the clock frequency is also present

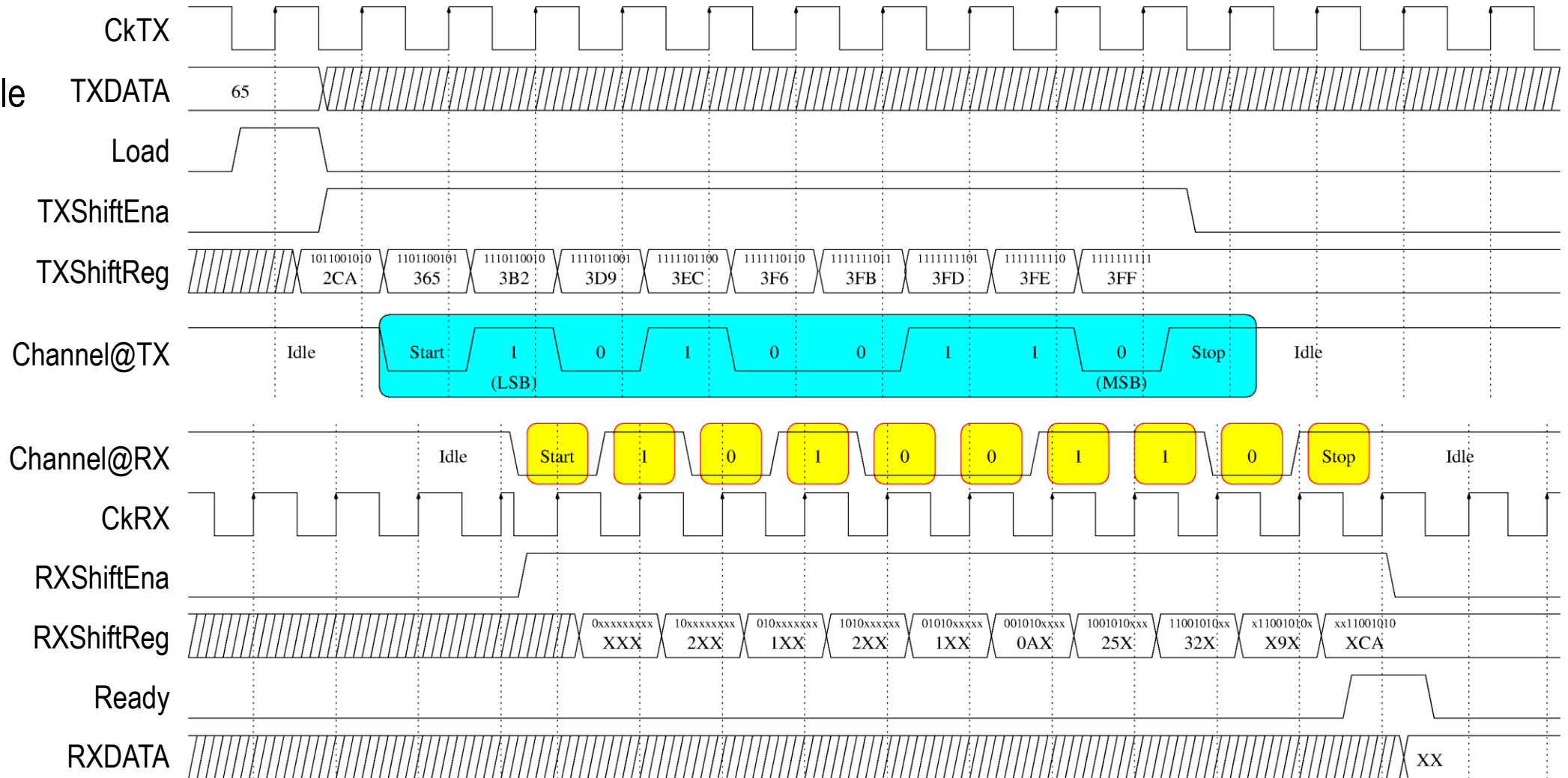
Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram
- UART Example
- Nominally isofrequentual clocks with tolerances
- Correct Transmission



Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram
- UART Example
- Nominally isofrequentual clocks with tolerances
- Incorrect Transmission



Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram
 - There is one degree of freedom
 - Determine (estimate or measure) all parameters
 - Keep the clock frequency f_{ck} and the number n of consecutive bits unknown
 - Fix the clock frequency and its tolerance, compute the maximum number of consecutive bits
 - t_{bit} is known and equal to t_{nom} when drawing the eye diagram
 - Easy, follow the eye diagram drawing explained before
 - Fix the number of consecutive bits and the clock tolerance, compute the maximum clock frequency
 - Reverse the process, leaving t_{bit} as unknown when drawing the eye diagram
 - Assume the eye does not close when drawing
 - Or consider all possible combinations to find the one that leads to best performance
 - Find the best Bit Rate
 - Sometimes the unknown is one of the other parameters

Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram
 - Other sources of error in the eye diagram
 - Clock jitter
 - Correct frequency, but edge instant may vary from one cycle to another
 - May be caused by variable delays
 - Discrete values for clock generators
 - Can only change the frequency in steps
 - Intersymbolic interference
 - Eye diagram used for many other communication protocols
 - Given in the specifications
 - Drivers and receivers should satisfy the specifications on voltage levels and timing
 - Determines how many consecutive bits can be transmitted before synchronization
 - See USB 2.0 specifications

Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram example

- Parameters

- $V_{OH} = 3.3\text{ V}, V_{OL} = -3.3\text{ V}, V_{IH} = 2\text{ V}, V_{IL} = -2\text{ V}$
 - $SR = 1\text{ V}/\mu\text{s}, N = 0.5\text{ V}$
 - $f_{ck} = 115.2\text{ kHz}, \varepsilon = 1\%$
 - $t_{su} = 200\text{ ns}, t_h = 150\text{ ns}$

- Bit time

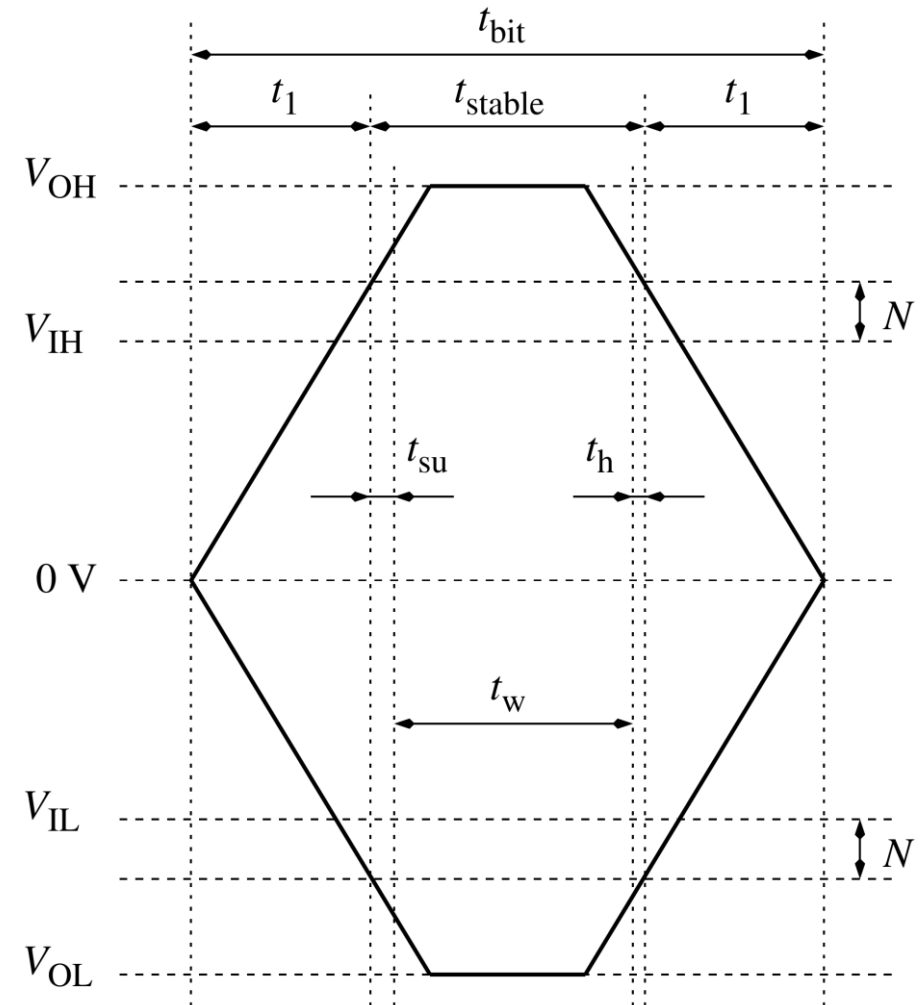
- $t_{bit} = \frac{1}{f_{ck}} = \frac{1}{115200} = 8.68\text{ }\mu\text{s}$

- Rising to $V_{IH} + N$ (or falling to $V_{IL} - N$)

- $t_1 = \frac{V_{IH} + N - 0}{SR} = \frac{2 + 0.5}{10^6} = 2.5\text{ }\mu\text{s}$

- Stable window

- $t_{stable} = t_{bit} - 2t_1 = 3.68\text{ }\mu\text{s}$



Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram example

- Parameters

- $V_{OH} = 3.3\text{ V}, V_{OL} = -3.3\text{ V}, V_{IH} = 2\text{ V}, V_{IL} = -2\text{ V}$
 - $SR = 1\text{ V}/\mu\text{s}, N = 0.5\text{ V}$
 - $f_{ck} = 115.2\text{ kHz}, \varepsilon = 1\%$
 - $t_{su} = 200\text{ ns}, t_h = 150\text{ ns}$

- Sampling window

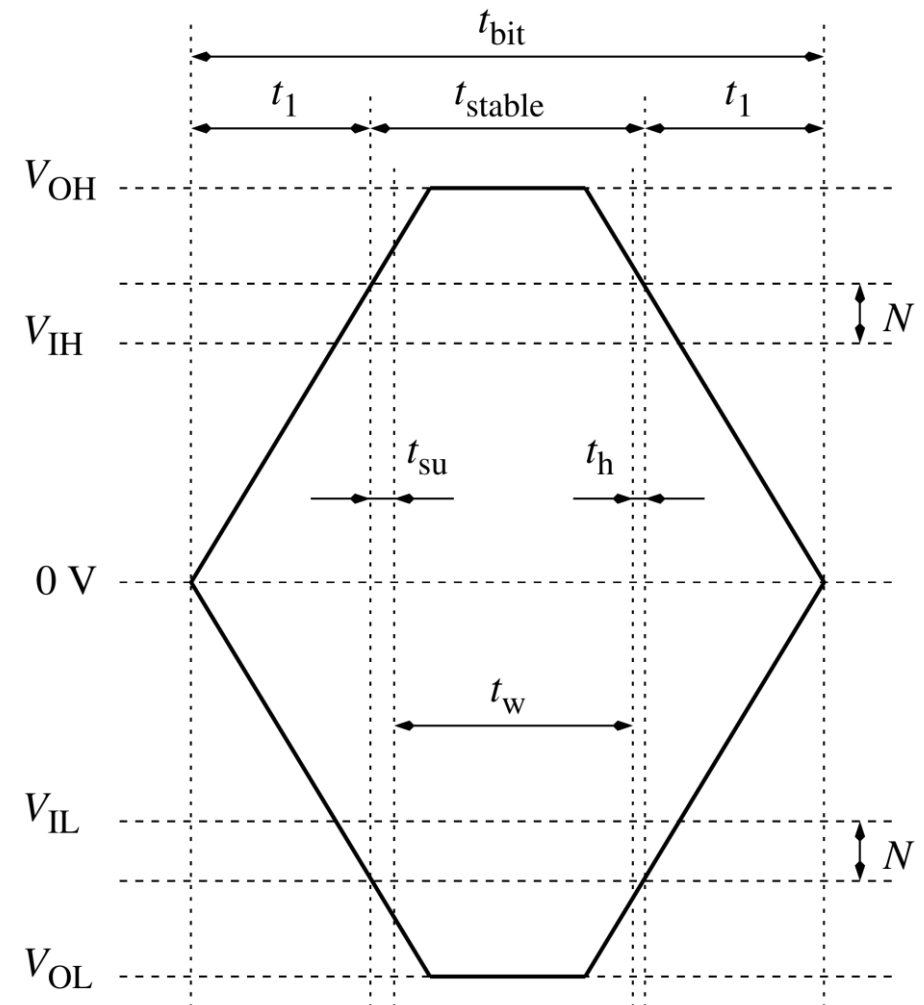
- $t_w = t_{stable} - t_{su} - t_h = 3.33\text{ }\mu\text{s}$

- Drift of a clock cycle

- $\Delta t_{bit} = \frac{t_{bit}}{100} = 86.8\text{ ns}$

- Maximum consecutive bits

- $n_{maxbits} = \frac{t_w/2}{\Delta t_{bit}} = 19$



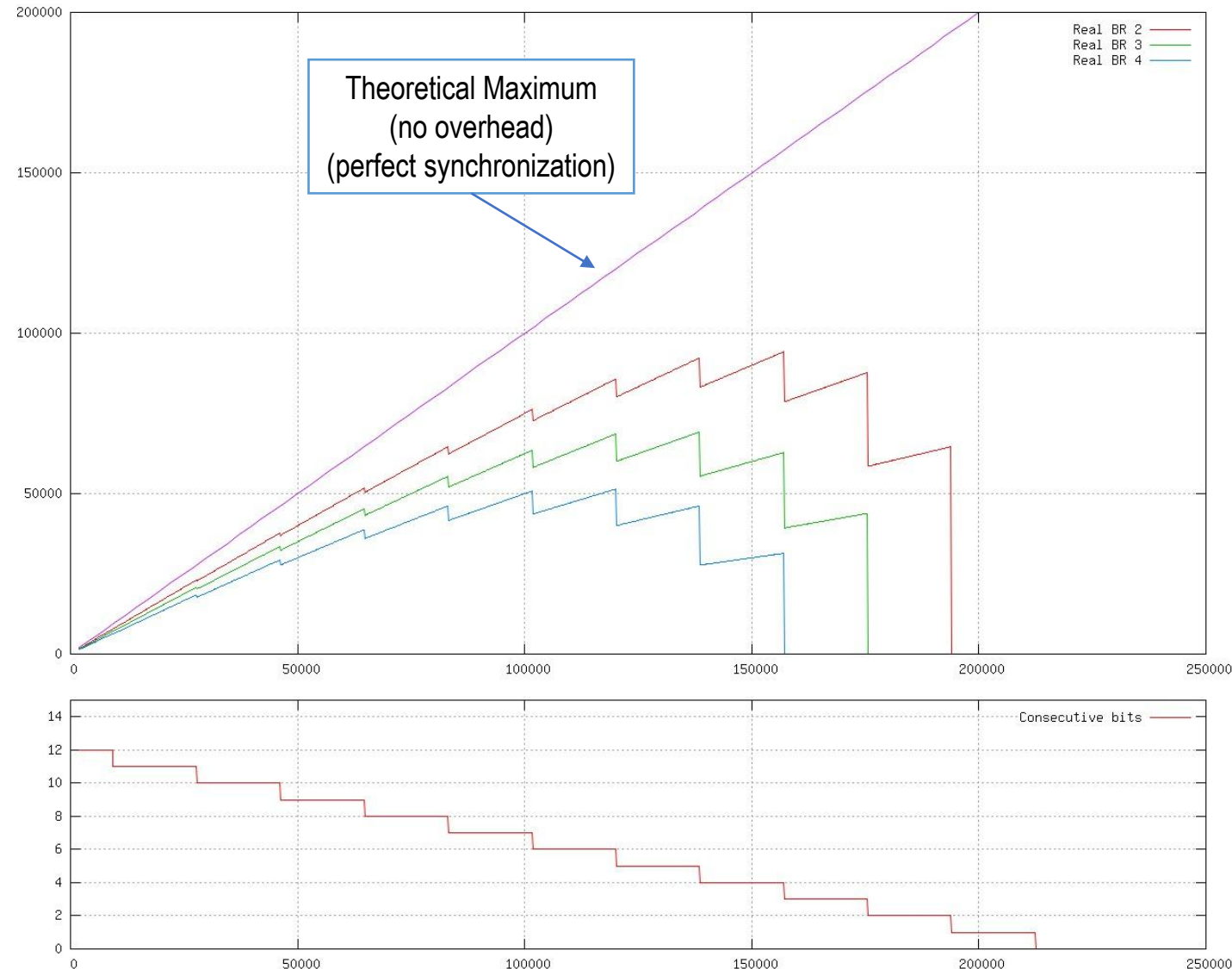
Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram
 - Eye diagram computation
 - Spreadsheet example of how to compute various parameters
 - Value on the left
 - Set by the user
 - Values on the right
 - Computed
 - See expressions

	A	B	C	D	E	F	G	H	I	J
1	Sampling window size and Bit Rate									
2	Parameter	Unit	Value			Parameter	Expression	Unit	Value	
3	Frequency (f_{ck})	kHz	57.600			Period	$1000000/C3$	ns	17361.1	
4	Tolerance (ϵ)	%	2.0			δt	$I3*C4/100$	ns	347.2	
5	Slew Rate (SR)	V/us	1.0			Rise and Fall time	$(C6-C7)/C5*1000$	ns	5000.0	
6	V_{OH}	V	5.00			Time-to-H	$((C8+C10/1000)-(C6-C7)/2)/C5*1000$	ns	1250.0	
7	V_{OL}	V	0.00			Time-to-L	$((C6-C7)/2-(C9-C11/1000))/C5*1000$	ns	2100.0	
8	V_{IH}	V	3.15			MaxSlewTime	$MAX(I6:I7)$	ns	2100.0	
9	V_{IL}	V	0.80			SamplingWindow	$I3-2*I8$	ns	13161.1	
10	NM_H	mV	600			CorrectedSW	$I9-C12-C13$	ns	13031.1	
11	NM_L	mV	400			nbit	$I10/2/(2*I4), INT(I11)$		9.4	9
12	t_{SU}	ns	100.0			RealBR (2 overh.)	$C3*J11/(J11+2)$	kbit/s	47.1	
13	t_H	ns	30.0			RealBR (3 overh.)	$C3*J11/(J11+3)$	kbit/s	43.2	
14						RealBR (4 overh.)	$C3*J11/(J11+4)$	kbit/s	39.9	

Universal Asynchronous Receiver Transmitter (UART)

- Eye Diagram
 - UART Transmission performance
 - Same parameters as previous slide
 - Different clock frequencies
 - From $f = 2 \text{ kHz}$
 - To $f = 220 \text{ kHz}$
 - Number of consecutive bits decreases as frequency increases
 - Effective Baud Rate
 - With 2, 3 or 4 overhead bits
 - Start Bit
 - Parity
 - 1 or 2 Stop Bits
 - It shows a maximum, and then quickly drops to zero



Synchronous Protocols

- Character oriented protocols
 - Message of $k \cdot n$ bits
 - Character synchronization by special bit sequence
- Bit oriented protocols
 - Any length message
 - Start of message identified by special bit sequences or special symbols
 - How to get character/message synchronization?
 - Encoding violations
 - Known sequences
- Clock recovery
 - Use transitions of the data signal
 - In each bit (synchronous carrier)
 - Periodically

Synchronous Protocols

- Channel bandwidth requirement
 - Highest fundamental frequency in the transmitted signal
 - The lower, the better, given a certain Bit Rate (f_{ck})
 - Cheaper channel
 - Find the worst case depending on the bits transmitted
- Number of voltage levels used in the channel
 - Standard logic uses 2
 - 0 V and V_{dd}

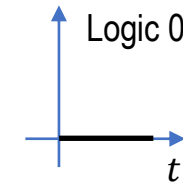
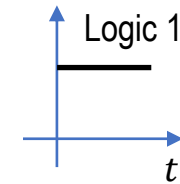
Encoding	Transitions	Bandwidth	Voltage Levels
...			
...			
...			
...			

Synchronous Protocols

- Non Return to Zero Encoding (NRZ)

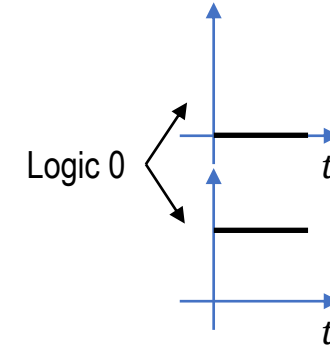
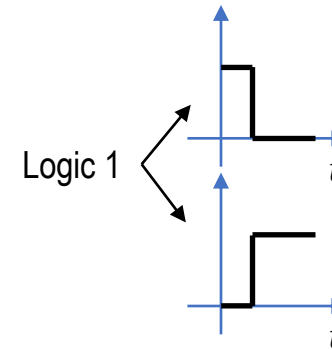
- NRZ-L (Level)

- Logic 1: high state
 - Logic 0: low state



- NRZ-M (Mark, also known as NRZ-I, Inverted)

- Logic 1: represented by a transition
 - Logic 0: no change



- NRZ-S (Space, also known as NRZ-I, Inverted)

- Logic 1: no change
 - Logic 0: represented by a transition

- Transitions

- Not guaranteed

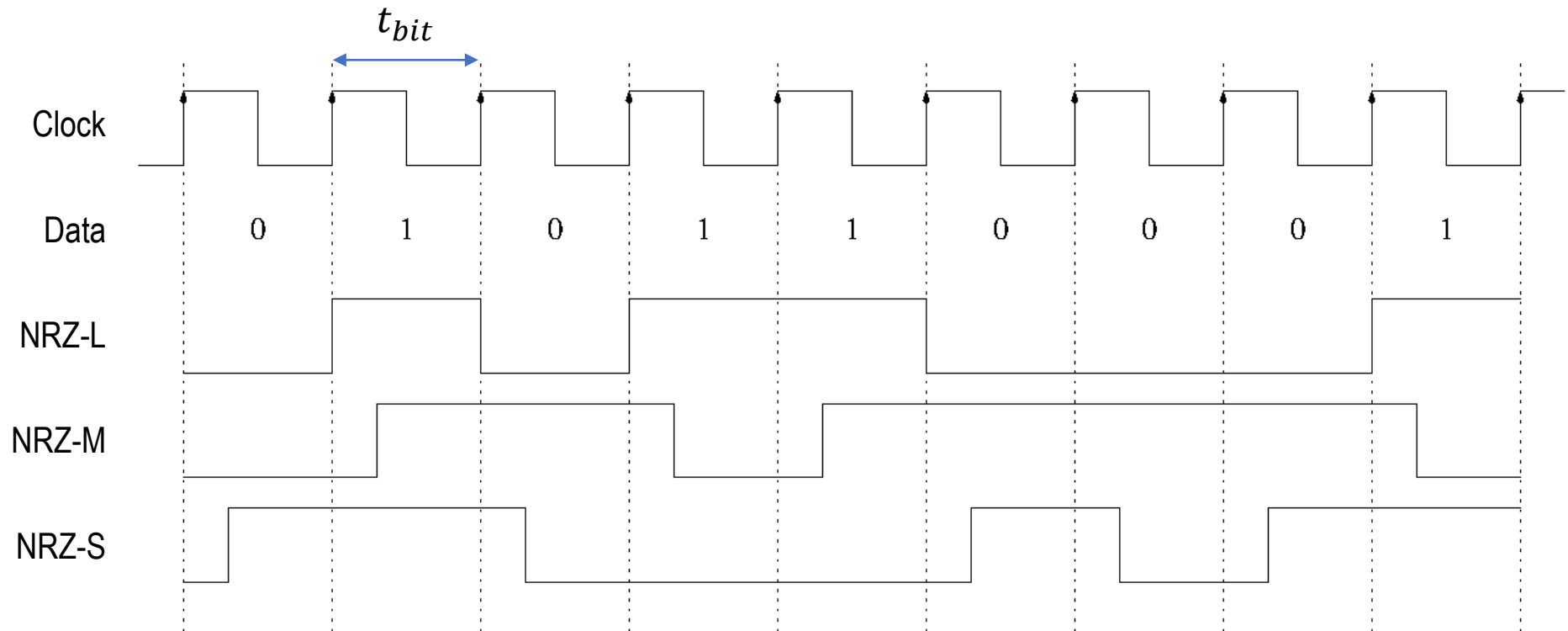
- Bandwidth

- $f_{max} = f_{ck}/2$

Encoding	Transitions	Bandwidth	Voltage Levels
NRZ	✗	$f_{ck}/2$	2

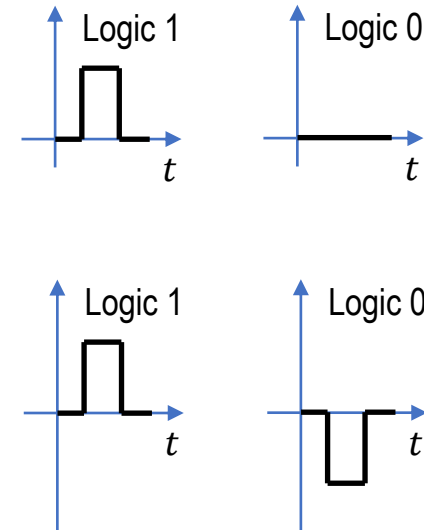
Synchronous Protocols

- Non Return to Zero Encoding (NRZ)
 - Examples



Synchronous Protocols

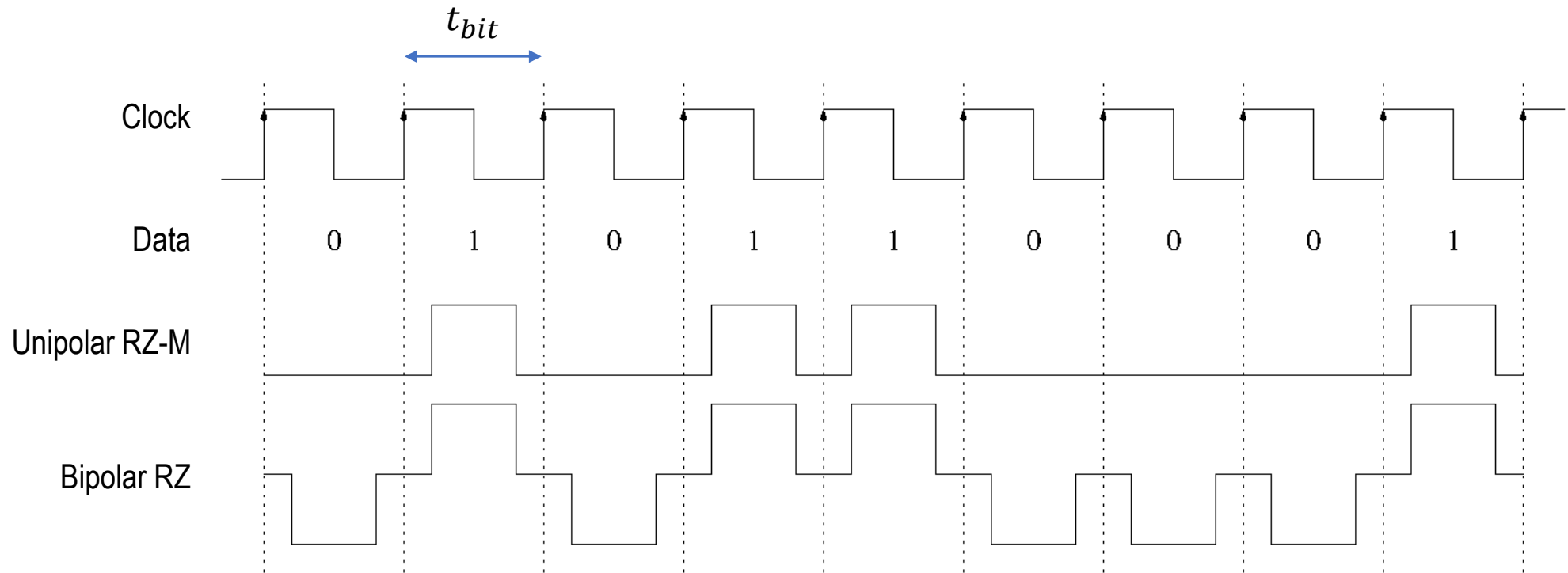
- Return to Zero Encoding (RZ)
 - Unipolar RZ-M
 - Logic 1: high pulse
 - Logic 0: low state
 - Bipolar RZ
 - Logic 1: positive pulse
 - Logic 0: negative pulse
 - Ternary code needs better SNR
 - Transitions
 - Not guaranteed in unipolar
 - Guaranteed in bipolar
 - Bandwidth
 - $f_{max} = f_{ck}$



Encoding	Transitions	Bandwidth	Voltage Levels
Unipolar RZ-M	✗	f_{ck}	2
Bipolar RZ	✓	f_{ck}	3

Synchronous Protocols

- Return to Zero Encoding (RZ)
 - Examples



Synchronous Protocols

- AMI RZ

- Alternate Mark Inverted – Return to Zero

- Logic 1: alternate level pulse
 - Logic 0: no pulse

- Transitions

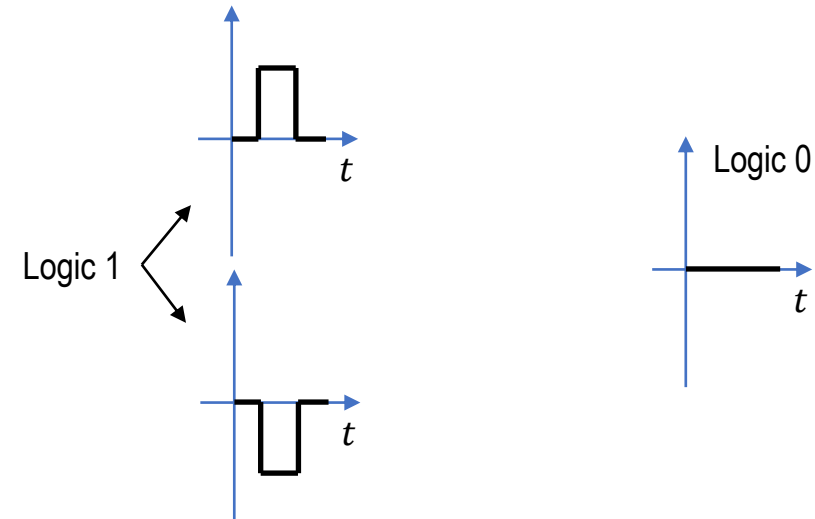
- Not guaranteed

- Bandwidth

- $f_{max} = f_{ck}/2$

- DC Component

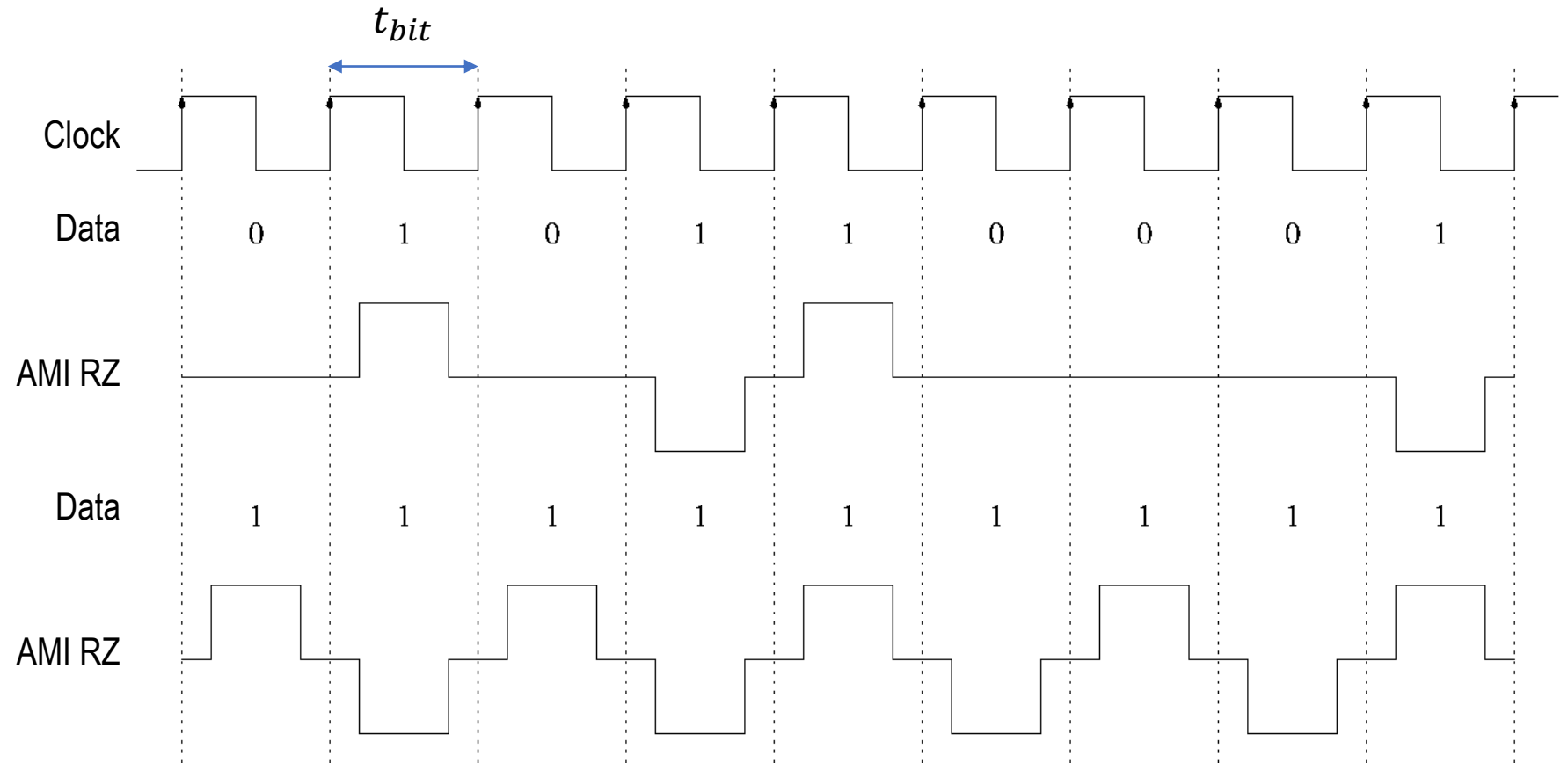
- Absent on average
 - Advantage in long connections



Encoding	Transitions	Bandwidth	Voltage Levels
AMI RZ	×	$f_{ck}/2$	3

Synchronous Protocols

- AMI RZ
 - Examples



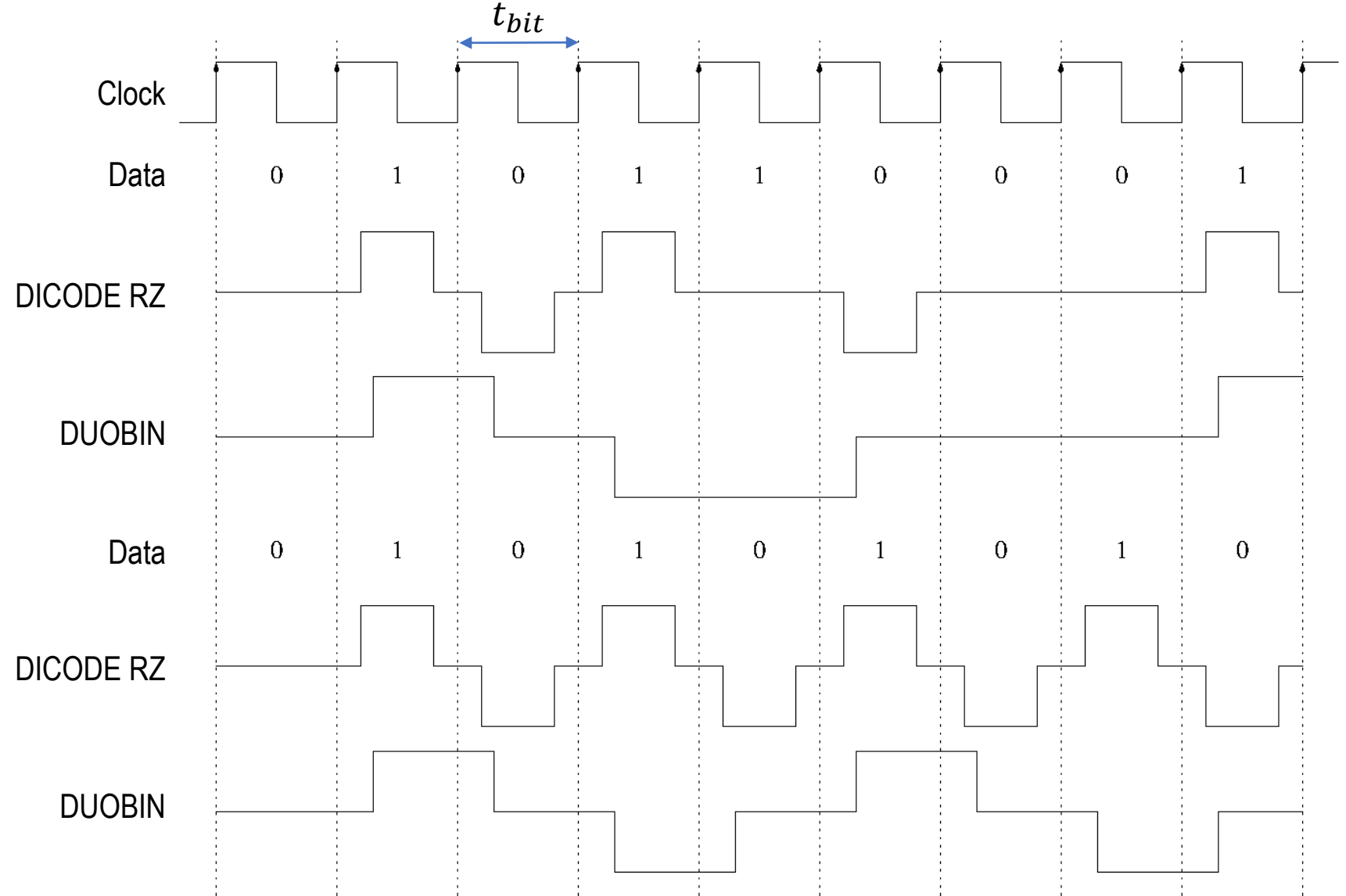
Synchronous Protocols

- DICODE RZ
 - Each input transition generates an output pulse
 - With alternate polarity
 - If there is no input transition, output remains stable
- DUOBIN (DICODE NRZ)
 - Each input transition generates an output transition
 - With alternate polarity
 - If there is no input transition, output remains stable
- Transitions
 - Not guaranteed
- Bandwidth
 - $f_{max} = f_{ck}/4$ in the best case

Encoding	Transitions	Bandwidth	Voltage Levels
DICODE RZ	✗	$f_{ck}/2$	3
DUOBIN	✗	$f_{ck}/4$	3

Synchronous Protocols

- DICODE RZ and DUOBIN
 - Examples



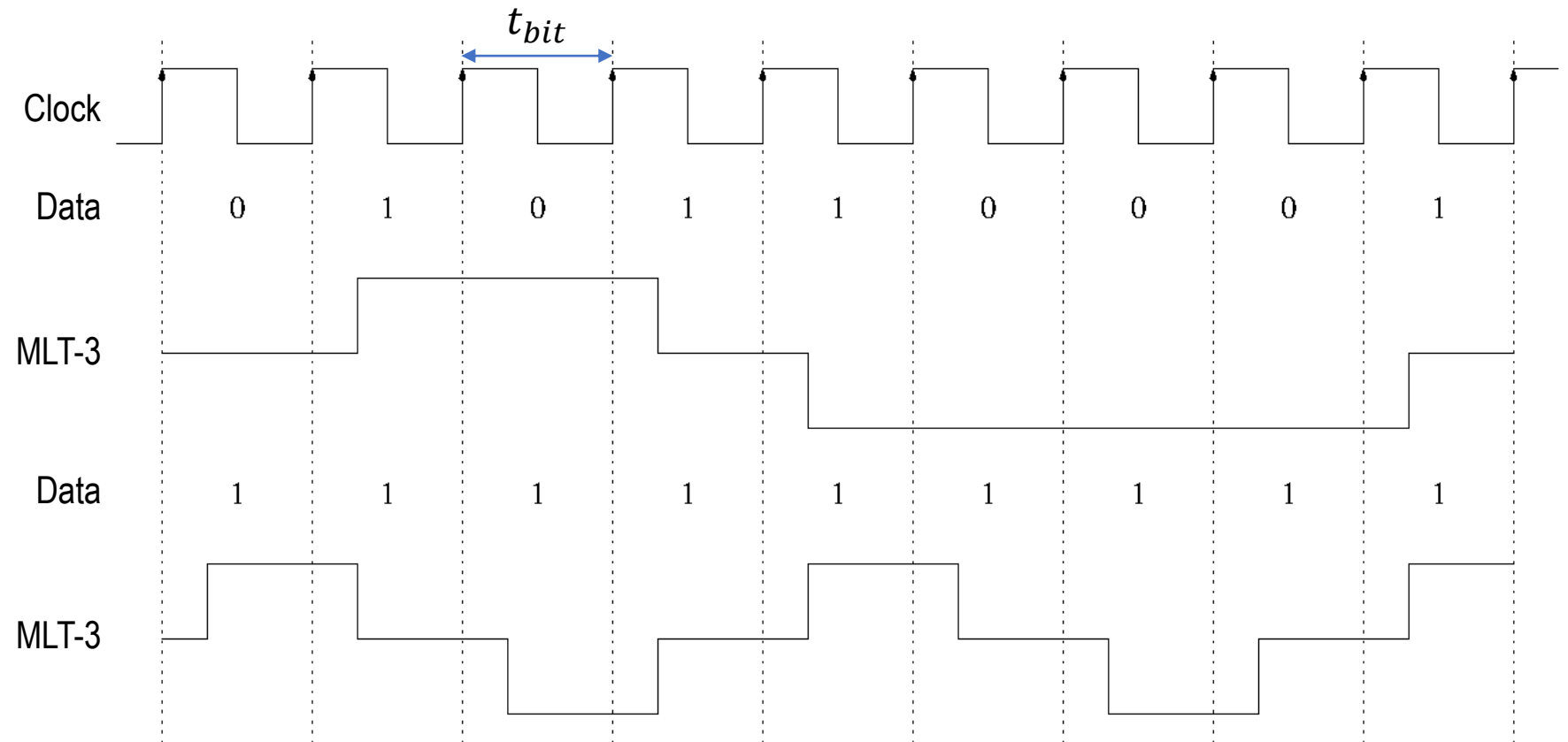
Synchronous Protocols

- MultiLevel Transmission encoding (MLT)
 - MLT-3 three levels
 - Logic 0: no change
 - Logic 1: output transition with alternate polarity
- Transitions
 - Not guaranteed
- Bandwidth
 - $f_{max} = f_{ck}/4$
- Used in Ethernet 100Base-TX

Encoding	Transitions	Bandwidth	Voltage Levels
MLT-3	☒	$f_{ck}/4$	3

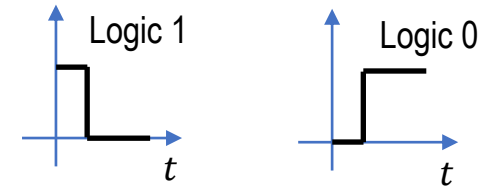
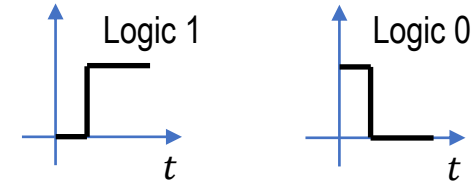
Synchronous Protocols

- MLT-3
 - Examples



Synchronous Protocols

- Manchester codes (phase modulation)
 - Also known as IEEE 802.3
 - Logic 1: low to high transition ($L \rightarrow H$)
 - Logic 0: high to low transition ($H \rightarrow L$)
 - Easily obtained starting from an NRZ-L encoding
 - Manchester = Data XOR Clock
 - Opposite encoding
 - Logic 1: high to low transition ($H \rightarrow L$)
 - Logic 0: low to high transition ($L \rightarrow H$)
- Transitions
 - Guaranteed
- Bandwidth
 - $f_{max} = f_{ck}$



Encoding	Transitions	Bandwidth	Voltage Levels
Manchester	✓	f_{ck}	2

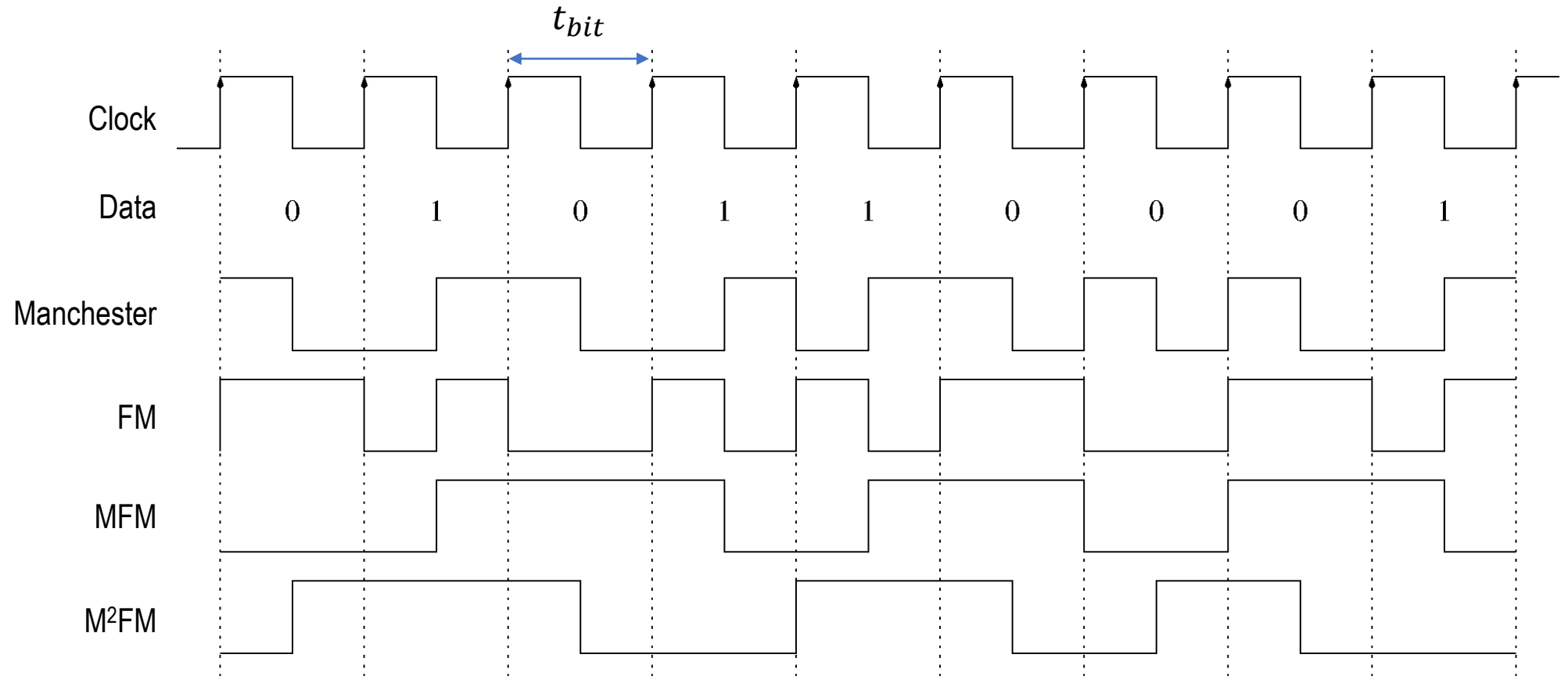
Synchronous Protocols

- Frequency Modulation (FM)
 - Also known as BiPhase (BiΦ-L) or Differential Manchester Encoding
 - Logic 1: a transition at the beginning of the bit, followed by a transition in the middle of the bit
 - Logic 0: a transition at the beginning of the bit
- Modified Frequency Modulation (MFM)
 - Logic 1: a transition in the middle of the bit
 - Logic 0: no transition if preceded by a 1, a transition at the beginning if preceded by a 0
- Modified Modified Frequency Modulation (M²FM)
 - Manchester : 2
- Transitions
 - Guaranteed
- Bandwidth
 - $f_{max} = f_{ck}/2$ in the best case

Encoding	Transitions	Bandwidth	Voltage Levels
FM (BiΦ-L)	✓	f_{ck}	2
MFM	✓	$f_{ck}/2$	2
M ² FM	✓	$f_{ck}/2$	2

Synchronous Protocols

- Manchester codes
 - Examples



Synchronous Protocols

- Summary of encodings
 - Need transitions to recover clock
 - Need low bandwidth
 - Performance
 - Costs
 - 2 voltage levels is preferable

Encoding	Transitions	Bandwidth	Voltage Levels
NRZ	✗	$f_{ck}/2$	2
Unipolar RZ-M	✗	f_{ck}	2
Bipolar RZ	✓	f_{ck}	3
AMI RZ	✗	$f_{ck}/2$	3
DICODE RZ	✗	$f_{ck}/2$	3
DUOBIN	✗	$f_{ck}/4$	3
MLT-3	✗	$f_{ck}/4$	3
Manchester	✓	f_{ck}	2
FM (BiΦ-L)	✓	f_{ck}	2
MFM	✓	$f_{ck}/2$	2
M ² FM	✓	$f_{ck}/2$	2

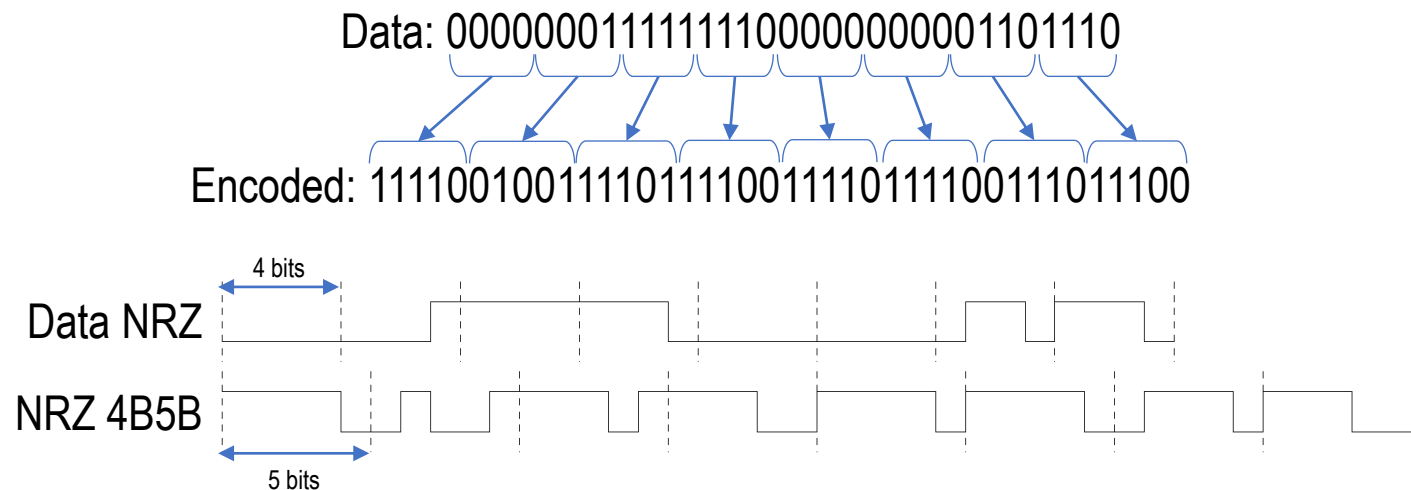
Synchronous Protocols

- Forcing transitions
 - Some encodings show excellent characteristics
 - Simplicity, low bandwidth, two voltage levels
 - ... but do not guarantee transitions
 - Cannot be used *as is* for clock extraction
 - $xByB$ encoding
 - x -bit block replaced by a y -bit block, with $y > x$
 - Presence of transitions
 - Deterministic, imposes a fixed overhead
- Bit stuffing
 - After a certain number of consecutive bits with no transitions
 - Insert a dummy bit that causes a transition
 - Non deterministic, data dependent

Synchronous Protocols

- $xByB$
 - Examples
 - 4B5B
 - 8B10B
 - 64B66B
 - Redundancy used for
 - Transition insertions
 - Not at every bit
 - Guaranteed every n bits
 - DC and low frequencies removal
 - Control characters
 - Can use NRZ, MLT-3 (Ethernet), ...
 - Increased overhead
 - Lower performance

4B	5B	4B	5B
0000	11110	1000	10010
0001	01001	1001	10011
0010	10100	1010	10110
0011	10101	1011	10111
0100	01010	1100	11010
0101	01011	1101	11011
0110	01110	1110	11100
0111	01111	1111	11101



Synchronous Protocols

- 8B10B low frequencies removal
 - Input byte (8 bits) divided into
 - Lower 5 bits, encoded using 5B6B
 - Upper 3 bits, encoded using 3B4B
 - When two codes are present in the tables
 - Alternate to keep long run equalization of 0s and 1s
 - Single codes already have the same number of 0s and 1s
 - Output 10 bits sent LSB to MSB
 - Example

• Input: 0x26, 0x94, 0x08, 0xD8 (00100110, 10010100, 00001000, 11011000)

- There are 21 zeros and 11 ones

• Output: 0110011001, 0010111101, 1110010100, 0011000110

- 20 zeros and 20 ones

First column First column Second column Second column

5B	6B (LSB first)	
00000	100111	011000
00001	011101	100010
00010	101101	010010
00011	110001	
00100	110101	001010
00101	101001	
00110	011001	
00111	111000	000111
01000	111001	000110
01001	100101	
01010	010101	
01011	110100	
01100	001101	
01101	101100	
01110	011100	
01111	010111	101000

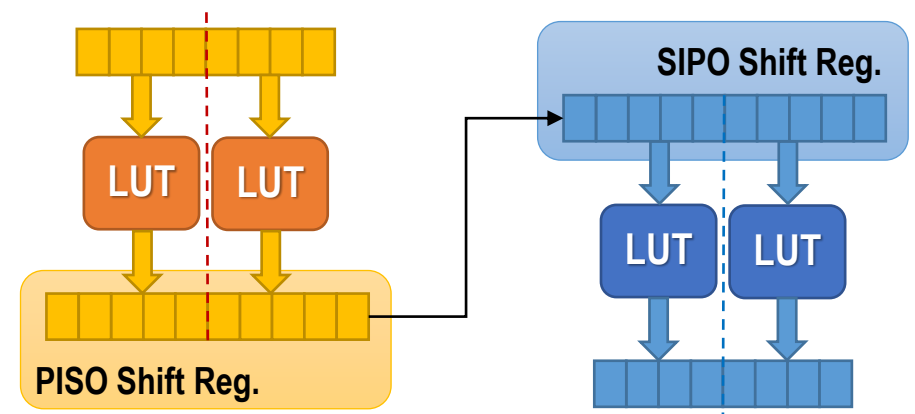
5B	6B (LSB first)	
10000	011011	100100
10001	100011	
10010	010011	
10011	110010	
10100	001011	
10101	101010	
10110	011010	
10111	111010	000101
11000	110011	001100
11001	100110	
11010	010110	
11011	110110	001001
11100	001110	
11101	101110	010001
11110	011110	100001
11111	101011	010100

3B	4B (LSB first)	
000	1011	0100
001	1001	
010	0101	
011	1100	0011
100	1101	0010
101	1010	
110	0110	
111	1110	0001

Same number of 0s and 1s reduces the low frequencies in the long run

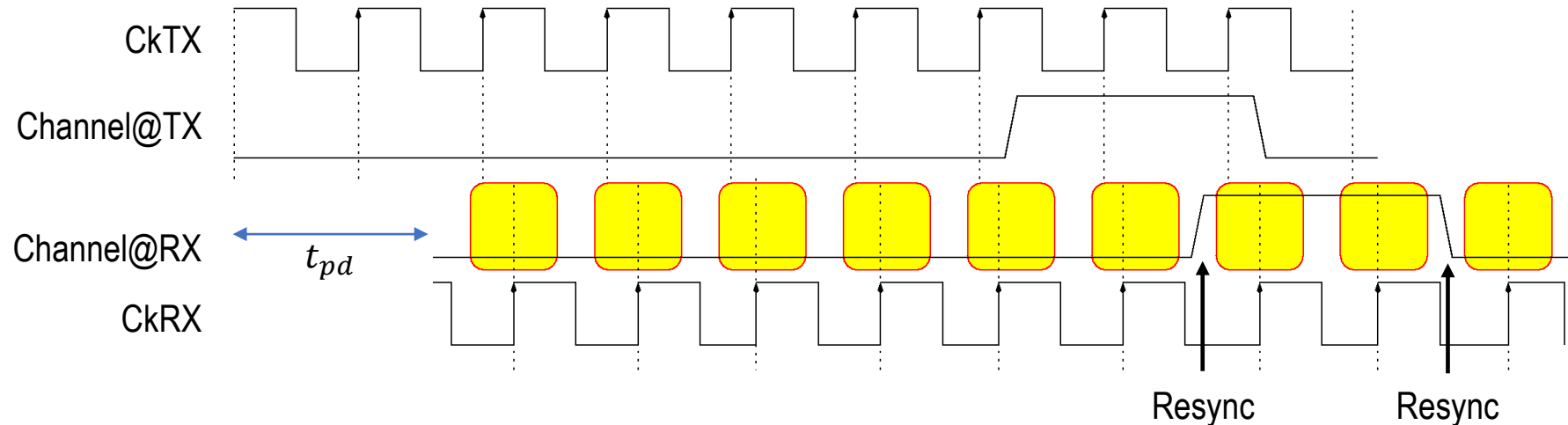
Synchronous Protocols

- $xByB$
 - Implementation of 4B5B
 - 4-input 5-output LUT when transmitting
 - The LUT contains the truth table of the previous slide
 - The 5 outputs go to the parallel inputs of a PISO Shift Register
 - 5-input 4-output LUT when receiving
 - The LUT contains the reverse truth table
 - The 5 inputs come from the parallel outputs of a SIPO Shift Register
 - May combine multiple LUTs for data on more than 4 bits
 - Example
 - Data on 8 bits
 - Transmitted data on 10 bits
 - 2 LUTs at the transmitter
 - 2 LUTs at the receiver



Synchronous Protocols

- Bit stuffing
 - Every n bits that don't cause a transition, insert a bit that causes a transition
 - The value of n is normally between 5 and 8
 - The number of acceptable consecutive bits without a transition determined using the eye diagram
 - It differs depending on the protocol
 - Transmitter and receiver clocks re-synchronized at transitions

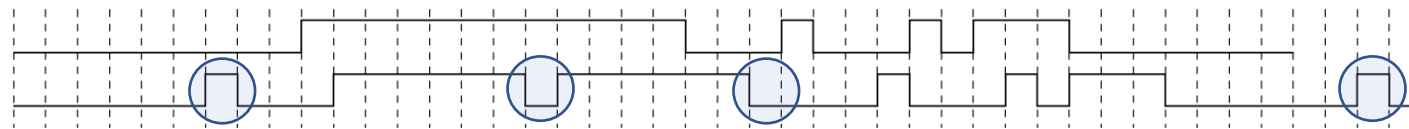


Synchronous Protocols

- Bit stuffing

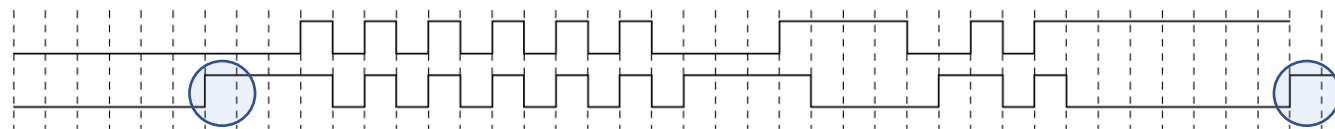
- Example with NRZ-L and $n = 6$

- Long sequences of 0s or long sequences of 1s do not generate transitions
 - 000000000111111111110001000101110000000
 - 00000010001111110111111000010001011100000010



- Example with NRZ-I (NRZ-M) and $n = 6$

- Long sequences of 0s do not generate transitions
 - 000000000111111111110001000101110000000
 - 0000001000111111111111100010001011100000010



Synchronous Protocols

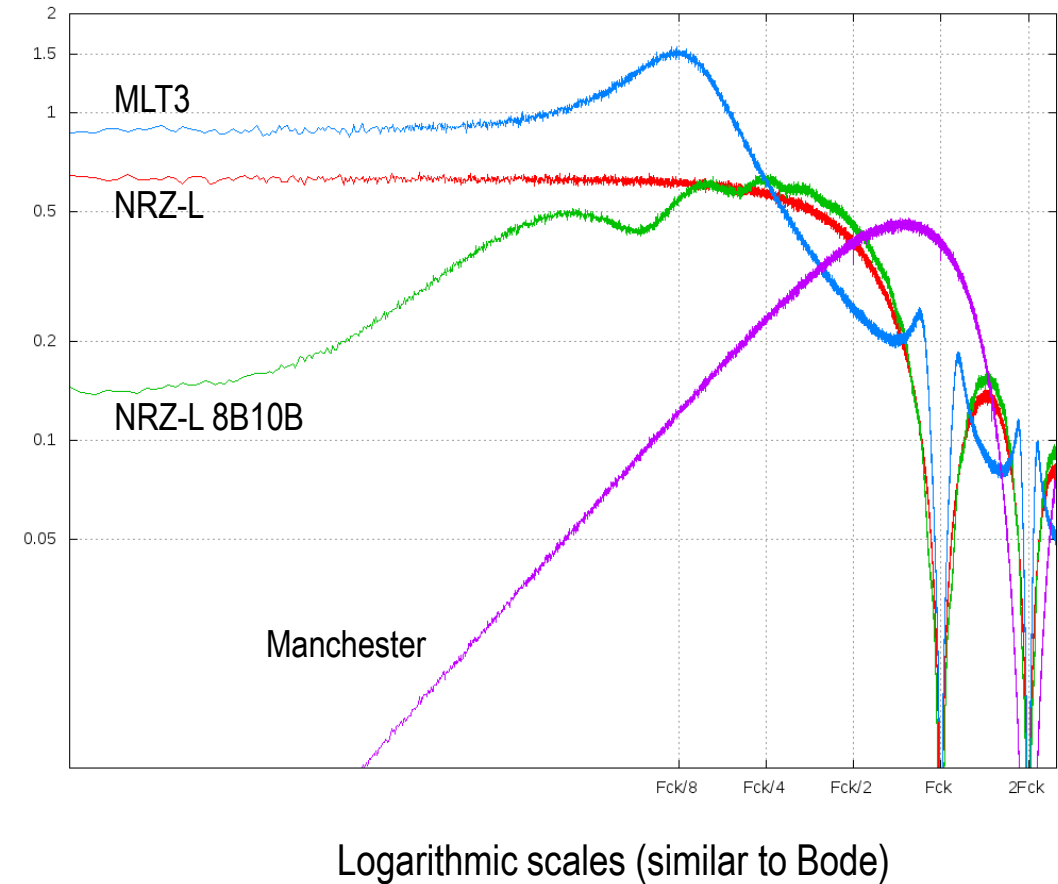
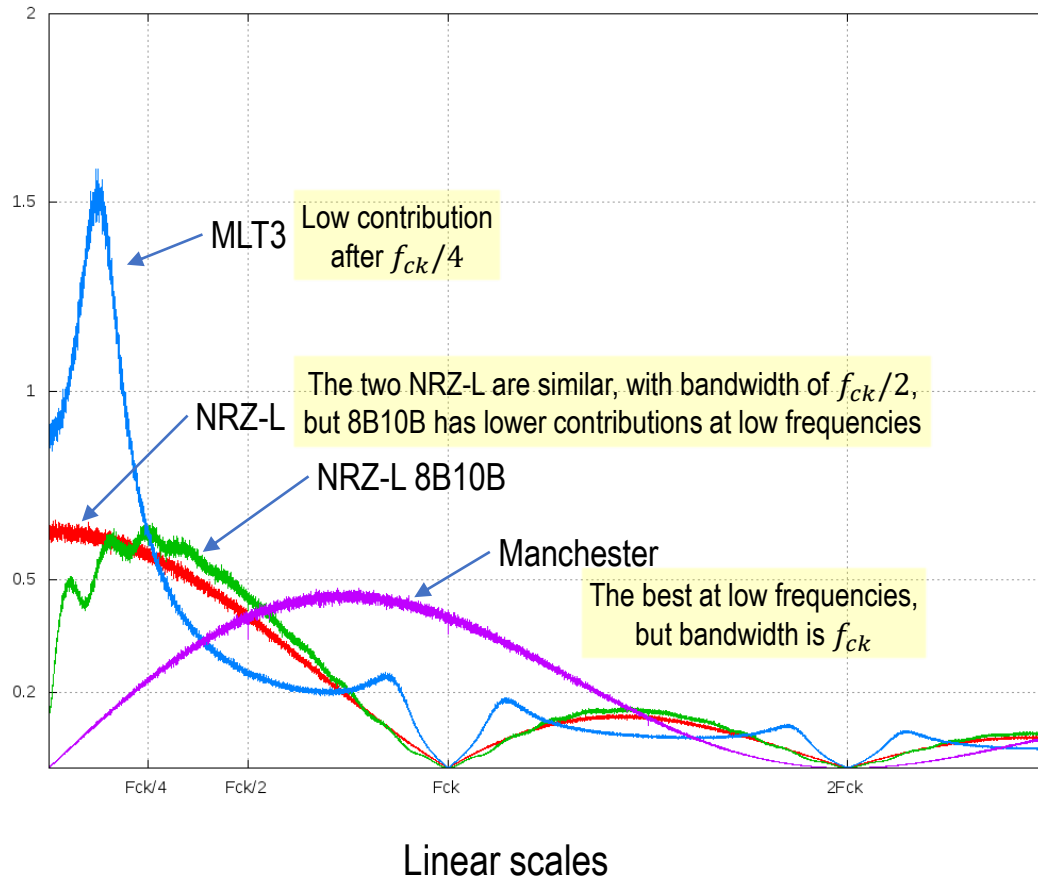
- Bit stuffing
 - Overhead due to added bits
 - Lower performance
 - But bits added only when necessary
 - Better than $xByB$ that always adds bits
 - Added bits are data dependent
 - Unpredictable length of the transmission
 - Non-determinism may be a problem in some cases
 - $xByB$ is predictable and deterministic
- Implementation
 - Small FSM at both transmitter and receiver
 - Sequence recognition

Synchronous Protocols

- Waveform and spectrum examples
 - DC and low frequency components may be hard to transmit
 - Especially if there are galvanic or optical transformers in the transmission chain
 - Some encodings guarantee the absence of DC and low frequency components
 - Manchester
 - Each transmitted bit is half high and half low
 - 8B10B
 - Equal number of 0s and 1s in the encoding in the long run
 - Spectrum examples
 - Obtained as the average of the spectra of 1000 different random bitstream of 8192 bits
 - Spectra show a superimposed $\sin x/x$ trend (Fourier transform of a box)
 - Zero at f_{ck} or $2f_{ck}$ and multiples
 - Compare with the bandwidths given in previous slides
 - There are still contributions above the bandwidth

Synchronous Protocols

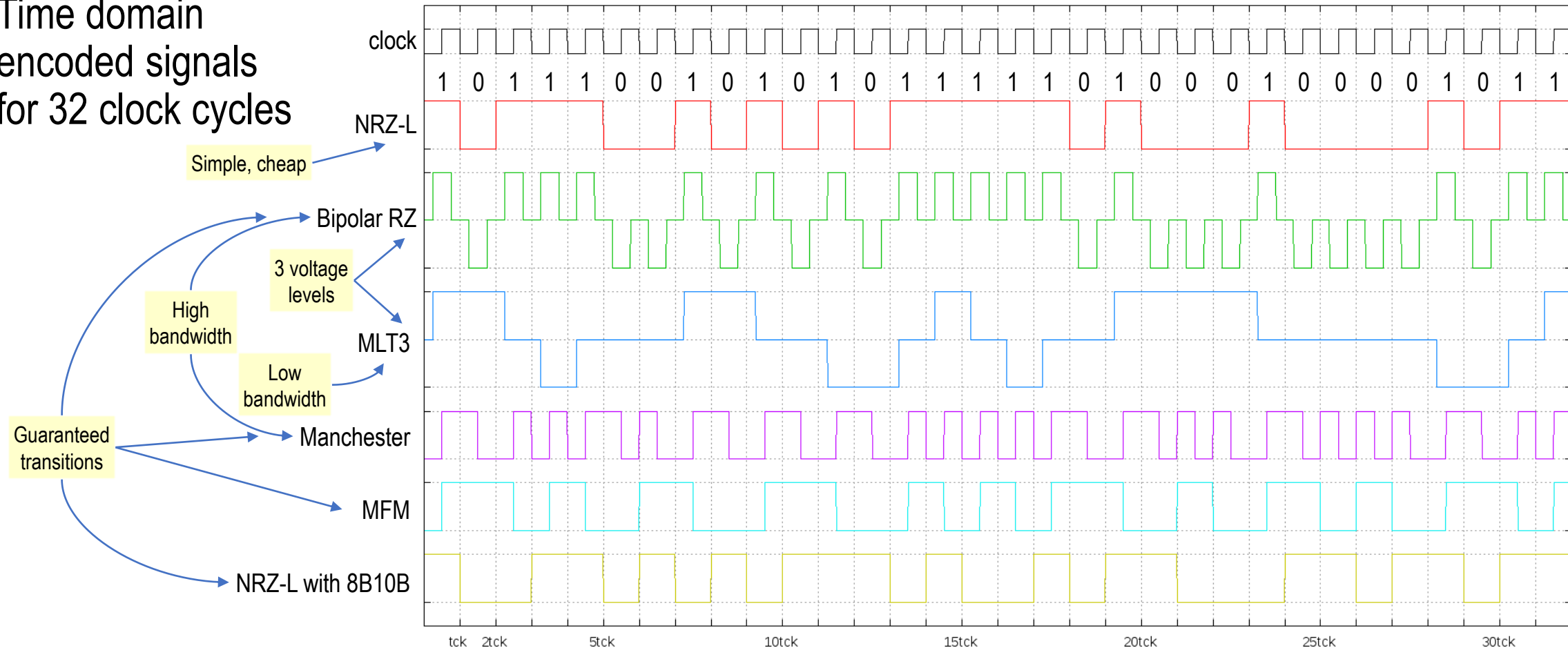
- Waveform and spectrum examples



Synchronous Protocols

- Waveform and spectrum examples

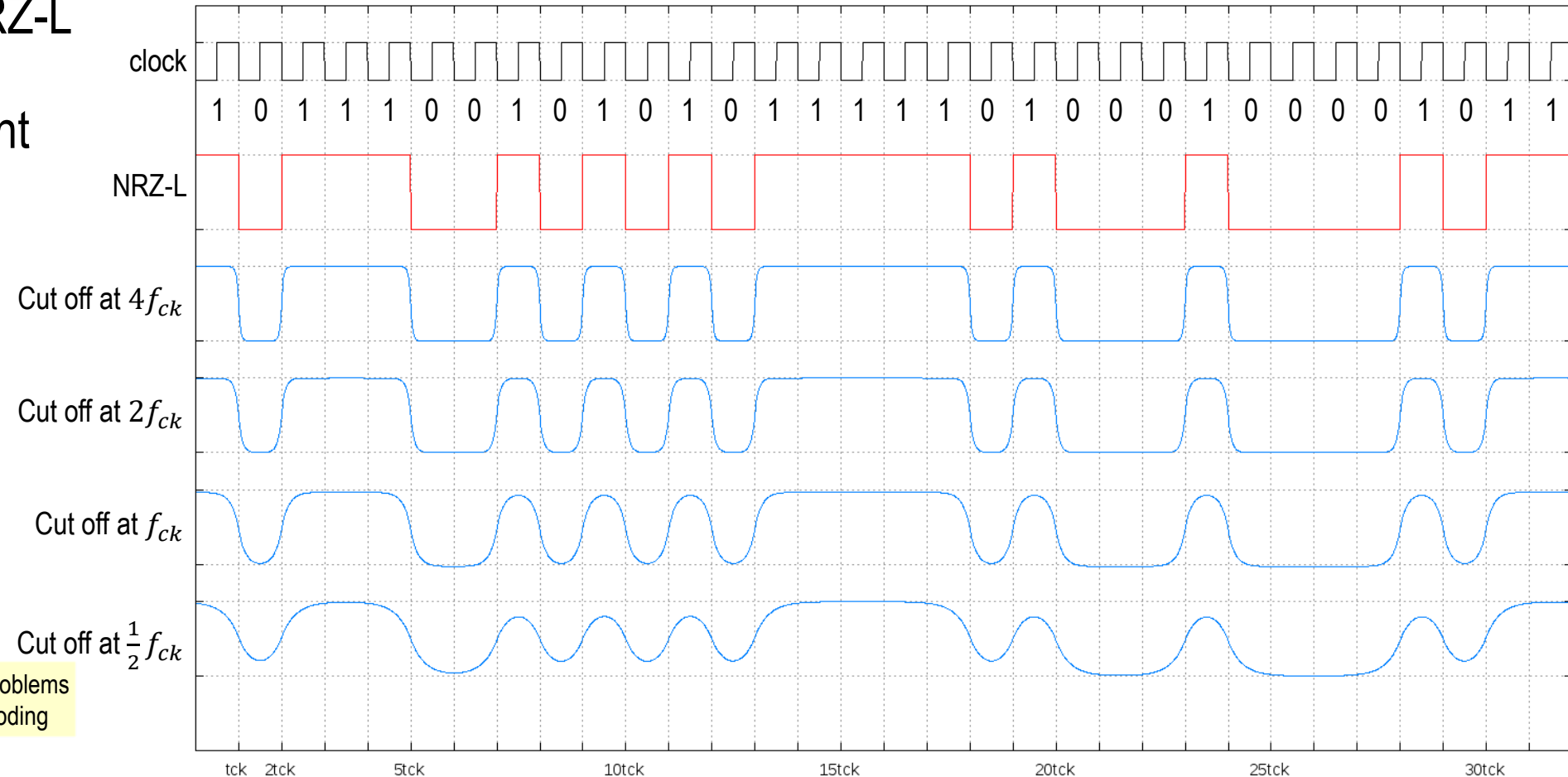
- Time domain encoded signals for 32 clock cycles



Synchronous Protocols

- Waveform and spectrum examples

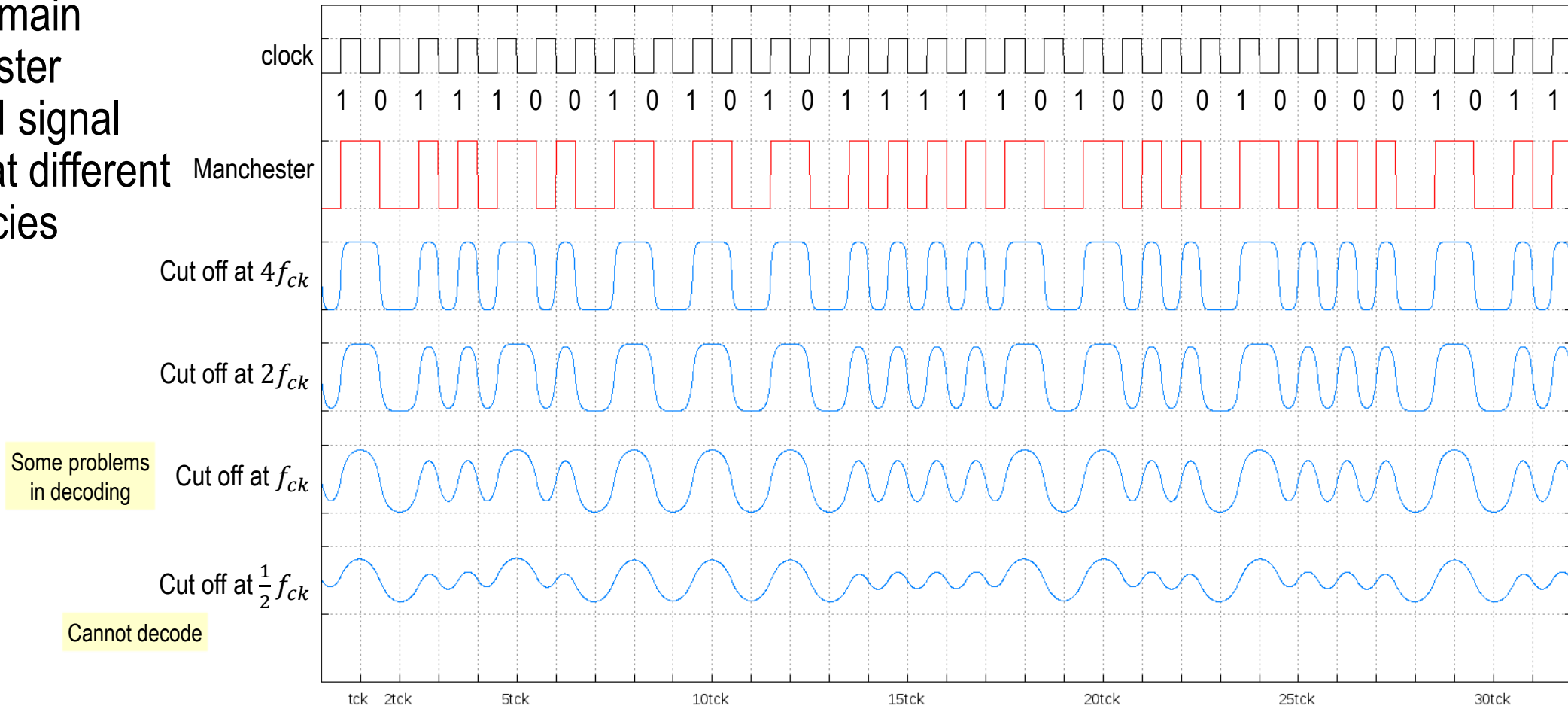
- Time domain NRZ-L
encoded signal
filtered at different
frequencies



Synchronous Protocols

- Waveform and spectrum examples

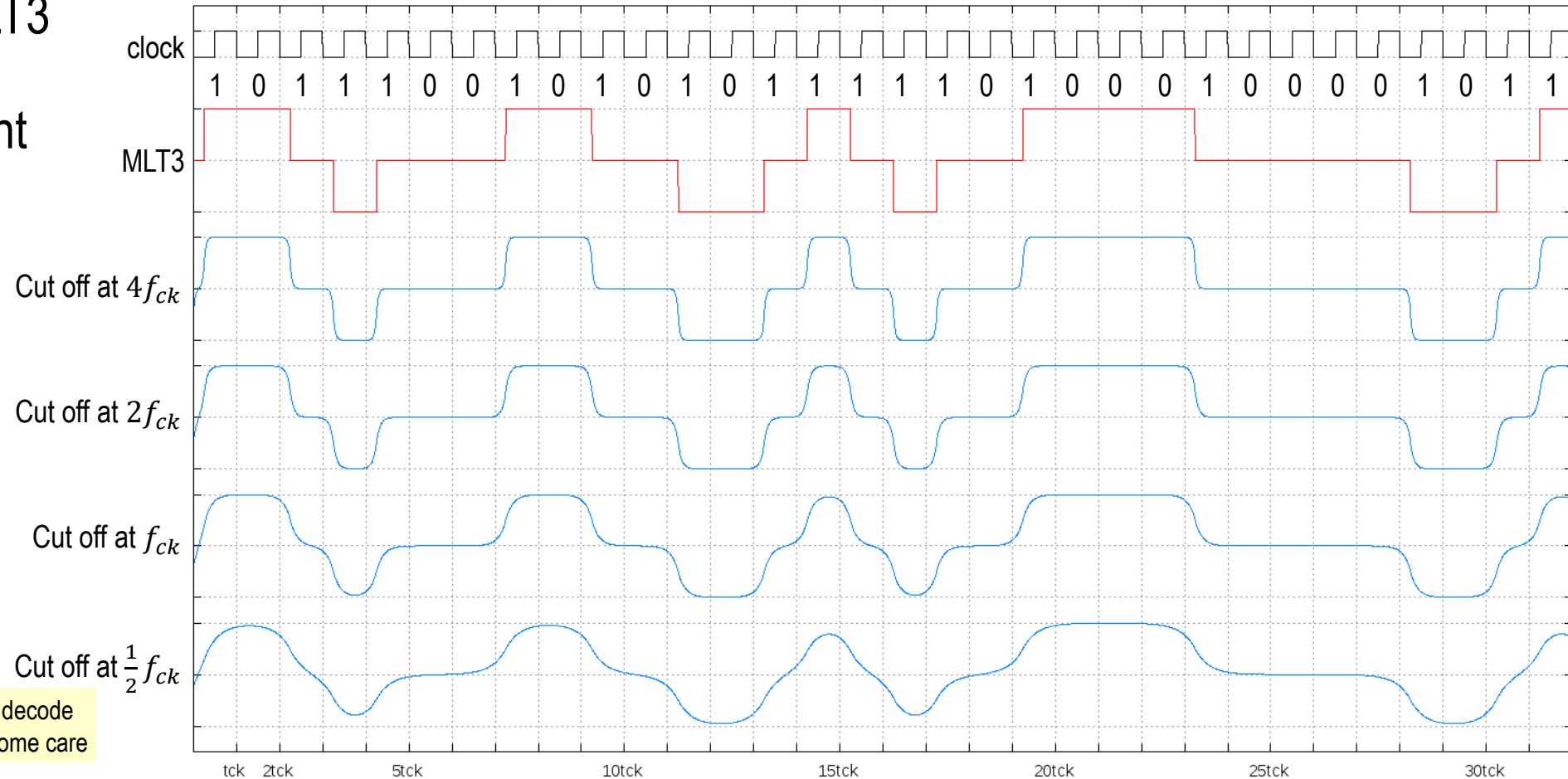
- Time domain
Manchester
encoded signal
filtered at different
frequencies



Synchronous Protocols

- Waveform and spectrum examples

- Time domain MLT3 encoded signal filtered at different frequencies



SPI and I²C

- Used to connect integrated circuits
 - On the same board
 - Wires are copper lines on the PCB
 - In different boards but with short connections
 - Organized as a network of several nodes
- Master and slave devices are identified
 - They have different roles
 - They can both be receiver and/or transmitter
- Serial protocols
 - Single wire for data transmission → serial
 - Bits are multiplexed in time

SPI and I²C

- Synchronous protocols
 - But don't use clock extraction
 - Separate wire for clock transmission
 - Skew problem, cannot achieve very high performance
 - Limited length of the interconnections
- Multiple nodes can drive the single serial line
 - Save costs
 - Share wires
 - Avoiding several expensive point to point communications
- Performance
 - From hundreds of Kbit/s to a few Mbit/s
 - Should also consider overheads

SPI and I²C

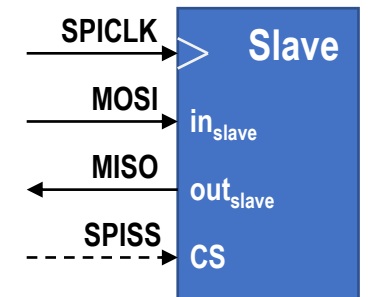
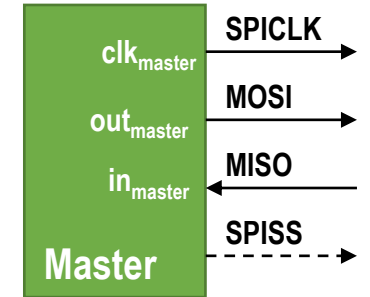
- Aspects to consider in a communication
 - Number of masters and slaves
 - Masters can initiate a transmission
 - Communication always between a master and a slave
 - Not between two slaves
 - Number of concurrent communications
 - Full duplex or half duplex
 - Arbitration between multiple masters
 - How to identify the target slave of a communication
 - Addressing
 - Bit synchronization
 - Who generates the clock and how it is exchanged between blocks
 - Avoiding dangerous conflicts in shared wires
 - Error checking

Serial Peripheral Interface (SPI)

- Single master, multiple slave
 - The number of slaves is not limited in principle
 - But usually does not exceed 4 or 5
- Full duplex transmission
 - One wire for data from master to slaves
 - Master Out Slave In (MOSI)
 - One wire for data from slaves to master
 - Master In Slave Out (MISO)
 - Master and slaves are always both transmitters and receivers
 - If a direction is not needed
 - Some (possibly random) data is sent anyway and discarded by the receiver
 - If a direction is never needed, the corresponding wire might be missing in the system

Serial Peripheral Interface (SPI)

- At least 3 wires, but often 4 wires or more
 - SPICLK
 - The clock, always generated by the master
 - MOSI
 - Master Out Slave In, the output data line of the master, and input data line of slaves
 - MISO
 - Master In Slave Out, the input data line of the master, and output data line of slaves
 - Multiple drivers, risk of dangerous conflicts
 - SPISS
 - The slave select, connected to the Chip Select input of slaves, and driven by the master
 - There might be multiple SPISS when connecting to several slaves
 - It might be absent when the master is connected to a single slave
 - Chip Select of slave is always active (i.e. connected to GND)
 - It might be absent for daisy chain connections, as well



Serial Peripheral Interface (SPI)

- Transmitting data
 - A predefined number n of bits is transferred in each transmission
 - All blocks in the SPI network have internal shift registers of size n
 - One bit per clock cycle is transmitted
 - NRZ encoding
 - Simple, cheap
 - No transition guaranteed
 - The separate clock provides the required synchronization
 - SPICLK is idle when there is no transmission
 - Constant level, depending on the defined *clock polarity* CPOL
 - CPOL=0, clock is low when idle
 - CPOL=1, clock is high when idle

Serial Peripheral Interface (SPI)

- Transmitting data (continued)
 - The Slave Select of the target slave is activated during the entire transmission
 - All other slaves with the SS not active
 - Keep their output (on MISO) in high impedance
 - Ignore what happens on MOSI and SPICLK
 - Master generates n clock cycles
 - The first clock transition denotes the beginning of a transmission
 - Data is transmitted MSB first
 - Data changes on a clock edge and is sampled on the opposite clock edge, according to the *clock phase* CPHA
 - CPHA=0, data on trailing edge, sample on leading edge
 - CPHA=1, data on leading edge, sample on trailing edge
 - Slave Select is deasserted to complete the transmission
 - The clock returns idle

Serial Peripheral Interface (SPI)

- Master and slaves shall agree on:
 - Number of bits in a transmission
 - 8, 16, 12, ...
 - Clock frequency
 - Clock generated by master, received by slaves
 - Slaves should support the master clock frequency
 - Or otherwise setup and hold time are violated at the slave
 - Clock Polarity (CPOL)
 - Clock Phase (CPHA)
 - CPOL and CPHA define three parameters
 - Idle clock level
 - Active edge for data output
 - Active edge for data sampling

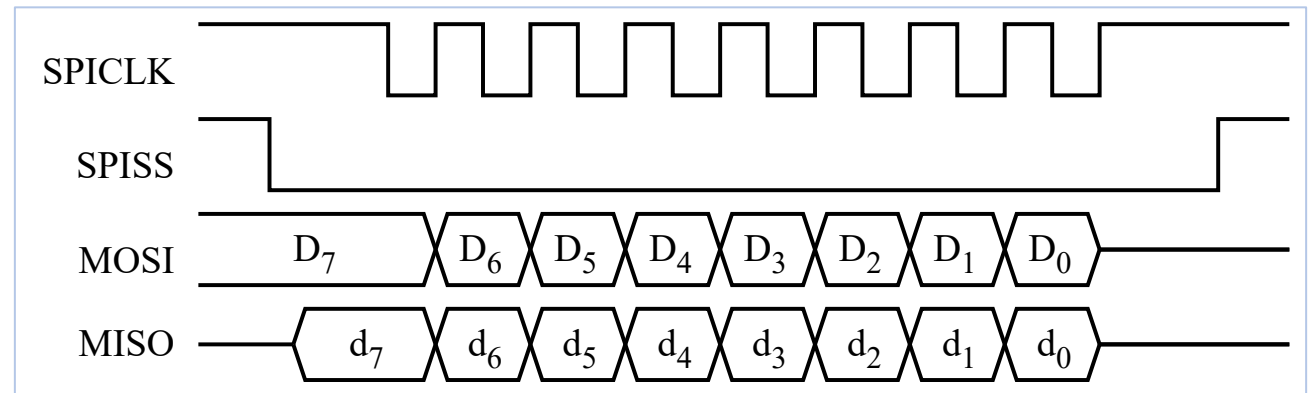
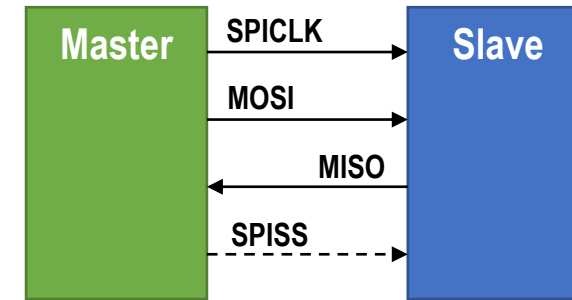
CPOL	CPHA	Idle CLK	Data output	Sampling
0	0	0	Falling	Rising
0	1	0	Rising	Falling
1	0	1	Rising	Falling
1	1	1	Falling	Rising

Serial Peripheral Interface (SPI)

- Single Master, Single Slave
 - SPISS not strictly needed

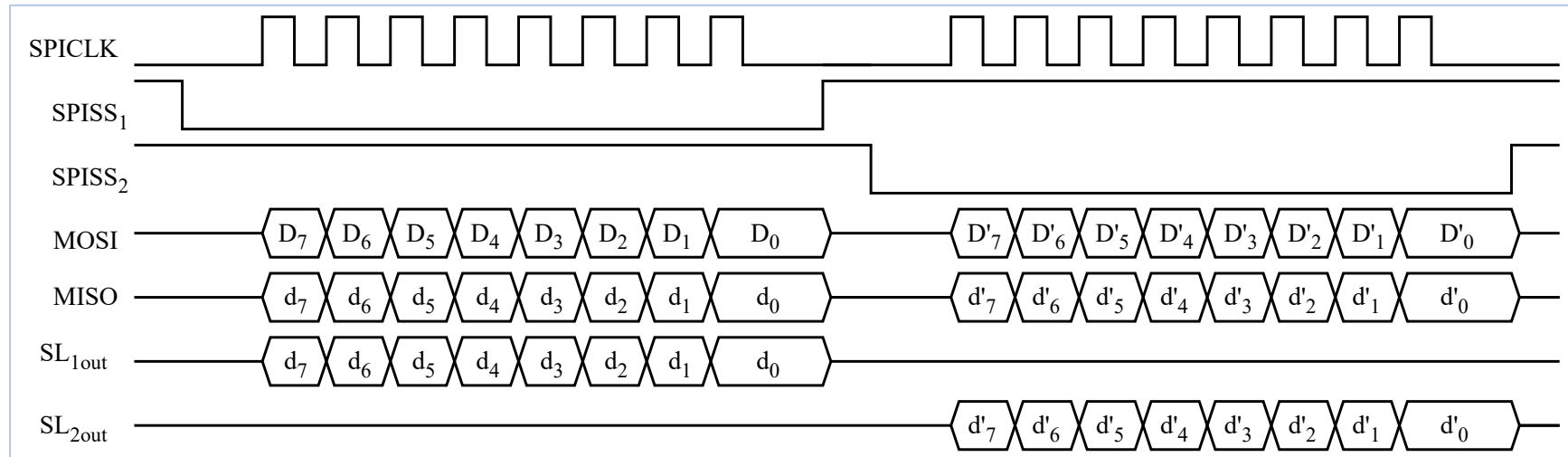
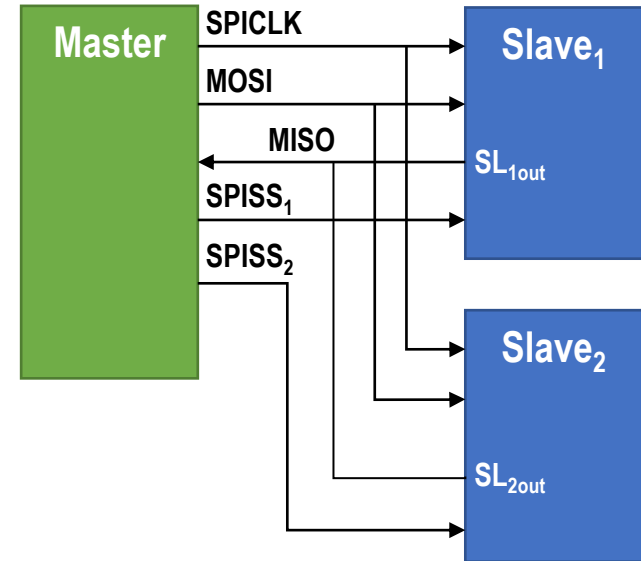
- Example

- Full duplex, $n = 8$
 - DATA = $D_7 \div D_0$ from Master to Slave
 - data = $d_7 \div d_0$ from Slave to Master
- CPOL = 1
 - Idle clock is high
- CPHA = 0
 - Data on trailing edge, sampling on leading edge
 - MSB ready on MOSI and MISO before clock starts
 - Because first clock edge is for sampling

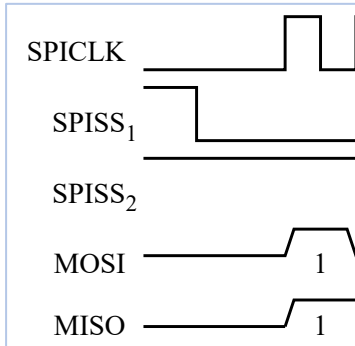


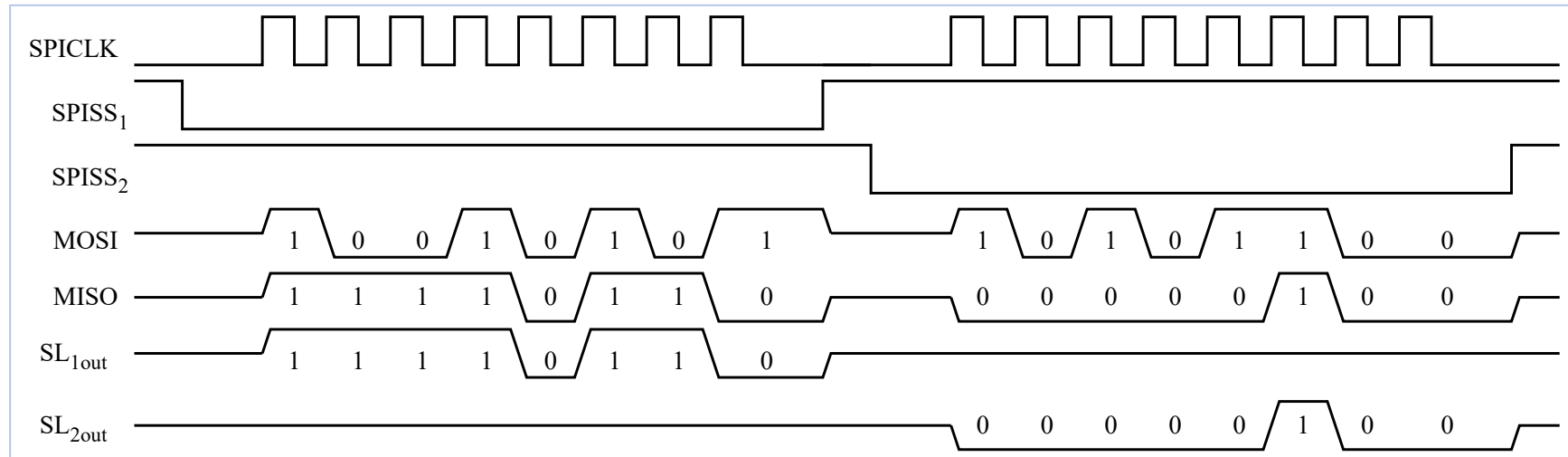
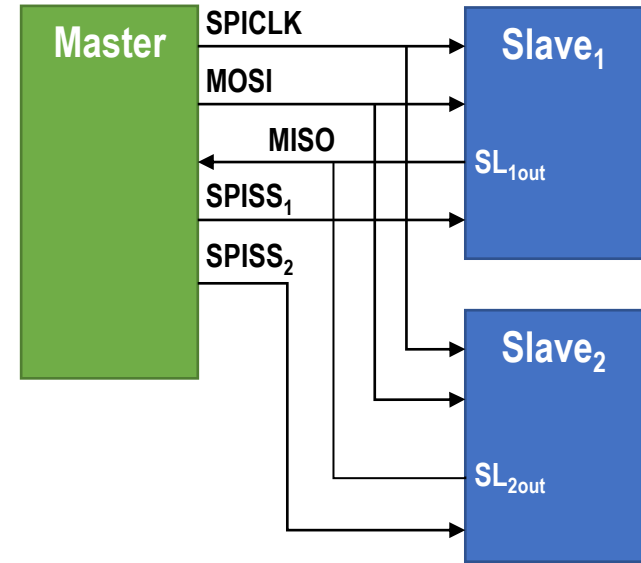
Serial Peripheral Interface (SPI)

- Single master, multiple slaves
- Example
 - Full duplex, $n = 8$, CPOL=0, CPHA=1
 - First transmission to/from Slave₁
 - DATA = $D_7 \div D_0$ from Master to Slave₁
 - data = $d_7 \div d_0$ from Slave₁ to Master
 - Second transmission to/from Slave₂
 - DATA' = $D'_7 \div D'_0$ from Master to Slave₂
 - data' = $d'_7 \div d'_0$ from Slave₂ to Master



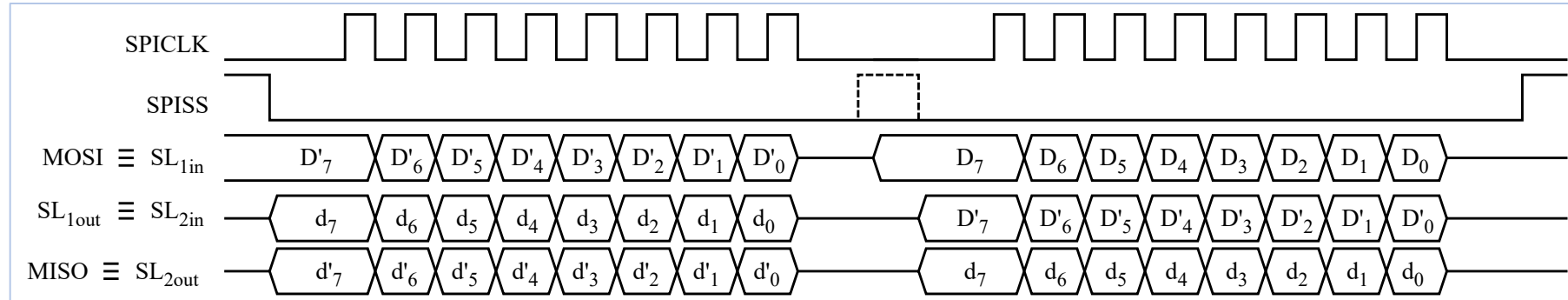
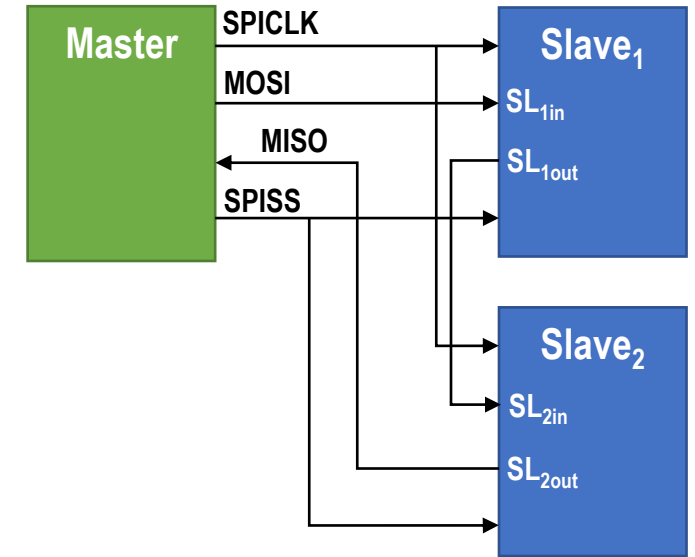
Interconnections and Communication Protocols

- Single master, multiple slaves
 - Numerical example
 - Full duplex, $n = 8$, CPOL=0, CPHA=1
 - First transmission to/from Slave₁
 - DATA = 0x95 from Master to Slave₁
 - data = 0xF6 from Slave₁ to Master
 - Second transmission to/from Slave₂
 - DATA' = 0xAC from Master to Slave₂
 - data' = 0x04 from Slave₂ to Master
- 
- The timing diagram illustrates the SPI communication protocol. It shows five signals: SPICLK, SPISS₁, SPISS₂, MOSI, and MISO. SPICLK is a periodic clock signal. SPISS₁ and SPISS₂ are slave select signals that pulse low when their respective slave is active. MOSI (Master Out Slave In) carries data from the master to the slaves, and MISO (Master In Slave Out) carries data from the slaves to the master. The diagram shows two data transfers: one to Slave₁ (DATA = 0x95) and one to Slave₂ (DATA' = 0xAC). The corresponding slave responses (data = 0xF6 and data' = 0x04) are shown on the MISO line.



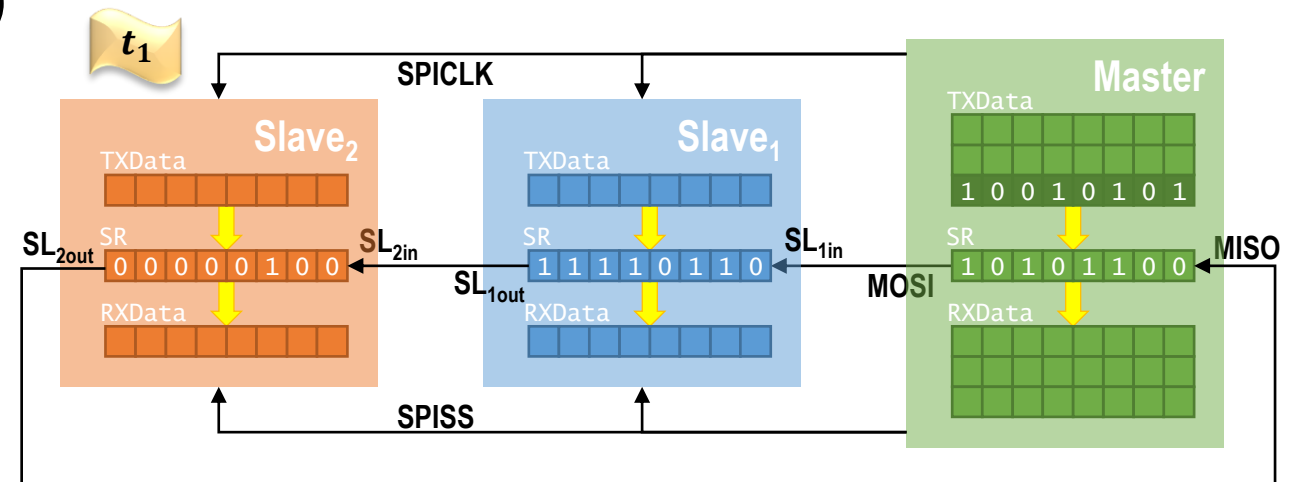
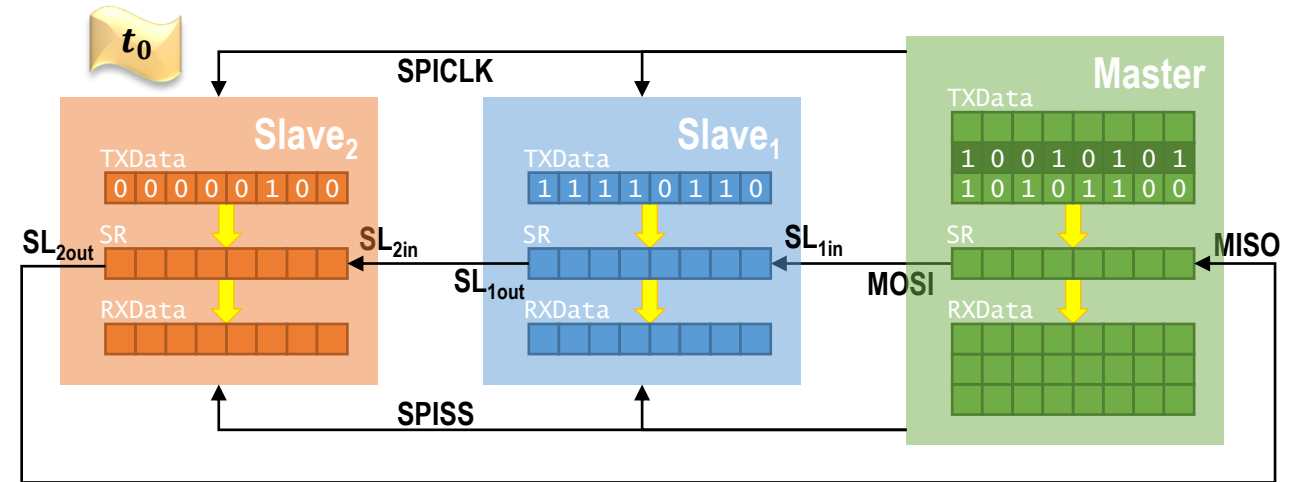
Serial Peripheral Interface (SPI)

- Single Master, multiple slaves, daisy chain
 - SPISS not strictly needed
 - No shared MISO
- Full duplex, $n = 8$, CPOL=0, CPHA=0
 - Every transmission involves all slaves
 - $DATA = D_7 \div D_0$ from Master to Slave₁
 - $data = d_7 \div d_0$ from Slave₁ to Master
 - $DATA' = D'_7 \div D'_0$ from Master to Slave₂
 - $data' = d'_7 \div d'_0$ from Slave₂ to Master
 - SPISS may show an intermediate transition (dashed)
 - Time between words to allow internal data storage and Shift Register updates



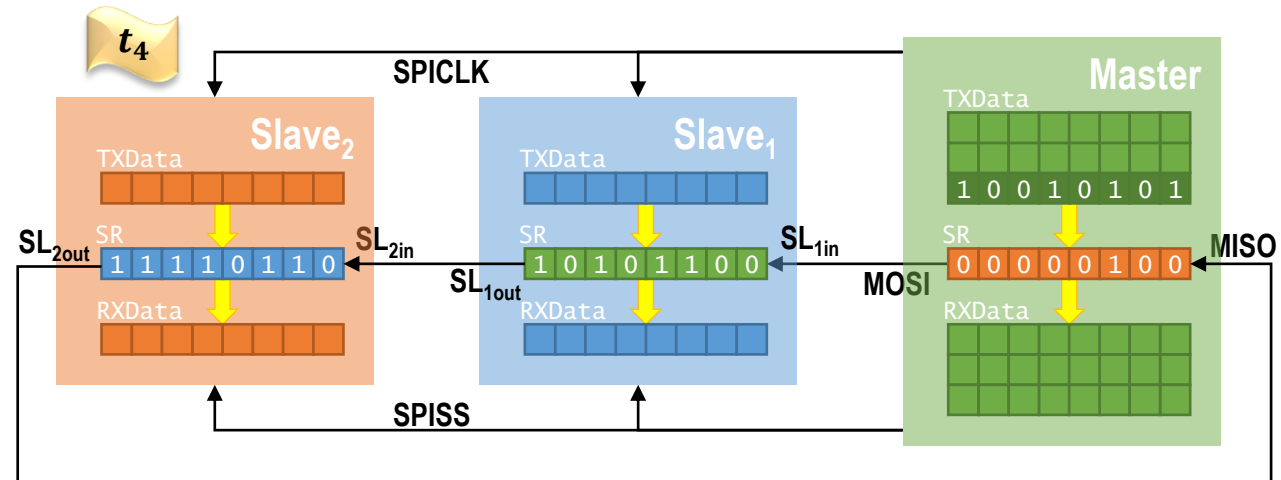
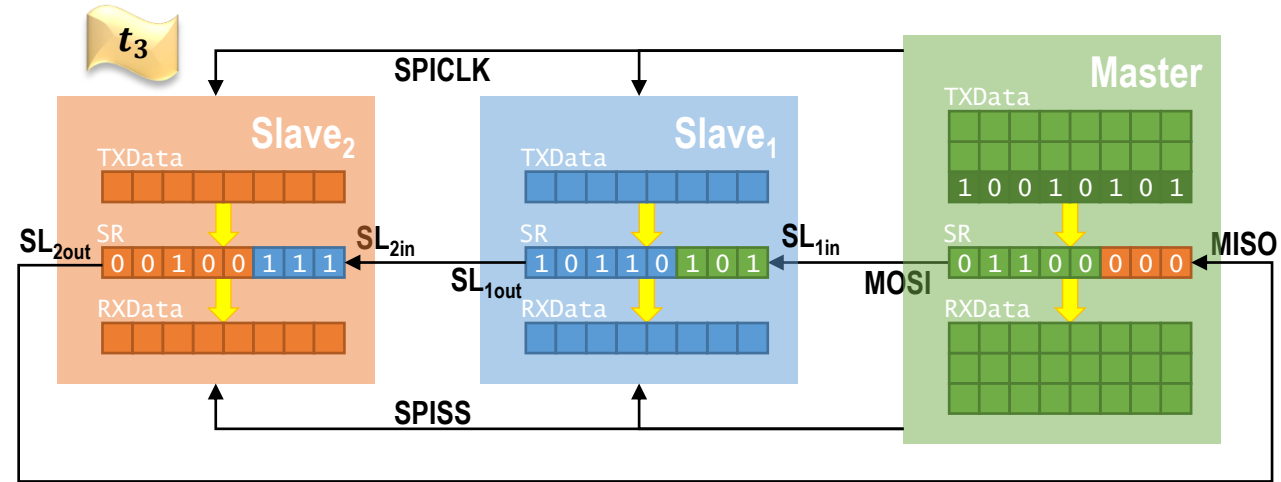
Serial Peripheral Interface (SPI)

- Single Master, multiple slaves, daisy chain, numerical example
 - Same data as before (time t_0)
 - DATA = 0x95 from Master to Slave₁ (darker green)
 - data = 0xF6 from Slave₁ to Master (blue)
 - DATA' = 0xAC from Master to Slave₂ (green)
 - data' = 0x04 from Slave₂ to Master (orange)
 - At the beginning (time t_1)
 - Transfer values from TXData to Shift Register
 - Master has two FIFOs for TXData and RXData
 - SPICLK and SPISS are inactive
 - TXData registers become available for new values



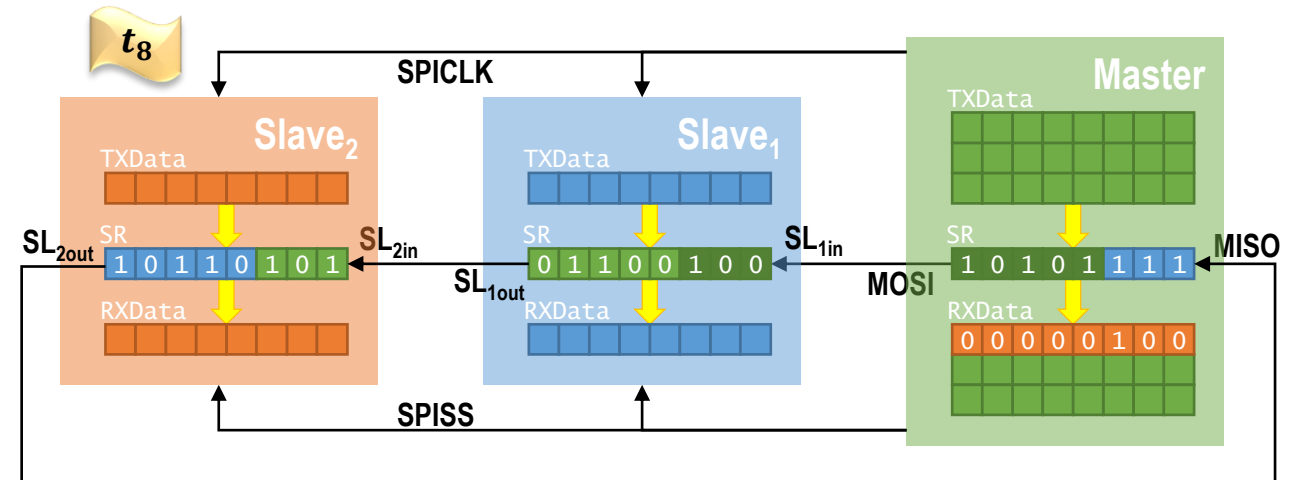
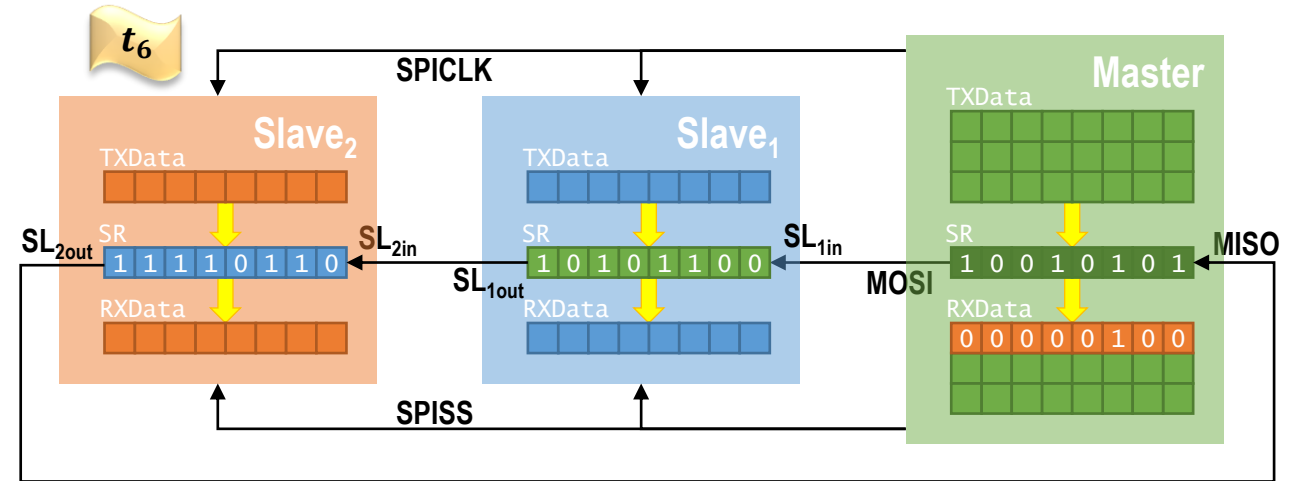
Serial Peripheral Interface (SPI)

- Single Master, multiple slaves, daisy chain, numerical example
 - Activate SPISS and start SPICLK (time t_2)
 - After three clock cycles (time t_3)
 - Data has partially moved to the successive shift registers
 - After eight clock cycles (time t_4)
 - Data has completely moved to the successive shift registers
 - SPICLK becomes inactive (time t_5)
 - SPISS may become inactive (time t_6)



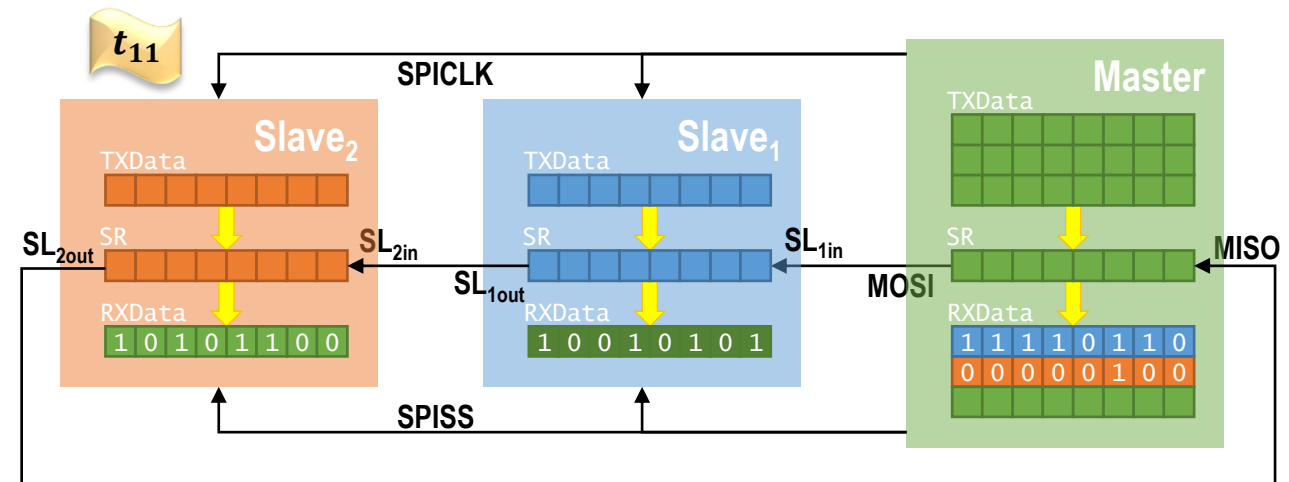
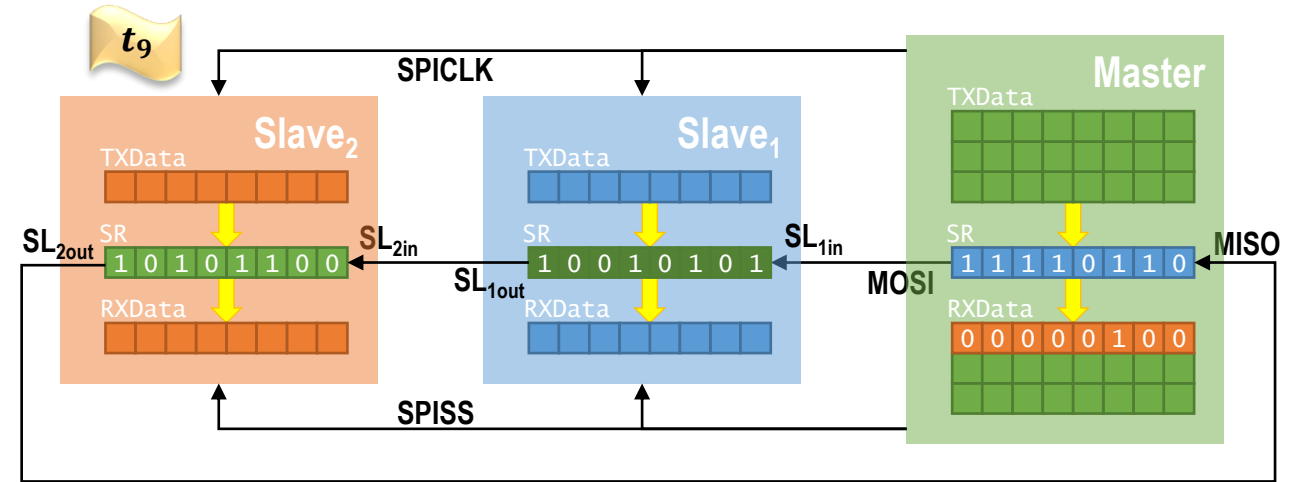
Serial Peripheral Interface (SPI)

- Single Master, multiple slaves, daisy chain, numerical example
 - After the first eight clock cycles, before the next transmission (time t_5 and t_6)
 - Master stores incoming data from the SR in the RXData FIFO
 - Master store outgoing data from the TXData FIFO to the SR
 - Slaves do not store any data yet
 - Activate SPISS and restart SPICLK (time t_7)
 - After three clock cycles (time t_8)
 - Data has partially moved to the successive shift registers



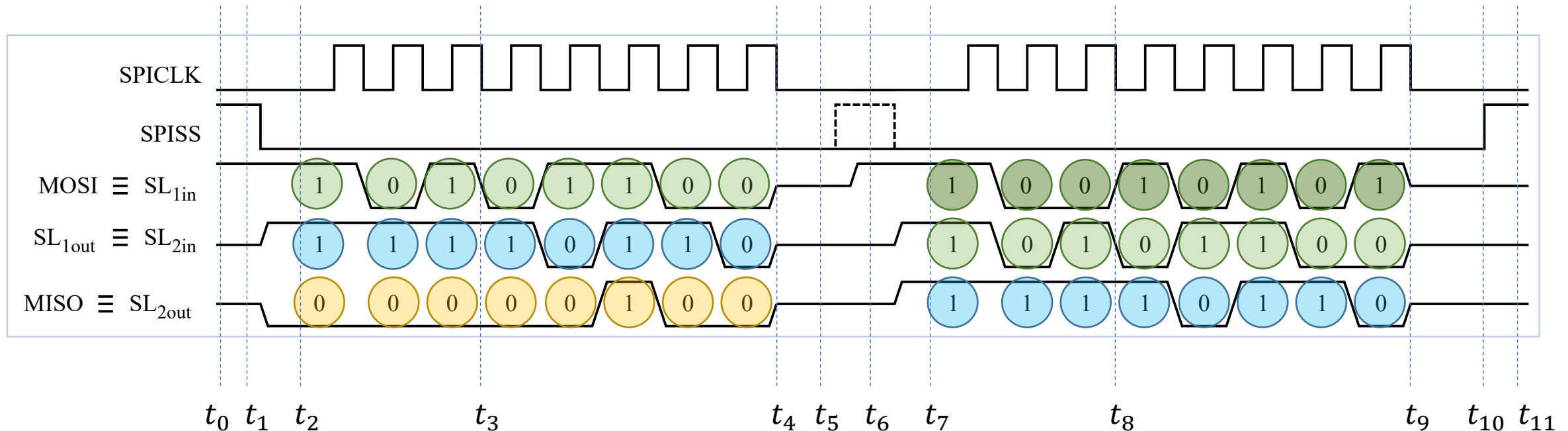
Serial Peripheral Interface (SPI)

- Single Master, multiple slaves, daisy chain, numerical example
 - After eight clock cycles (time t_9)
 - Data has completely moved to the successive shift registers
 - SPICLK and SPISS become inactive (time t_{10})
 - The transmission is completed (time t_{11})
 - Master and slaves store incoming data from the SR in their respective RXDatas
 - The RXDatas should now be saved in some other place
 - Blocks are now ready for a new transmission



Serial Peripheral Interface (SPI)

- Single Master, multiple slaves, daisy chain, numerical example
 - Data between t_4 and t_7 and between t_9 and t_{10} may or may not go to the high impedance state, depending on the implementation
 - When SPISS is inactive, the slave outputs SL_{1out} and SL_{2out} are certainly in the high impedance state



Serial Peripheral Interface (SPI)

- Arbitration
 - Not needed, there is only a single master
 - Slaves cannot initiate a transmission
- Addressing
 - The target slave is chosen using Chip Select signals
 - One separate signal for each slave, expensive
 - Not easy to change after fabrication
 - In daisy chain connection all slaves are always involved
- Bit synchronization
 - Clock always generated by the master
 - Configurable polarity and active edges
 - A slow slave cannot stop the master

Serial Peripheral Interface (SPI)

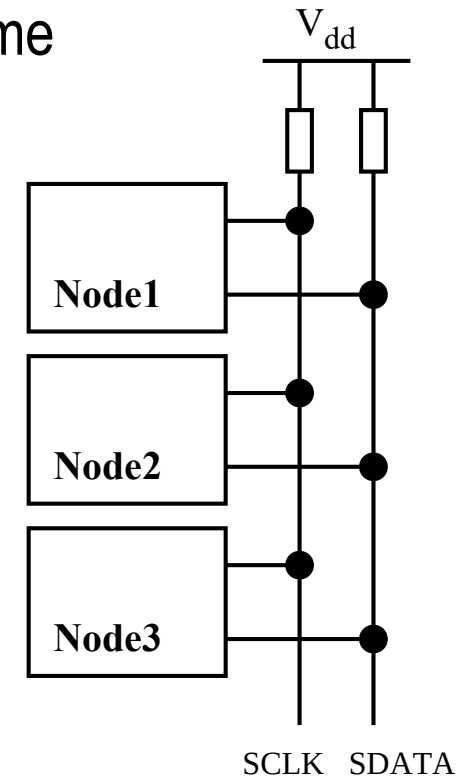
- Avoiding conflicts on shared busses
 - The only shared wire is MISO
 - It can be driven by multiple slaves
 - A single SPISS should be active at any given time
 - All other slaves keep their output in high impedance
- Error checking
 - No specific feature available for error detection and/or correction
 - Higher protocol levels may implement error checking techniques
- Performance
 - Several Mbit/s, or up to tens of Mbit/s
 - Sometimes used for SD Card memories
 - Short interconnections (tens of centimeters)

Inter Integrated Circuit (IIC, I²C)

- Multiple Masters, Multiple Slaves
 - Only masters can initiate a transfer
 - Multiple masters require arbitration
- Only two wires
 - SCLK: clock
 - SDATA: serial data
 - Single data line, half duplex communication
 - Separate data and clock → skew problem limits performance
 - 100 kbps, 400 kbps
 - Extensions to 1 Mbps, 3.4 Mbps, 5 Mbps
 - No select signals for slaves
 - The protocol requires addressing

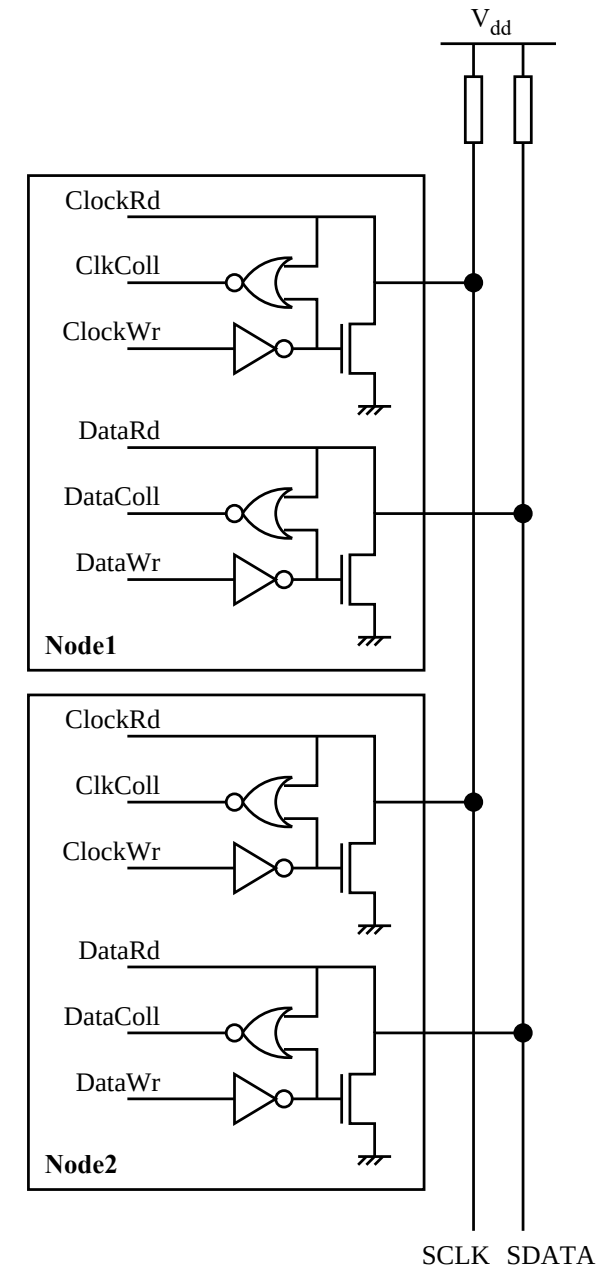
Inter Integrated Circuit (IIC, I²C)

- All devices can read and write the SDATA and SCLK lines
 - SCLK line mainly controlled by masters
 - Slaves can only *stretch* the clock duration if they are slow (cooperative clock)
 - SDATA line controlled by a single transmitter (master or slave) at any given time
 - The protocol should avoid collisions
- Open Drain architecture for both SCLK and SDATA
 - Logic 0: the line is driven to GND by the low side output transistor of a node
 - Logic 1: the low side transistor is OFF, the line is left floating
 - An external pull-up resistor brings the line to the high voltage
 - Pull-up of the order of a few k Ω
 - High time constant, low performance
 - When I²C is idle, both lines are normally high



Inter Integrated Circuit (IIC, I²C)

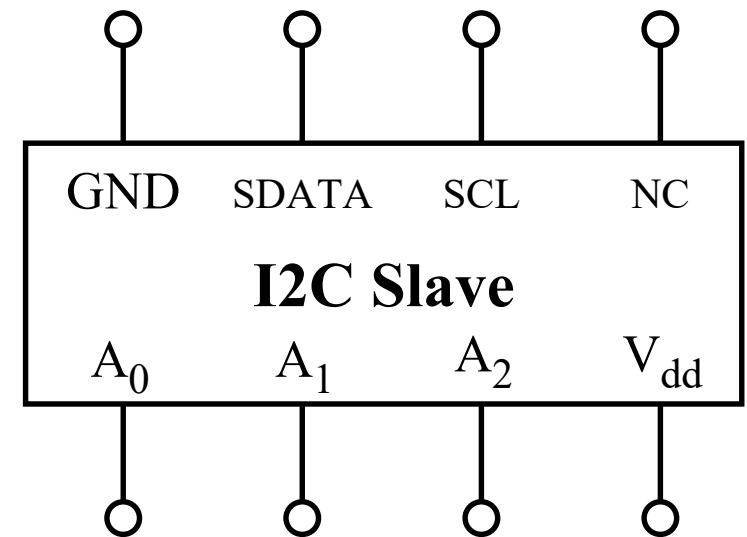
- Detecting collisions on SCLK and/or SDATA
 - Collision: a node generates 0, another generates 1
 - The output nMOS transistor of the node generating 0 is ON
 - The output nMOS transistor of the node generating 1 is OFF
 - There is no short circuit, no damages to the system
 - The line goes to 0
 - All nodes constantly monitor the SCLK and SDATA lines
 - The device generating the 1 detects the collision
 - The device generating the 0 does not notice
- Collisions on SCLK
 - Clock stretching
- Collisions on SDATA
 - Arbitration



Inter Integrated Circuit (IIC, I²C)

- Addressing

- Each slave in an I²C network is assigned a unique address on 7 bits
 - Maximum of 128 slaves, but typically much less
 - Some addresses are reserved
 - Masters do not need an address
 - There exist a protocol extension to 10-bit addresses
- Address is often hardwired in a slave by connecting pins to V_{dd} or GND
 - Some bits are not configurable (usually the MSBs) to save on the number of pins
- The master sends the address of the target slave at the beginning of a transmission



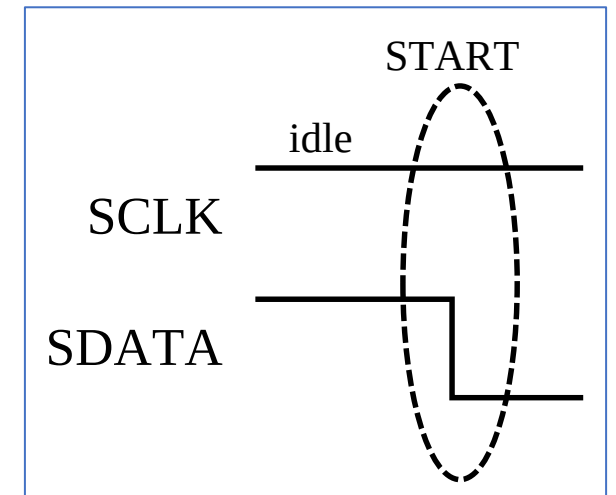
I2C Address: 0110A₂A₁A₀

Inter Integrated Circuit (IIC, I²C)

- I²C format
 - SCLK and SDATA lines are normally high when there is no transmission
 - All low side nMOS output transistors are OFF
 - RULE: SDATA line allowed to change only when SCLK is low
 - Two exceptions
 - START configuration: SDATA makes a high to low transition while SCLK is high
 - STOP configuration: SDATA make a low to high transition while SCLK is high
 - All communications begin with a START configuration
 - An addressing phase follows
 - A data exchange phase in a single direction is next
 - A STOP configuration concludes the communication

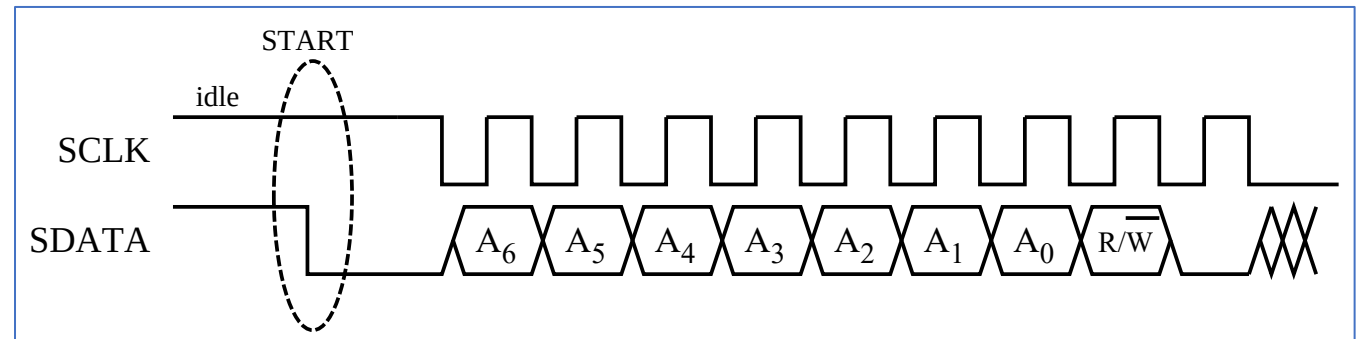
Inter Integrated Circuit (IIC, I²C)

- I²C format (continued)
 - START Configuration
 - Both SCLK and SDATA are high when idle
 - A master that wants to start a communication simply drives the SDATA line low
 - Activates the corresponding nMOS transistor
 - Keeps the SDATA line low for at least one bit time
 - Assume only one master generates a START configuration
 - The bus is now considered busy
 - All nodes detect the START configuration
 - Other masters wait for the communication to conclude before trying to take control of the I²C bus
 - All slaves will monitor the bus traffic to analyze the addressing phase
 - START is not used to synchronize clocks like in UART
 - Slaves don't have their own clock



Inter Integrated Circuit (IIC, I²C)

- I²C format (continued)
 - Addressing
 - The master activates the clock and generates on SDATA the target slave address on 7 bits, MSB first, one bit per clock cycle
 - SDATA changes only when SCLK is low
 - Usually shortly after the falling edge of SCLK
 - Slaves sample SDATA immediately after the rising edge of SCLK
 - The following bit indicates if the communication is a read or a write
 - $R/\overline{W} = 1 \rightarrow \text{Read}$, $R/\overline{W} = 0 \rightarrow \text{Write}$
 - Acknowledge bit: the master releases the SDATA line and generates a clock cycle
 - The target slave drives SDATA low
 - 9 bits transmitted in total



Inter Integrated Circuit (IIC, I²C)

- I²C format (continued)

- Data

- The master generates 9 clock cycles
 - 8 bits for data, 1 bit for acknowledge

- Write

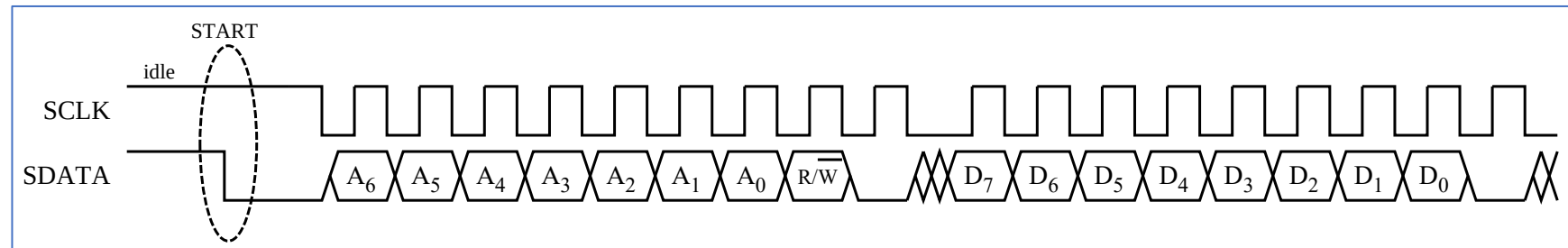
- The master regains control of SDATA and generates 8 data bits, MSB first
 - The slave generates ACK on the last bit (SDATA=0)

- Read

- The slave keeps control of SDATA and generates 8 data bits, MSB first
 - The master generates ACK on the last bit (SDATA=0)

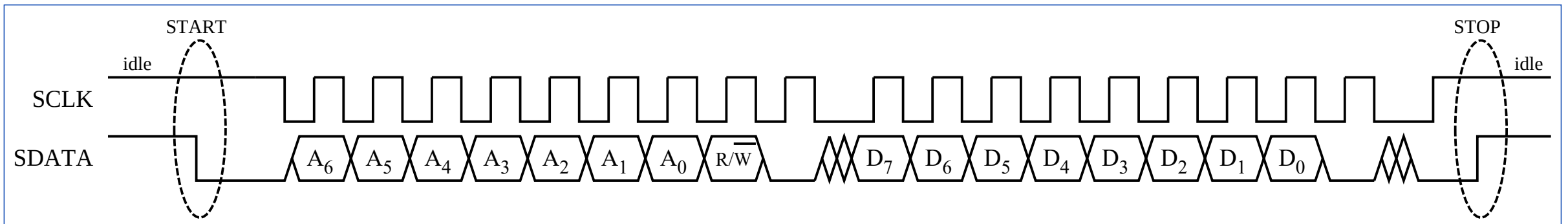
- A byte is transferred

- Multiple bytes can be transferred one after the other



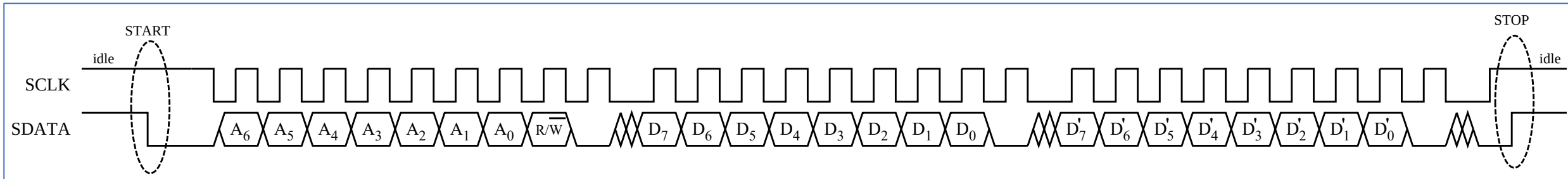
Inter Integrated Circuit (IIC, I²C)

- I²C format (continued)
 - STOP
 - After the last data byte, the master keeps SDATA low while releasing the SCLK
 - SCLK goes high
 - The master then releases SDATA as well
 - SDATA makes a low to high transition → STOP configuration
 - The bus is now idle again
 - All nodes, including other masters, detect the STOP
 - A new transmission, possibly from a different master, can start

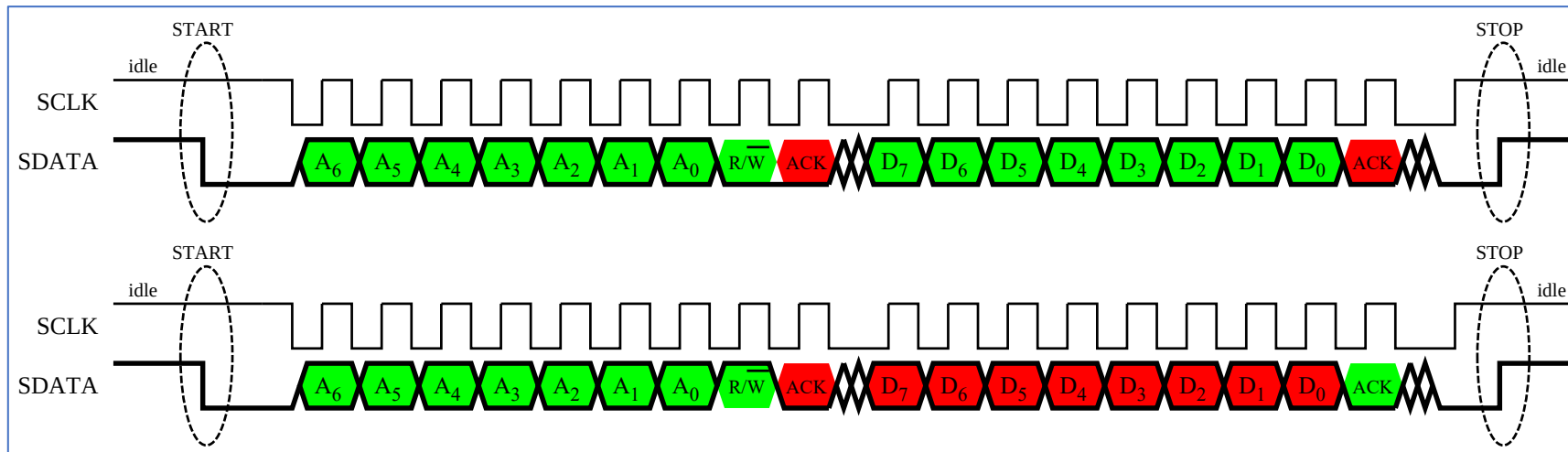


Inter Integrated Circuit (IIC, I²C)

- I²C format (continued)
 - Transferring multiple bytes



- Difference between reading and writing



Master writes to slave

■ Master drives SDATA

■ Slave drives SDATA

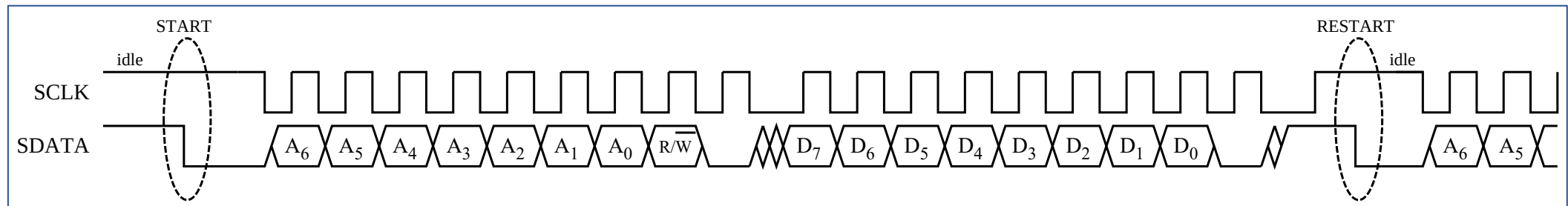
Master reads from slave

Inter Integrated Circuit (IIC, I²C)

- I²C format (additional details)
 - I²C always transmits blocks of 9 bits
 - 8 bits of data, followed by an ACK
 - The first 8 bits are driven by the transmitter (TX)
 - Always a master for the addressing phase
 - A master or a slave in the data phase
 - The ACK bit is driven by the receiver (RX)
 - Always a slave for the addressing phase
 - A master or a slave in the data phase
 - Before and after ACK the SDATA line may show oscillations while control passes from TX to RX and viceversa
 - Oscillations only happen while SCLK is low, and SDATA must become stable before the rising edge of SCLK
 - Avoid violating the rule

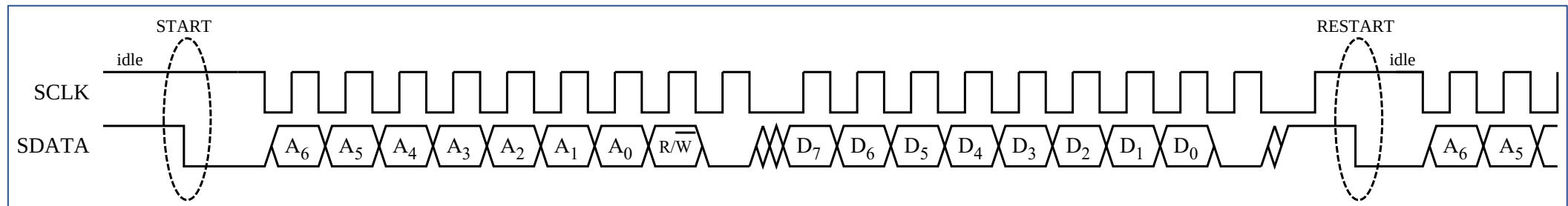
Inter Integrated Circuit (IIC, I²C)

- I²C format (additional details)
 - RESTART configuration
 - Sometimes a master needs to write a command to a slave, and then immediately read an answer
 - It requires two transmissions, the first in writing, the second in reading
 - The master doesn't want other masters to gain control of the bus before the second (reading) transmission
 - At the end of the first (writing) transmission:
 - The master does not generate a STOP configuration
 - The master generates a START configuration, again
 - Sets SDATA high, releases SCLK, sets SDATA low → RESTART
 - The bus never becomes idle, no other master can gain control



Inter Integrated Circuit (IIC, I²C)

- I²C format (additional details)
 - RESTART configuration (continued)
 - At the beginning of the second transmission, addressing is required again
 - Even if targeting the same slave as before
 - But the $R/\overline{W}$ bit can be different from the first transmission
 - Example: write command ($R/\overline{W} = 0$) in the 1st transmission, read answer ($R/\overline{W} = 1$) in the 2nd transmission
 - At the end of the second transmission, a regular STOP configuration is generated
 - Or another RESTART if needed
 - Restart can be used more in general for multiple transmissions without releasing the bus

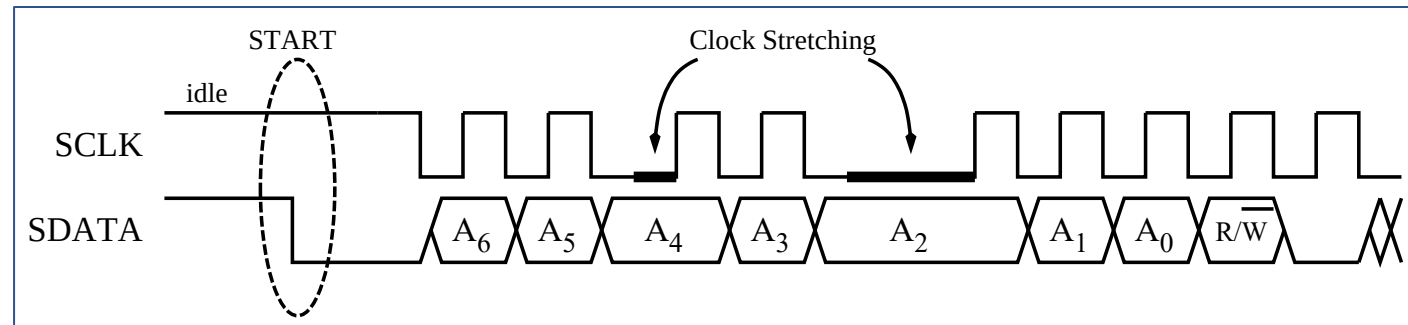


Inter Integrated Circuit (IIC, I²C)

- I²C format (additional details)
 - ACK after the addressing phase
 - It means the slave is ready for transmission
 - If ACK is missing (i.e. SDATA=1), the transmission is immediately aborted
 - ACK after the data phase, writing to slave
 - It means the slave received the data
 - It doesn't mean it was received correctly, no error detection on bits
 - If ACK is missing, data was not received correctly
 - Slave missed a clock pulse, it was not ready, the rule was violated (from the point of view of the slave), ...
 - ACK after the data phase, reading from slave
 - It means the master is ready to receive an additional byte
 - If ACK is missing, no more bytes are requested by the master

Inter Integrated Circuit (IIC, I²C)

- I²C format (additional details)
 - Clock stretching
 - Normally the clock on SCLK is generated by a master, for all bits
 - A slave should be sufficiently fast to operate at the clock frequency generated by the master
 - In this case, it only receives the clock and never writes it
 - A slow slave may extend the time at which the clock is low
 - The slave activates its own nMOS transistor on the SCLK line, while it is already forced low by the master
 - When the master releases SCLK, the line remains low
 - The master detects the collision and waits for the SCLK line to be released by the slave, as well
 - The communication then proceeds as usual
 - If clock stretching is used, individual bits may have different bit time
 - Not all masters and slaves support clock stretching

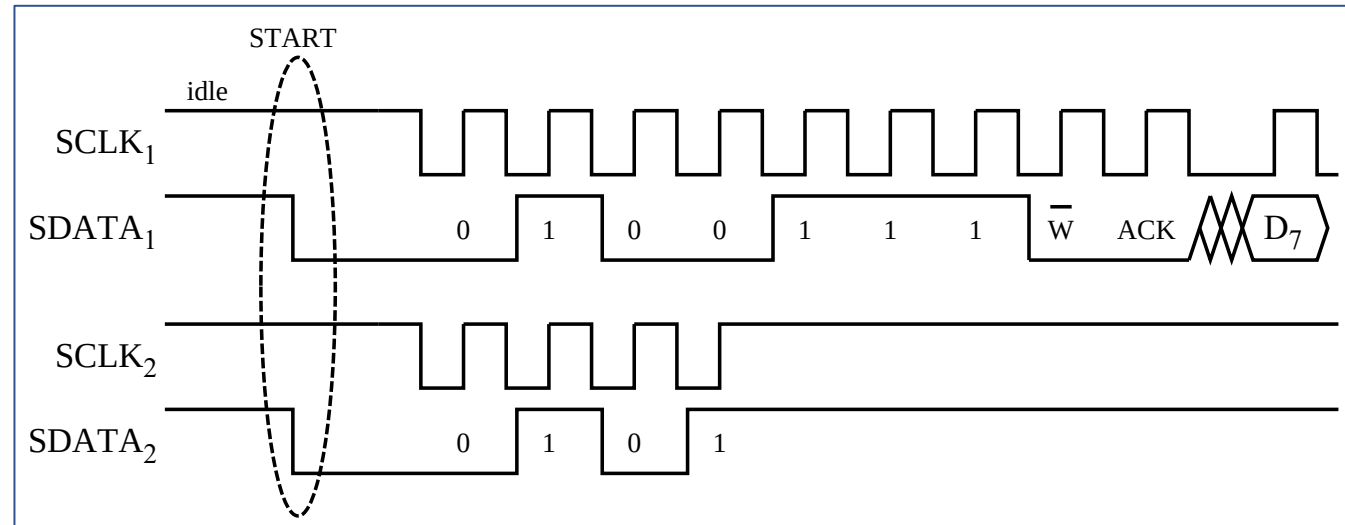


Inter Integrated Circuit (IIC, I²C)

- I²C format (additional details)
 - Arbitration
 - Multiple master may wish to start a transmission at the same time
 - If the bus is busy, a master shall not start a transmission
 - If two masters Master₁ and Master₂ (or more) concurrently generate a START configuration, they won't notice it
 - Each master believes he is the only owner of the bus
 - Assume in the following that they generate the same clock
 - During the addressing phase, they (probably) generate two different addresses
 - If at a certain bit Master₁ generates a 0 and Master₂ generates a 1
 - The line goes low
 - Master₁ does not notice any problem
 - Master₂ detects a collision on SDATA
 - Master₂ lost the contention and stops the transmission
 - Master₁ completes the transmission

Inter Integrated Circuit (IIC, I²C)

- I²C format (additional details)
 - Arbitration (continued)
 - The lowest generated slave address always wins contention
 - Slaves with a low address have a higher priority
 - If masters target the same slave, a write transmission wins over a read transmission
 - If both masters issue a write transmission, collision detection moves to the data phase
 - If both masters generate the same write transmission concurrently, the slave receives the data only once
 - If both masters issue a read transmission, they both receive the same data from the slave
 - Collisions on SCLK are interpreted as if a slave is stretching the clock
 - Not all masters support arbitration



Inter Integrated Circuit (IIC, I²C)

- I²C Performance
 - Bits are transmitted at the clock frequency rate
 - The addressing phase imposes an overhead
 - Burst transactions (i.e. sending multiple bytes after an addressing phase) limit the overhead
 - Not always possible
 - The ACK bit imposes an overhead
 - Implements basic error checking and flow control mechanisms
 - Rising time is much higher than falling time
 - Due to the high value of the pull-up resistors and other parasitics
 - High performance extensions use current sources to increase performance

Inter Integrated Circuit (IIC, I²C)

- I²C Performance

- Example

- Clock frequency $f_{clk} = 400 \text{ kHz}$
 - Clock period $t_{bit} = \frac{1}{f_{clk}} = 2.5 \mu s$
 - Master sends 3 data bytes to a slave
 - START Configuration: around 2 bit times
 - Addressing: 9 bits
 - Data: $3 \cdot 9 = 27$ bits
 - STOP Configuration: around 2 bit times
 - Delays: around 3 bit times (between blocks of 9 bits)
 - Total time for transmission
 - $t_{3bytes} = (2 + 9 + 27 + 2 + 3)t_{bit} = 43 \cdot t_{bit} = 43 \cdot 2.5 \cdot 10^{-6} = 107.5 \mu s$
 - Effective Bit Rate
 - $BR = \frac{n_{bits3bytes}}{t_{3bytes}} = \frac{3 \cdot 8}{107.5 \cdot 10^{-6}} = 223 \text{ kbps}$

Significant drop of the Bit Rate compared to the clock frequency due to the overhead

Inter Integrated Circuit (IIC, I²C)

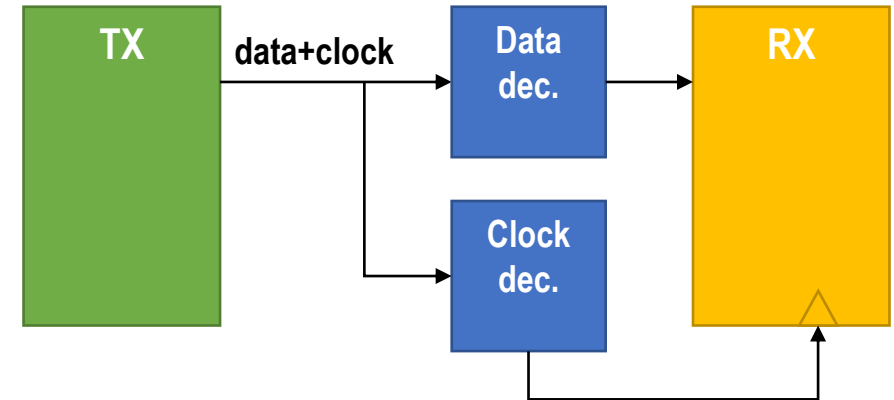
- Arbitration
 - Needed, there may be multiple masters
 - Collision on SDATA during addressing phase
- Addressing
 - The target slave is chosen by generating an address
 - Slaves have unique 7 bits addresses
 - Imposes an overhead
- Bit synchronization
 - Clock generated by the master
 - A slow slave can stop the master
 - Clock stretching

Inter Integrated Circuit (IIC, I²C)

- Avoiding conflicts on shared busses
 - Open drain architecture prevents damages in case of conflicts
 - Addressing avoid conflicts between slaves
 - Conflicts used for clock stretching and arbitration
- Error checking
 - ACK bit used to implement a low level form of error detection
 - Higher protocol levels may implement more sophisticated error checking techniques
- Performance
 - Hundreds of Kbit/s, or up to some Mbit/s
 - Slow because of the pull-up resistors
 - Short interconnections (tens of centimeters)

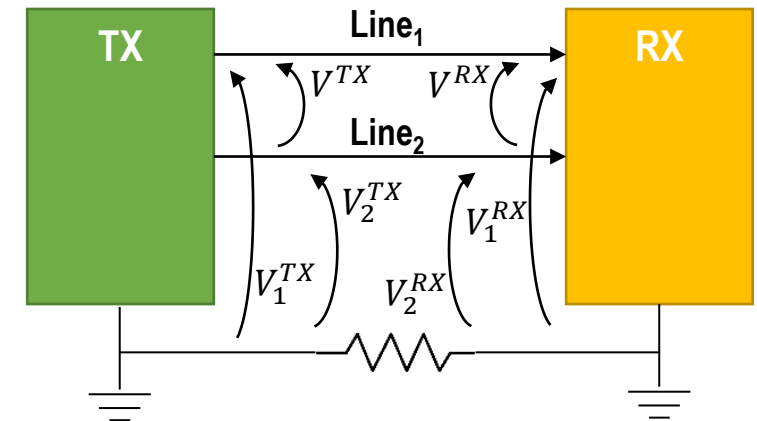
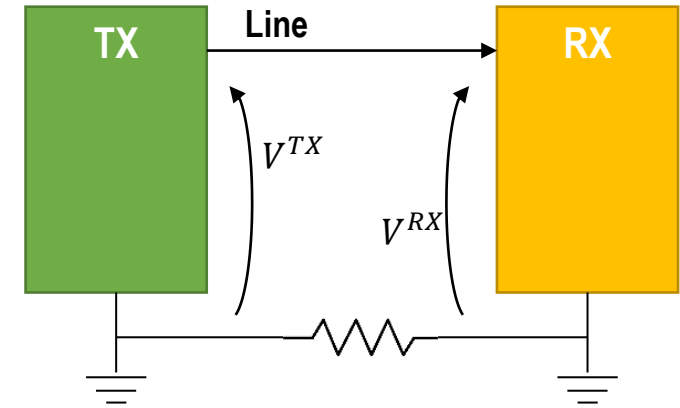
Controller Area Network (CAN)

- Increase communication rate
 - Serial communication with embedded clock
 - High clock frequency
 - Maximum frequency not affected by skew
 - Not for very short interconnections (i.e. within an IC)
 - Requires special encodings and circuits
 - Decoders at the receiver
 - To extract the clock information
 - To extract the data stream
- Increase interconnection length
 - Skew is very high for long interconnections
 - Serial communication with embedded clock
 - Serial is also cheaper than parallel, especially for long interconnections
 - Differential signaling



Controller Area Network (CAN)

- Differential signaling
 - Due to GND differences, with a single wire it is always $V^{TX} \neq V^{RX}$
 - Especially for long interconnections
 - May prevent correct transmission
 - Encode information as the voltage difference between two wires
 - Assume V_{CM} is the difference between the two GND (Common Mode noise)
 - $V_1^{RX} = V_1^{TX} + V_{CM}, V_2^{RX} = V_2^{TX} + V_{CM}$
 - $V^{TX} = V_1^{TX} - V_2^{TX}, V^{RX} = V_1^{RX} - V_2^{RX}$
 - $V^{RX} = V_1^{TX} + V_{CM} - (V_2^{TX} + V_{CM}) = V_1^{TX} - V_2^{TX} = V^{TX}$
 - Common mode noise is cancelled!



Controller Area Network (CAN)

- Differential signaling
 - Any other unwanted interference that equally affects both wires induces common mode noise
 - Cancelled by differential signaling
 - Wires affected by different noise
 - Noise source close to one wire, for instance
 - Twist the pair of wires so that the interference alternatively affects one wire and the other
 - Assume the interference is uniform along multiple twists
 - Noise becomes common mode and is cancelled by differential signaling
 - Shield the pair to reject interferences



4 Unshielded Twisted Pairs (UTP)



4 Shielded Twisted Pairs (STP)

Controller Area Network (CAN)

- Defined by Bosch in 1980
- Multi-node topology
 - Multiple blocks on the same bus
 - All blocks act as masters
 - Supports one-to-many communication
 - Supports arbitration between nodes
- Developed for automotive applications
 - Used also in other contexts
 - Designed for noisy environments
- Protocol defines several aspects of the communication
 - Physical (electrical) level
 - Transaction (frame format) level
 - Software (application) level

Controller Area Network (CAN)

- CAN performance
 - Low Speed (ISO 11519)
 - Up to 125 kbps
 - High Speed (ISO 11898)
 - Up to 1 Mbps
 - Interconnection length
 - Up to 40 meters @ 1 Mbps
 - Compared to tens of centimeters of SPI and I²C
 - Up to 6 km @ 10 kbps
 - Used for instance in large buildings or between buildings
 - Up to 2032 nodes per bus
 - But usually limited to max 100 nodes per bus

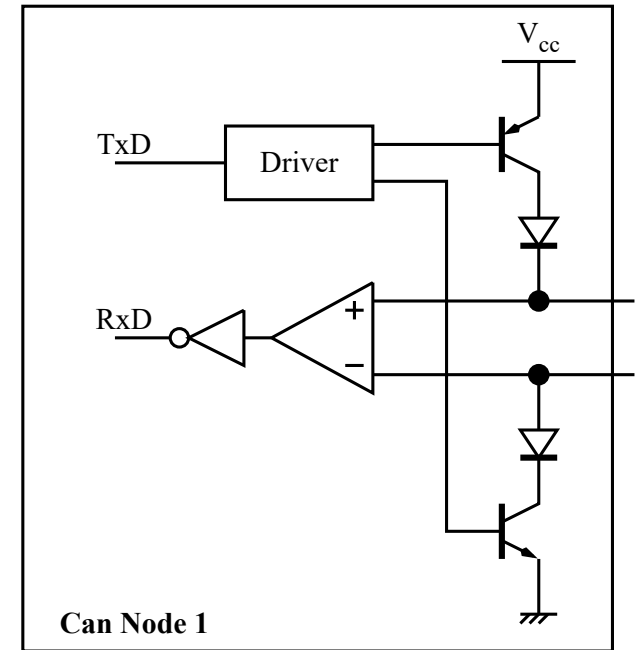
Controller Area Network (CAN)

- CAN Protocol
 - Differential signaling on two wires
 - CANH and CANL
 - The two wires are used for data transfer
 - Not one for data and the other for clock, like in I²C
 - Data encoded as the difference CANH - CANL
 - Allows for long interconnections
 - Embedded clock encoding
 - There is no dedicated wire for the clock
 - Does not suffer the problem of skew
 - Clock is recovered from data (Clock/Data Recovery, CDR)
 - Allows for fast interconnections, high bit rates

Controller Area Network (CAN)

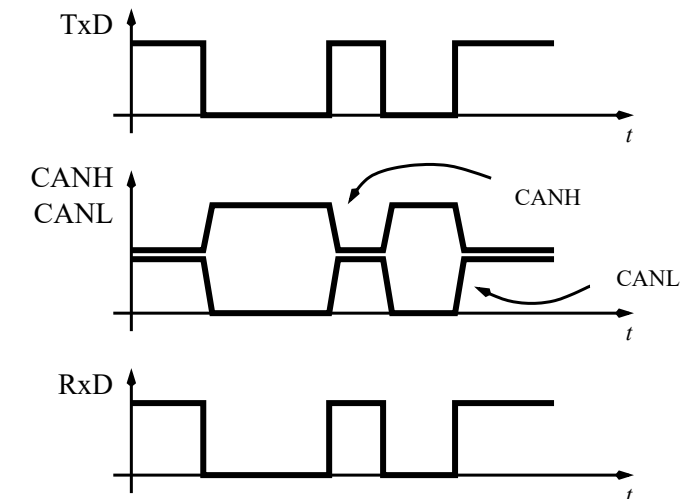
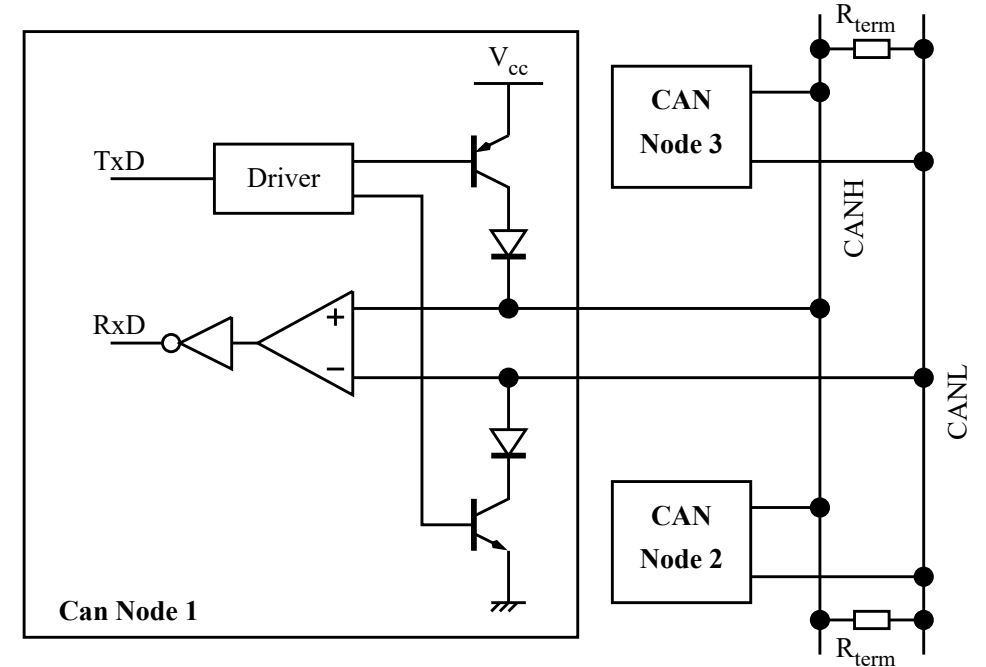
- CAN Transceiver

- Open collector driving of CANH and CANL
 - Driving: High side *pnp* for CANH, low side *nnp* for CANL
 - Reading: differential amplifier $V_{out} = \text{CANH} - \text{CANL}$
 - With an inverter at the output
- Terminating resistances between CANH and CANL
 - $R_{term} = 120\ \Omega$
 - On both sides of the bus
 - The resistance between the two CAN lines is $60\ \Omega$
 - Used to equalize the two wires voltage values when floating
 - Used to avoid reflections on the transmission line, especially for long interconnections
- Logic levels
 - Logic 0 (dominant): CANH=high (5V), CANL=low (0V)
 - Difference in reading is high, output of the input amplifier is low
 - Logic 1 (recessive): CANH=float, CANL=float
 - Difference in reading is 0 (because of the equalizing resistance), output of the input amplifier is high



Controller Area Network (CAN)

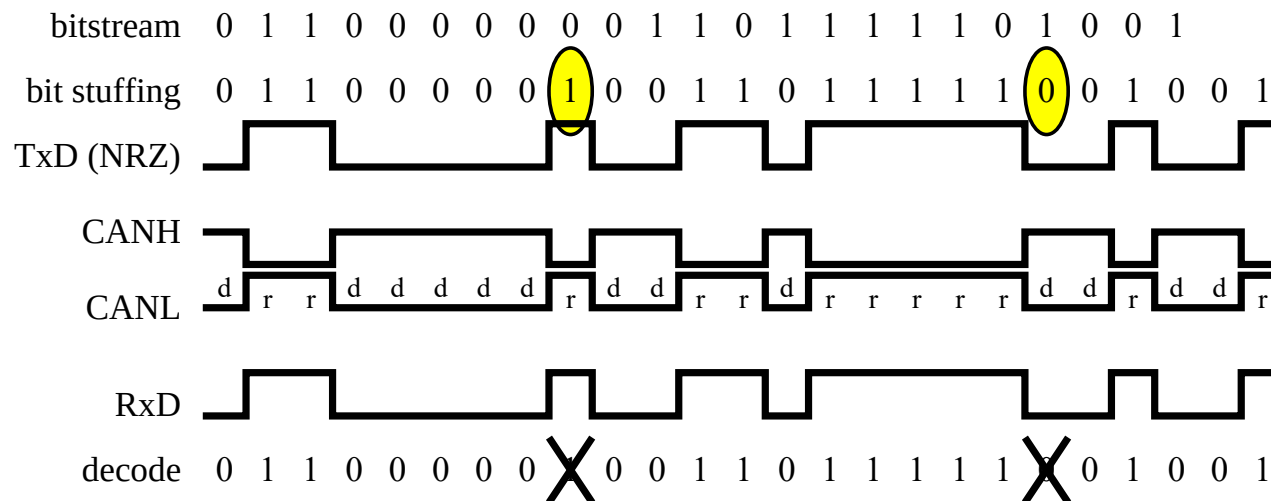
- Example of transceiver and waveforms
 - All nodes on the bus present the same interface
 - When not transmitting, each node generates a recessive bit (i.e. $TxD=1$)
 - CANH and CANL are floating
 - A dominant bit during transmission always wins over a recessive bit
 - No damages, no short circuit due to the open collector driving
- The TxD and RxD signals use NRZ encoding



Controller Area Network (CAN)

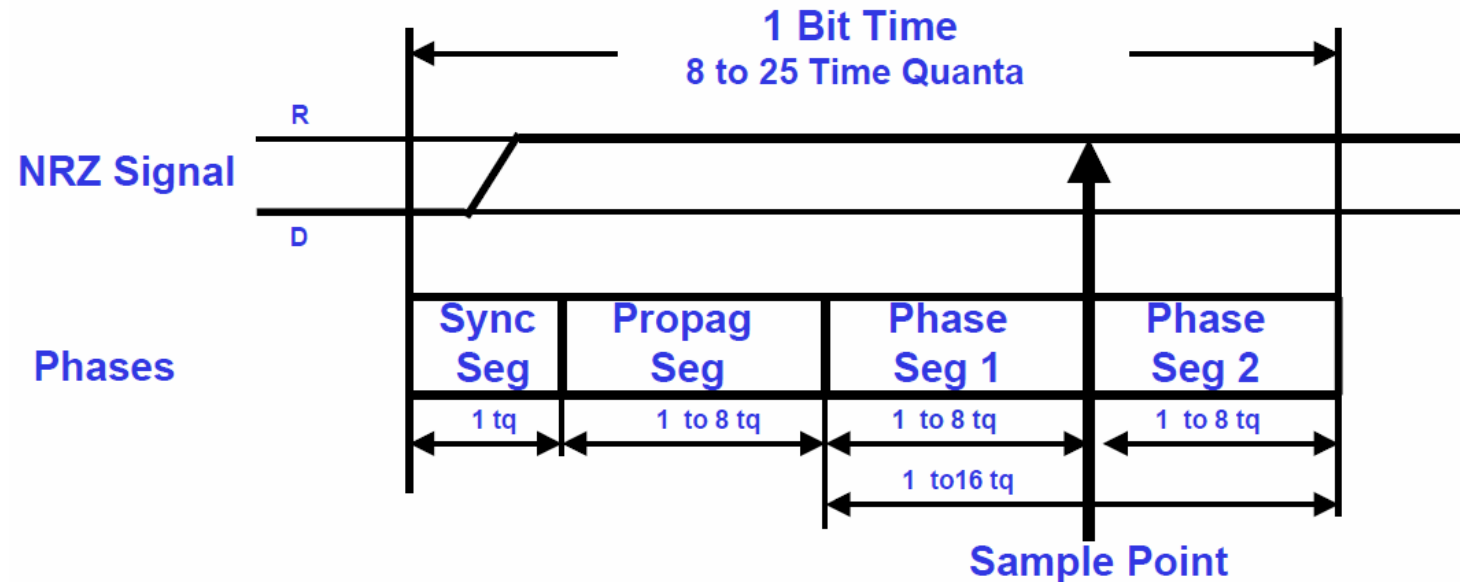
- NRZ Encoding

- No guaranteed transitions
- Bit stuffing after 5 consecutive identical bits
 - After 5 dominant (0), insert a recessive (1)
 - After 5 recessive (1), insert a dominant (0)
 - If 5 consecutive identical bits are followed by the opposite value, bit stuffing is applied anyway
 - The receiver deletes stuffed bits when decoding the incoming bitstream



Controller Area Network (CAN)

- Eye diagram
 - Bit time divided into *time quanta* (8 to 25)
 - Synch segment
 - Propagation segment
 - Phase segment 1
 - Phase segment 2
 - Sampling between Phase₁ and Phase₂ segments
 - Example (with 16tq per bit time)
 - Synch = 1tq
 - Prop = 4 tq
 - Phase₁ = 5tq
 - Phase₂ = 6tq

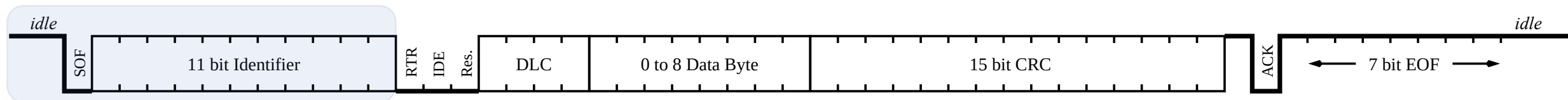


Controller Area Network (CAN)

- Synchronization
 - All devices synchronized at each recessive to dominant transition
 - Transitions assumed to happen at the beginning of a bit time
 - Each device momentarily adjusts Phase₁ and Phase₂ lengths to synch
 - All devices start in low speed
 - Speed can be modified later at run-time
- Addressing
 - When transmitting, each node generates an 11 bit address
 - Known as *identifier* in CAN
 - It is not the address of the destination node
- Arbitration
 - Similar to I²C
 - Lower identifier wins contention (dominant vs. recessive)
 - Lower identifier means higher priority

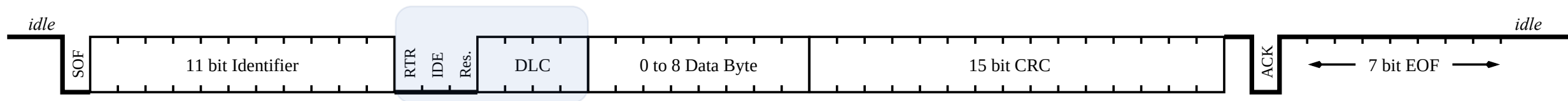
Controller Area Network (CAN)

- CAN protocol frame format (for data frames)
 - Idle, no transmission
 - Always recessive, all outputs are OFF
 - Start Of Frame (SOF)
 - A dominant bit generated by a node
 - Guarantees a recessive to dominant transition from the idle state
 - All other nodes synchronize their own clock
 - All other nodes avoid transmitting until the end of the frame
 - Identifier (address)
 - Field on 11 bits
 - Can be extended to 29 bits using IDE



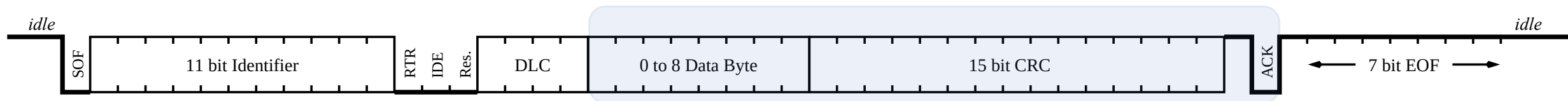
Controller Area Network (CAN)

- CAN protocol frame format (continued)
 - Remote Transmission Request (RTR)
 - Dominant for data frames
 - ID Extension (IDE)
 - Dominant for normal 11 bits identifier, recessive for 29 bits identifier
 - Reserved
 - Always dominant (but recessive is accepted at the receiver)
 - Data Length Code (DLC)
 - Number of subsequent data bytes
 - Field on 4 bits
 - Acceptable values 0 to 8



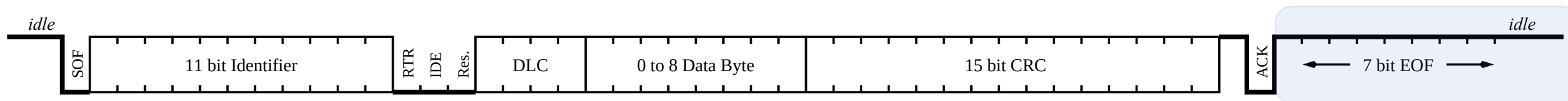
Controller Area Network (CAN)

- CAN protocol frame format (continued)
 - Data field
 - 0 to 8 bytes (depends on DLC, 0 to 64 bits)
 - Always multiples of 8 bits
 - Cyclic Redundancy Code (CRC)
 - Field on 15 bits
 - CRC delimiter
 - 1 bit, always recessive
 - Acknowledge (ACK)
 - Transmitter generates recessive
 - Receivers generate dominant



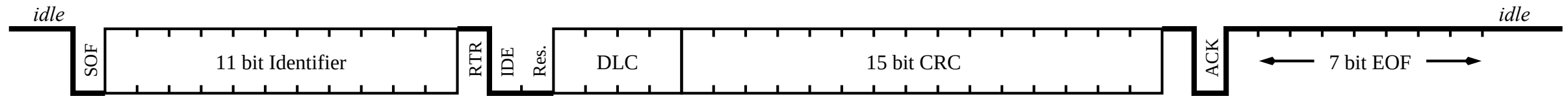
Controller Area Network (CAN)

- CAN protocol frame format (continued)
 - ACK Delimiter
 - 1 bit, always recessive
 - End Of Frame (EOF)
 - 7 consecutive bits, all recessive
 - Intentional violation of bit stuffing rule
 - Bit stuffing applied to all fields, except CRC delimiter, ACK and EOF
 - Bus returns idle
 - Always recessive
 - Other nodes can now start a transmission



Controller Area Network (CAN)

- CAN protocol frame format (RTR frame)
 - If a node requires data from another node
 - RTR frame
 - RTR bit recessive
 - DLC indicates the number of requested bytes of data
 - There is no data field



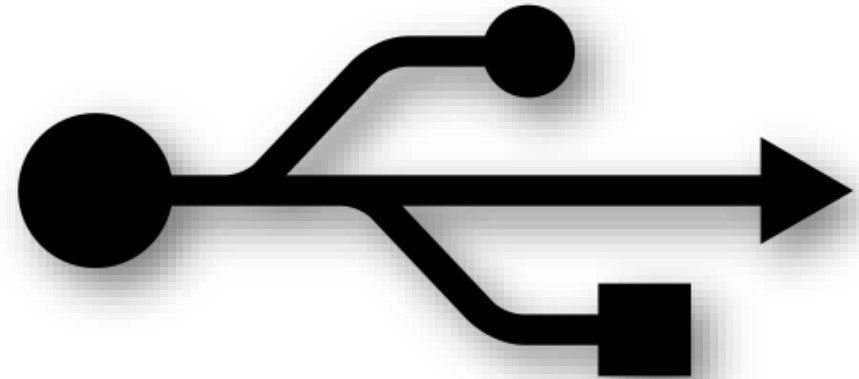
- If two or more masters start a transmission concurrently
 - Arbitration on identifier
 - Lower identifier wins
 - The other masters detect a collision when generating recessive and reading dominant
 - If equal identifier, arbitration on RTR (data frame wins), IDE (normal frame wins) or DLC

Controller Area Network (CAN)

- Higher protocol levels
 - At the electrical level, all nodes on the bus are masters
 - Any node can start a transmission, at any given time
 - Arbitration on identifiers used to manage priorities
 - At higher levels of the protocol, some node have additional tasks
 - Configure the bus and the other nodes
 - Change the communication speed, for instance
 - Detect error conditions
 - Request retransmissions
 - Manage the data exchange
 - Several different higher level protocols on CAN physical layer

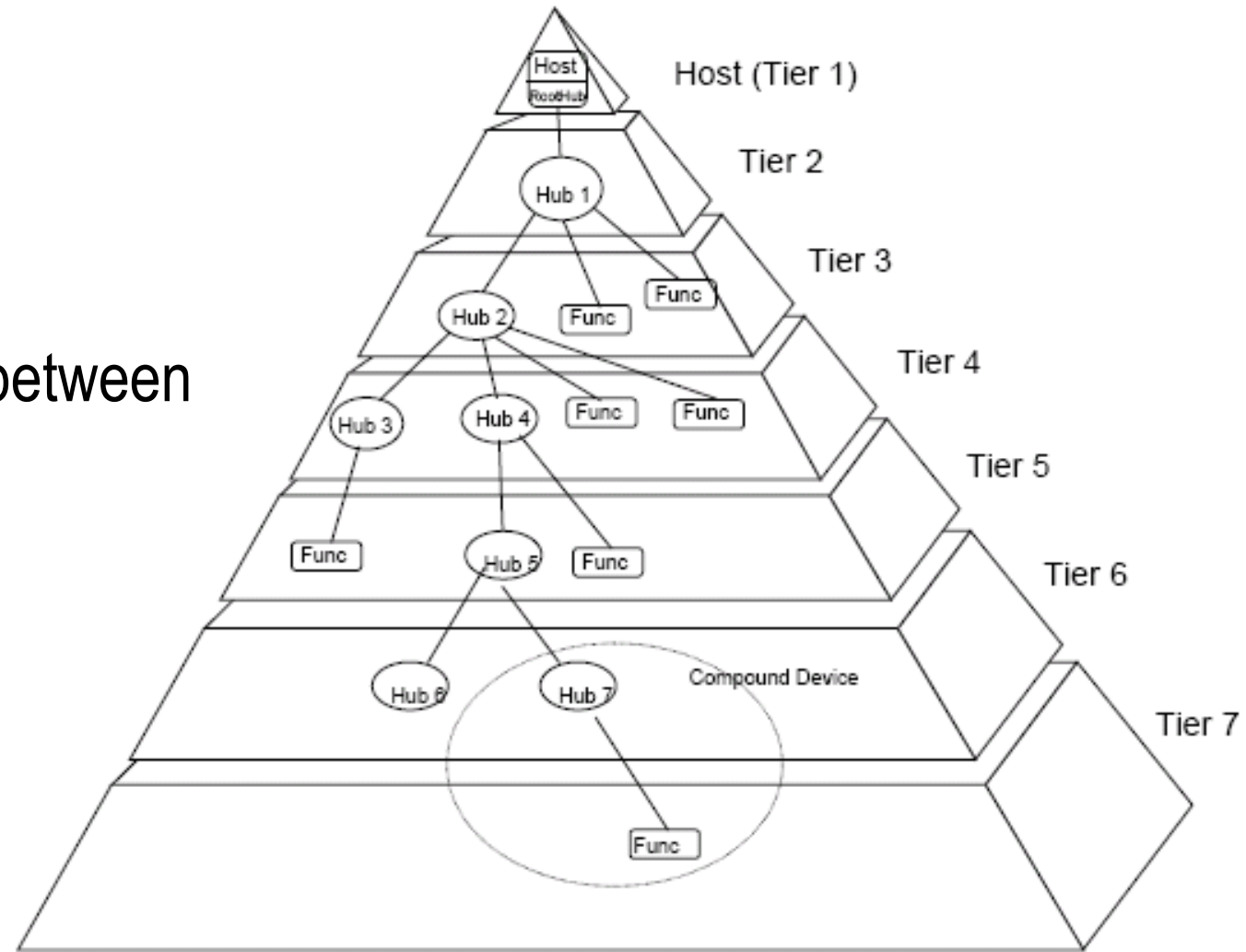
Universal Serial Bus (USB)

- Substitutes several kind of legacy connections
 - RS232, Centronics, PS/2, ...
- Hot Plug&Play
- Three speeds in USB 2.0
 - Low speed = 1.5 Mbps
 - Full speed = 12 Mbps
 - High speed = 480 Mbps
- USB 3.0 (2008) and 3.1 (2013) enhancements
 - SuperSpeed = 5 Gbps
 - 8B10B Encoding
 - SuperSpeed+ = 10 Gbps
 - 128B132B Encoding



Universal Serial Bus (USB)

- USB Network has a tree structure
 - Single master
 - Up to 127 slaves
 - Up to 7 levels
- Always point-to-point communication between the single master and a slave
 - Hubs virtualize connections
- Nomenclature
 - Master = Host
 - Slave = Function, Device

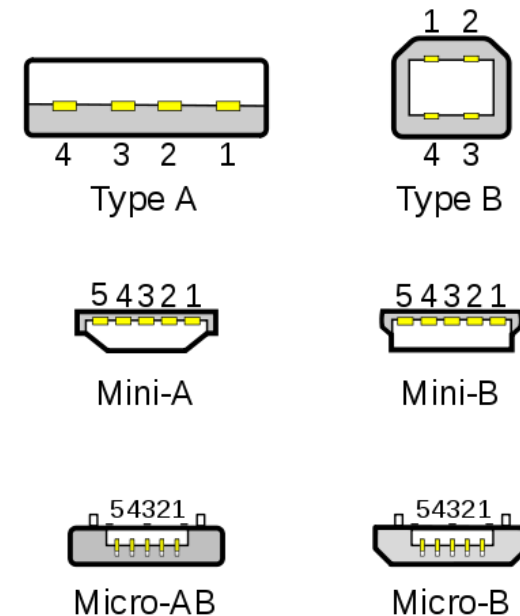
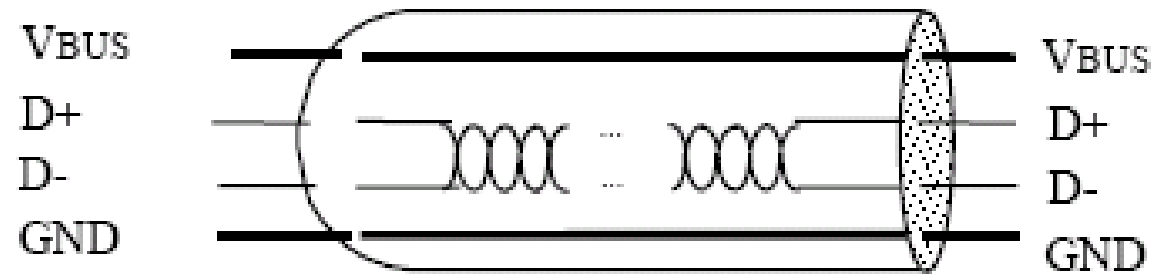


Universal Serial Bus (USB)

- USB standard defines the behavior at several abstraction levels
 - Mechanical
 - Electrical
 - Protocol
 - Transactions
 - Multiple transfers
 - Driver
 - Application and managing software

Universal Serial Bus (USB)

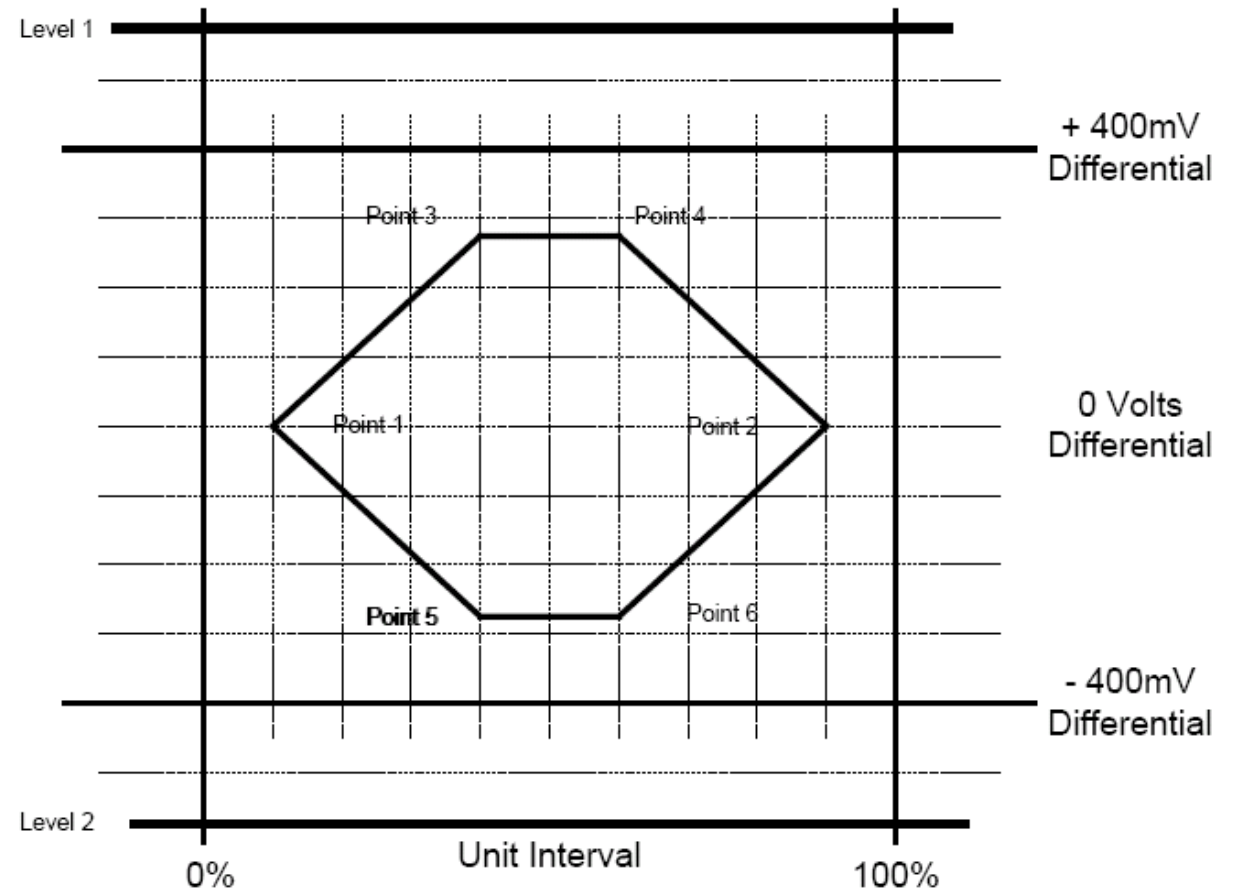
- Mechanical standard
 - 4 wires
 - Two for power supply
 - V_{BUS} , GND
 - 5V, 500mA
 - Two for signal (differential)
 - D+, D-
 - Shielded twisted pair
- Several kind of connectors
 - Host and function
 - Miniconnectors
 - Digital cameras, mini hard disk, mobile phones



Universal Serial Bus (USB)

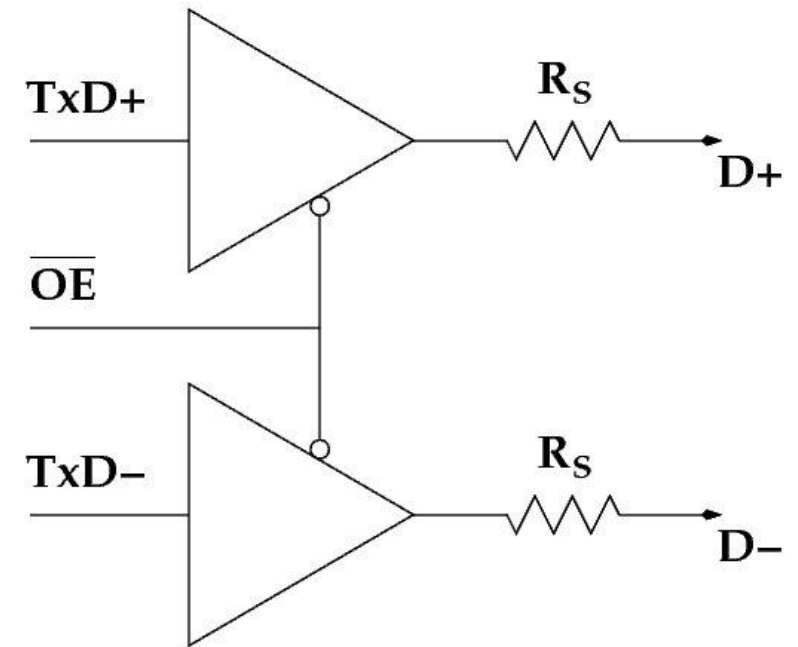
- Electrical standard
 - Differential coding
 - Voltage levels for low speed and full speed
 - Currents for high speed
 - 17.78mA on 45 Ω resistors
 - Output = D+ – D-
 - Line Impedance $Z_{\infty} = 90 \Omega$
 - Standard defines the eye diagram

	Voltage Level (D+ - D-)	Time (% of Unit Interval)
Level 1	575 mV	N/A
Level 2	-575 mV	N/A
Point 1	0 V	10% UI
Point 2	0 V	90% UI
Point 3	275 mV	40% UI
Point 4	275 mV	60% UI
Point 5	-275 mV	40% UI
Point 6	-275 mV	60% UI



Universal Serial Bus (USB)

- Electrical standard
 - Driver full speed
 - CMOS Buffer with low output resistance
 - R_S to adapt
 - full-speed in USB 1.1: $R_S = 28\ \Omega \div 44\ \Omega$
 - full-speed in USB 2.0: $R_S = 40.5\ \Omega \div 49.5\ \Omega$
 - Driver high speed
 - Current in only one wire
 - D+ (J symbol), output = 400 mV
 - D- (K symbol), output = -400 mV



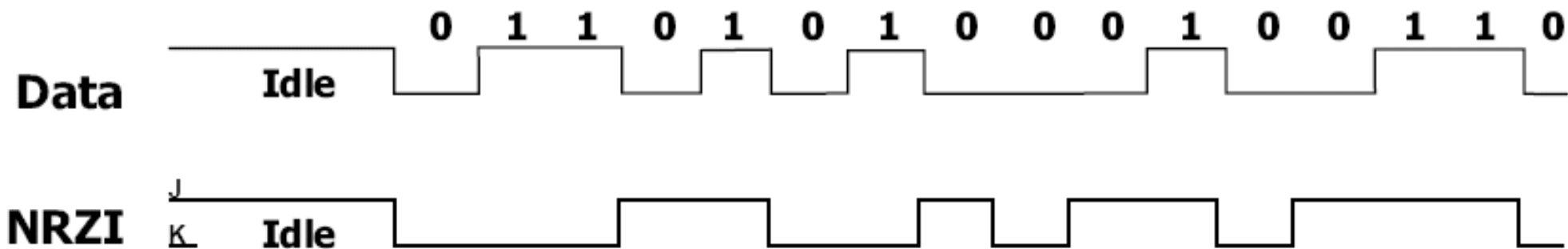
Universal Serial Bus (USB)

- Electrical standard
 - Levels for low and full speed

Bus State	Signaling Levels		
	At originating source connector (at end of bit time)	At final target connector	
		Required	Acceptable
Differential "1"	$D+ > V_{OH}(\text{min})$ and $D- < V_{OL}(\text{max})$	$(D+) - (D-) > 200 \text{ mV}$ and $D+ > V_{IH}(\text{min})$	$(D+) - (D-) > 200 \text{ mV}$
Differential "0"	$D- > V_{OH}(\text{min})$ and $D+ < V_{OL}(\text{max})$	$(D-) - (D+) > 200 \text{ mV}$ and $D- > V_{IH}(\text{min})$	$(D-) - (D+) > 200 \text{ mV}$
Single-ended 0 (SE0)	$D+ \text{ and } D- < V_{OL}(\text{max})$	$D+ \text{ and } D- < V_{IL}(\text{max})$	$D+ \text{ and } D- < V_{IH}(\text{min})$
Single-ended 1 (SE1)	$D+ \text{ and } D- > V_{OSE1}(\text{min})$	$D+ \text{ and } D- > V_{IL}(\text{max})$	
Data J state: Low-speed Full-speed	Differential "0" Differential "1"	Differential "0" Differential "1"	
Data K state: Low-speed Full-speed	Differential "1" Differential "0"	Differential "1" Differential "0"	

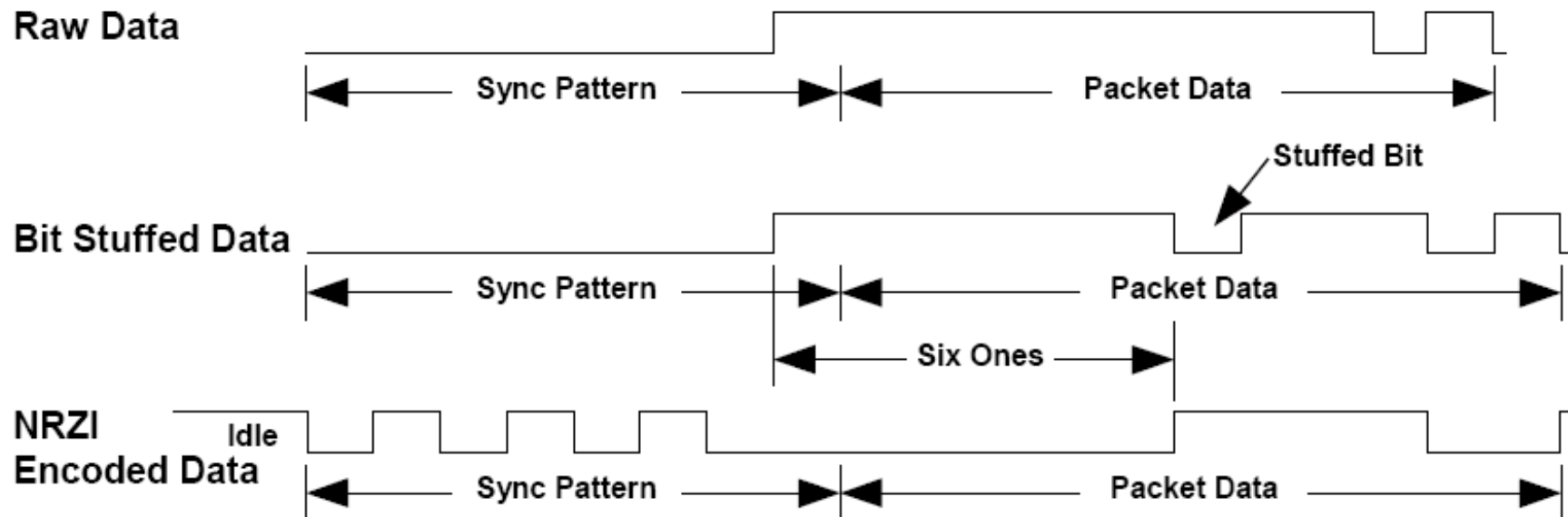
Universal Serial Bus (USB)

- Electrical standard
 - NRZI coding
 - Logic 0: transition (at the beginning of a bit)
 - Logic 1: maintains previous value
 - Symbols: J and K
 - See previous table for low and full speed
 - D+ and D- in high speed
 - Single ended symbols
 - SE0: D+ = D- = V_{OL}
 - SE1: D+ = D- = V_{OH}



Universal Serial Bus (USB)

- Electrical standard
 - Synchronization
 - Serial synchronous communication with embedded clock
 - PLL at termination locks the clock frequency of the transmitted signal
 - Long logic 1 sequences prevent clock recovery
 - Bit stuffing: every 6 logic 1, a logic 0 is inserted

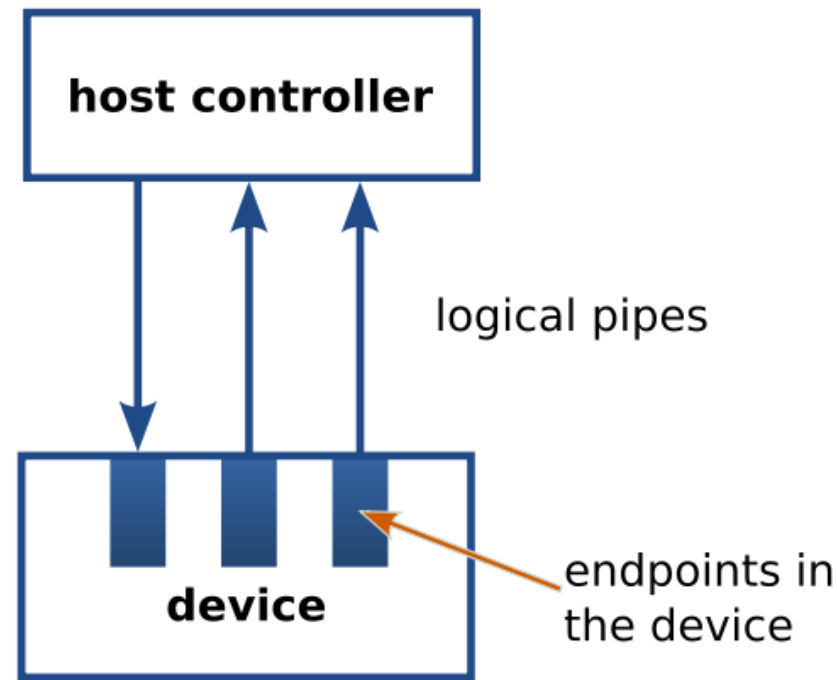


Universal Serial Bus (USB)

- USB Protocol
 - Single master → no need of arbitration
 - Each function (FUN) is assigned a unique address
 - Between $1 \text{ e } 127$
 - Assigned when it is initially plugged in to the network
 - Each FUN provides a maximum of 32 (16+16) end-points
 - Indexes 0 to 15
 - For each index, there can be an end-point for reading, and another for writing
 - 16 reading end-points
 - 16 writing end-points
 - Each end-point is unidirectional
 - End-point 0 always present

Universal Serial Bus (USB)

- USB Protocol
 - Communication on USB based on *pipes* (logical channels)
 - Connection between the host controller (SW) and an end-point in the device
 - For each pipe it can be defined:
 - Data bandwidth
 - Type of transfer
 - Buffer sizes
 - ...

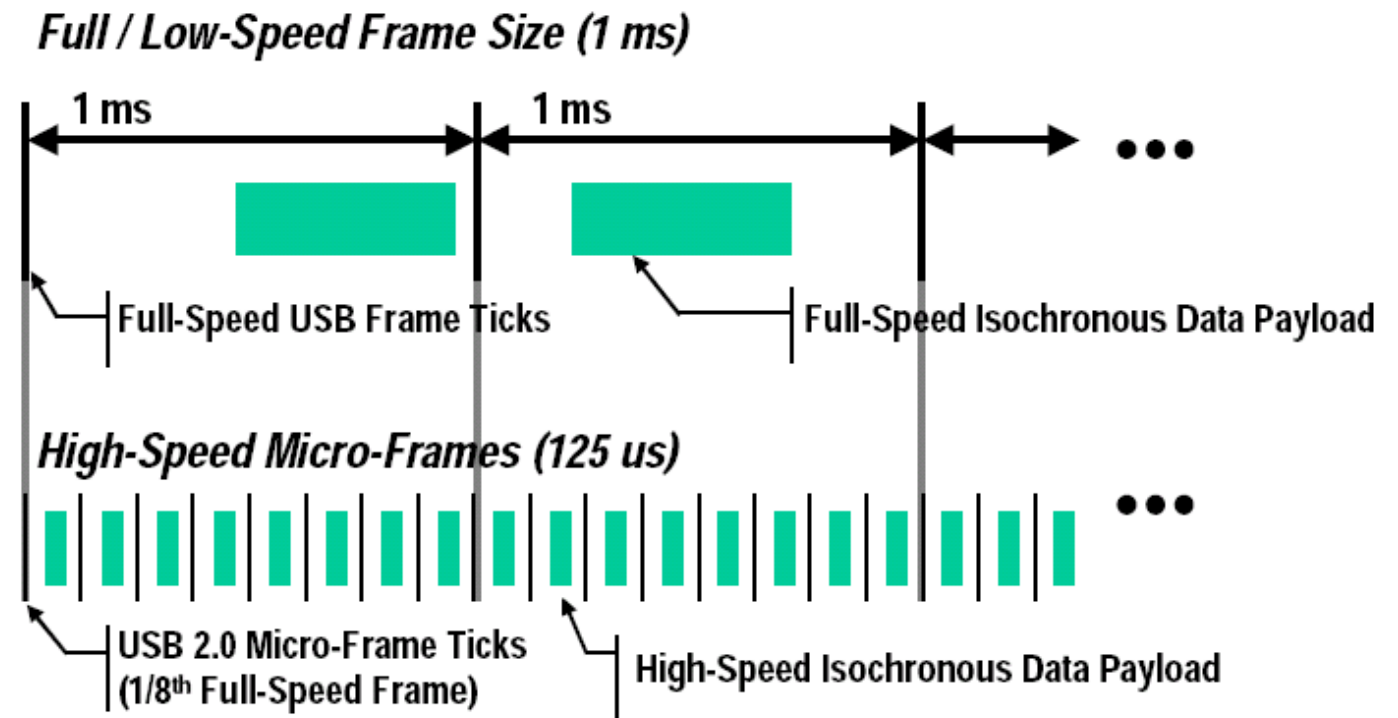


Universal Serial Bus (USB)

- USB Protocol
 - Pipe can be of *stream* or *message* type
 - Stream pipe
 - *Isochronous*
 - Guaranteed bandwidth, possibly losing data (audio, video, ...)
 - *Interrupt*
 - Quick answer and guaranteed latency (keyboard, mouse, ...)
 - *Bulk*
 - Large sporadic transfers, served using a best-effort bandwidth (file transfers, ...)
 - Message pipe
 - *Control*
 - Short commands to devices and status messages (configuration on end-point 0, ...)

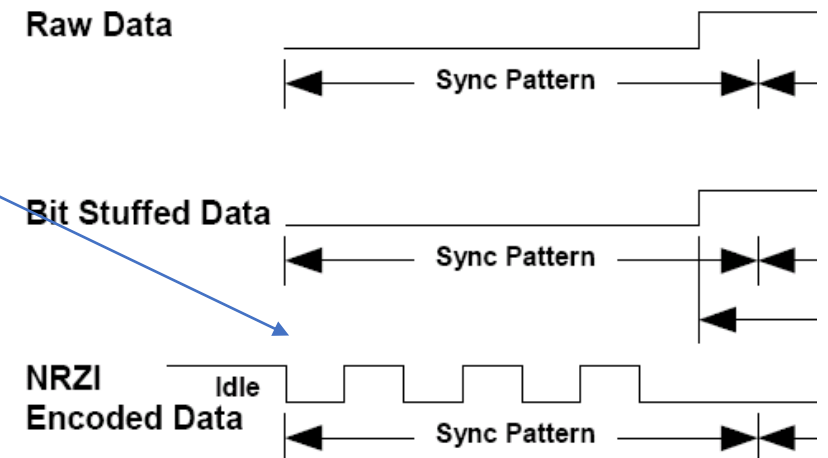
Universal Serial Bus (USB)

- USB Protocol
 - Each communication on a pipe is divided in one or more transactions
 - Communications happen within a frame:
 - Low speed and full speed: 1 ms
 - High speed: 125 μ s
 - Within a frame there can be multiple transactions
 - Part of the frame time is dedicated to certain kind of transactions
 - Guarantees latency and throughput



Universal Serial Bus (USB)

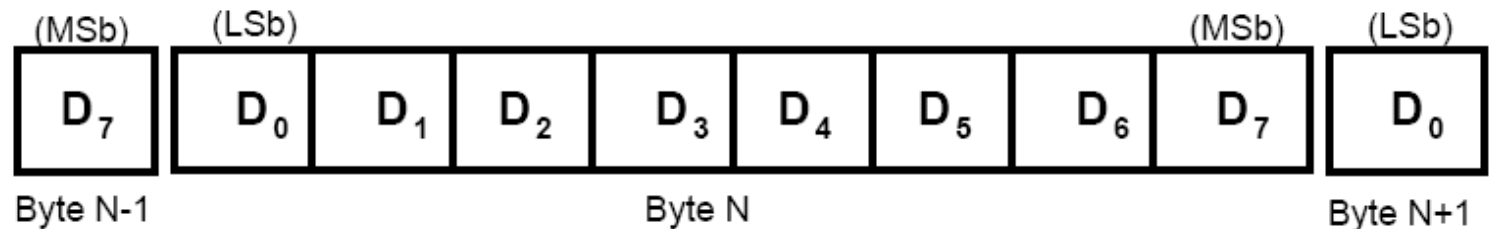
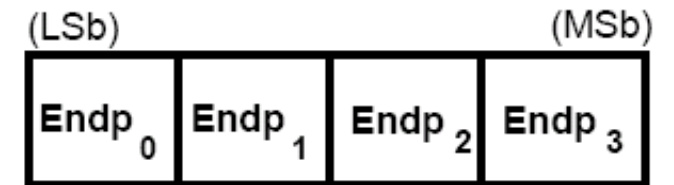
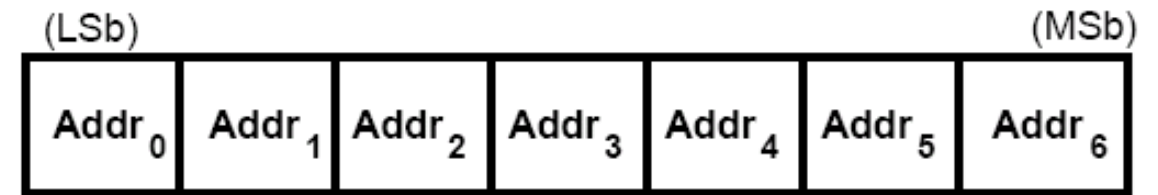
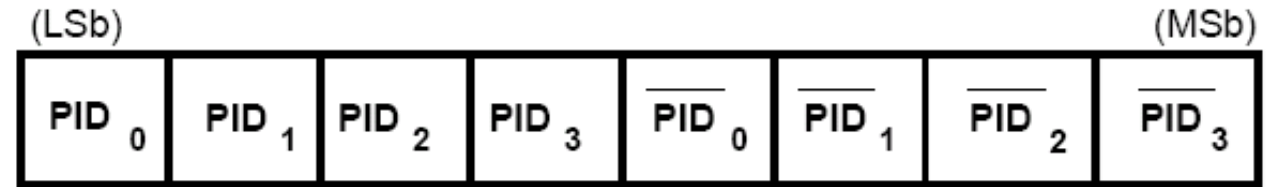
- USB Protocol
 - Each transaction is divided into packets
 - Each packet starts with a SYNC sequence and terminates with an EOP
 - SYNC
 - Large number of transitions to lock the Phase Lock Loop
 - 3 KJ followed by 2 K in low and full speed (8 symbols)
 - 15 KJ followed by 2 K in high speed (32 symbols)
 - EOP
 - Single-ended signal (low and full speed)
 - Intentional violation of the bit stuffing rule (high speed)
 - Each packet is divided into fields
 - PID, Address, End-point, Frame, Data, CRC
 - The type of packet is defined by the field called PID



Universal Serial Bus (USB)

- USB Protocol Field structure

- PID: 4+4 bit
- Address: 7 bit
- End-point: 4 bit
- Frame: 11 bit counter
- Data: up to 1024 byte
 - Groups of 8 bits
- CRC:
 - Token packet: 5 bit
 - Data: 16 bit



Universal Serial Bus (USB)

- USB Protocol Types of Packet
 - Token
 - Start of a communication (addressing)
 - Setup of a device
 - Start of a frame
 - Data
 - Input or Output
 - DATA0, DATA1 and DATA2
 - Handshake
 - Acknowledge and No-Acknowledge
 - Special
 - Error and Split

Universal Serial Bus (USB)

- USB Protocol Types of Packet
 - There are 16 distinct packet types
 - Divided in 4 classes
 - PID specifies the packet type
 - PID always follows a SYNC
 - PID<1:0> determines the packet class
 - The type of the packet also determines its format

PID Type	PID Name	PID<3:0>*	Description
Token	OUT	0001B	Address + endpoint number in host-to-function transaction
	IN	1001B	Address + endpoint number in function-to-host transaction
	SOF	0101B	Start-of-Frame marker and frame number
	SETUP	1101B	Address + endpoint number in host-to-function transaction for SETUP to a control pipe
Data	DATA0	0011B	Data packet PID even
	DATA1	1011B	Data packet PID odd
	DATA2	0111B	Data packet PID high-speed, high bandwidth isochronous transaction in a microframe (see Section 5.9.2 for more information)
	MDATA	1111B	Data packet PID high-speed for split and high bandwidth isochronous transactions (see Sections 5.9.2, 11.20, and 11.21 for more information)

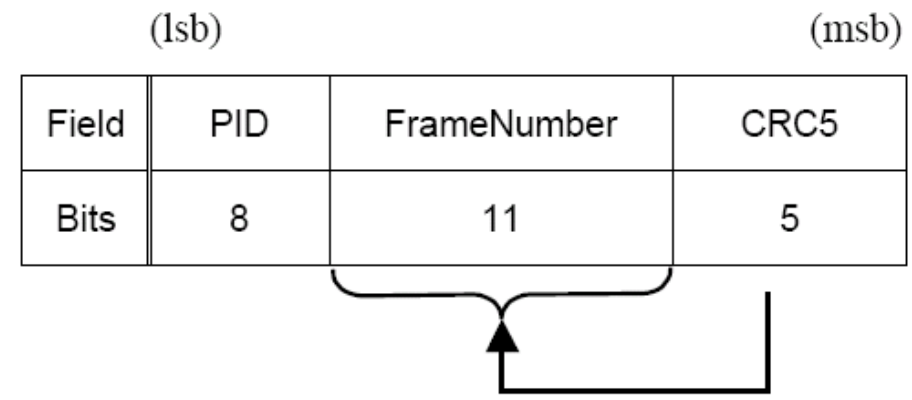
Universal Serial Bus (USB)

- USB Protocol Types of Packet
 - There are 16 distinct packet types
 - Divided in 4 classes
 - PID specifies the packet type
 - PID always follows a SYNC
 - PID<1:0> determines the packet class
 - The type of the packet also determines its format

PID Type	PID Name	PID<3:0>*	Description
Handshake	ACK	0010B	Receiver accepts error-free data packet
	NAK	1010B	Receiving device cannot accept data or transmitting device cannot send data
	STALL	1110B	Endpoint is halted or a control pipe request is not supported
	NYET	0110B	No response yet from receiver (see Sections 8.5.1 and 11.17-11.21)
Special	PRE	1100B	(Token) Host-issued preamble. Enables downstream bus traffic to low-speed devices.
	ERR	1100B	(Handshake) Split Transaction Error Handshake (reuses PRE value)
	SPLIT	1000B	(Token) High-speed Split Transaction Token (see Section 8.4.2)
	PING	0100B	(Token) High-speed flow control probe for a bulk/control endpoint (see Section 8.5.1)
	Reserved	0000B	Reserved PID

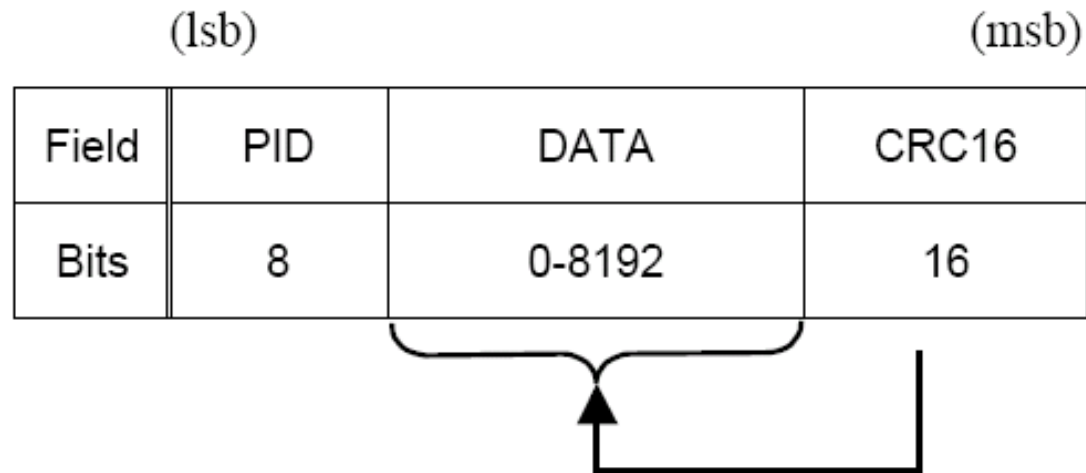
Universal Serial Bus (USB)

- USB Protocol Token Packet
 - Always generated by the master
 - Specifies the device and end-point to communicate with (Rx or Tx)
 - PID + ADDR + ENDP + CRC5
 - $8 + 7 + 4 + 5 \text{ bit} = 3 \text{ byte}$
 - Specifies the start of a frame
 - PID + FRAME + CRC5 = $8 + 11 + 5 \text{ bit} = 3 \text{ byte}$



Universal Serial Bus (USB)

- USB Protocol Data Packet
 - Generated by Host or Fun
 - Contains communication data
 - Preceded by a token packet to specify who shall transmit or receive



Universal Serial Bus (USB)

- USB Protocol Handshake and Special Packet
 - Report the transmission status
 - ACK: no errors in communication (CRC and bit-stuffing)
 - NAK: cannot receive (busy) or nothing to transmit
 - STALL: FUN needs servicing (reset)
 - ERR: transmission error (CRC or bit-stuffing)

(lsb) (msb)

Field	PID
Bits	8

Universal Serial Bus (USB)

- USB Protocol Bulk Transaction

- Bulk packets sent only when there is time (best-effort)

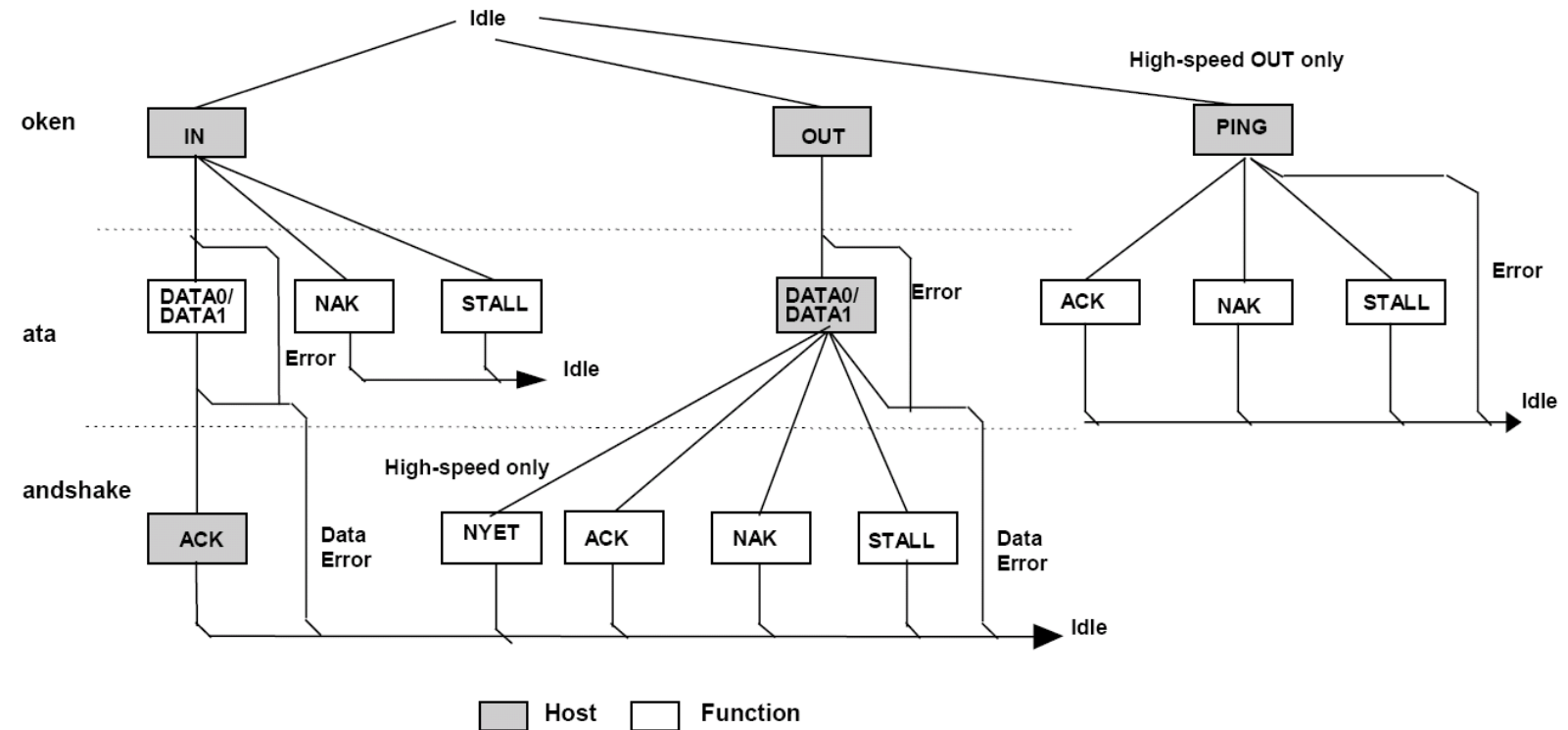
- Latency not guaranteed
- Throughput not guaranteed

- Data packet limited to 512 bytes maximum payload

- Use multiple bulk transactions to transfer more data

- Error checking through handshake between transmitter and receiver

- Retry in case of an error (no error correction)



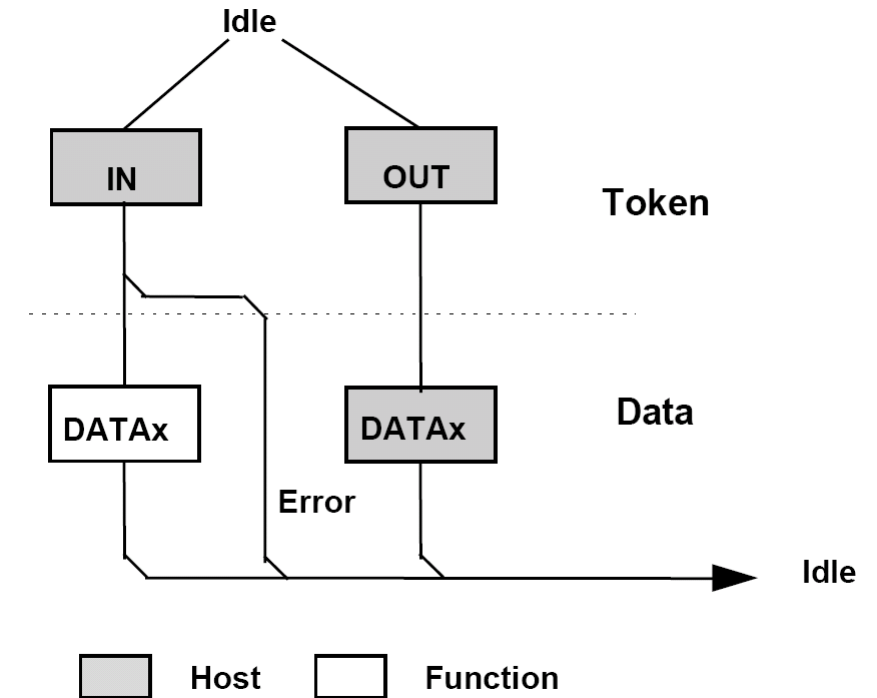
Universal Serial Bus (USB)

- USB Protocol Data Toggle Synchronization
 - Error checking and synchronization also use the alternation of some PIDs
 - DATA0
 - DATA1
 - DATA2 for high speed / high bandwidth
 - The transmitter alternatively generates
 - Data packets with PID=DATA0
 - Data packets with PID=DATA1
 - An FSM at the receiver side verifies the correct PID toggling

Universal Serial Bus (USB)

- USB Protocol Isochronous Transaction

- Guaranteed latency and throughput
- Only in full speed and in high speed
 - Full speed: up to 90% of frame time reserved for periodic transactions (isochronous and interrupt)
 - High speed: up to 80% of frame time reserved for periodic transactions (isochronous and interrupt)
- Each end-point specifies how often it wants to send or receive an isochronous packet
 - Full speed: no more than 1 isochronous packet (maximum 1024 bytes) per frame per each end-point
 - 1024 byte per frame $\rightarrow 1024 \text{ byte} / 1 \text{ ms} = 1 \text{ MBytes/s}$
 - High speed: no more than 3 isochronous packet (maximum 1024 bytes each) per frame per each end-point
 - 1024 byte per frame $\rightarrow 1024 \text{ byte} / 125 \mu\text{s} = 8 \text{ MBytes/s}$
 - 3072 byte per frame $\rightarrow 3072 \text{ byte} / 125 \mu\text{s} = 24 \text{ MBytes/s}$



Universal Serial Bus (USB)

- USB Protocol Isochronous Transaction
 - Full speed
 - Clock at 12MHz, Frame = 1ms, 90% isochronous
 - How many isochronous transactions can be transmitted?
 - Bits in a frame: $12.000.000 * 0.001 = 12.000$ bits
 - Bytes reserved: $12.000 * 0.9 = 10.800$ bits = 1350 bytes
 - Only one maximum length isochronous transaction will fit
 - Packets can be of variable length, and each end-point specifies which is the maximum acceptable length
 - High speed
 - Clock at 480MHz, Frame = 125 μ s, 80% isochronous
 - Bits in a frame: $480.000.000 * 125 \cdot 10^{-6} = 60.000$ bits
 - Byte reserved: $60.000 * 0.8 = 48.000$ bits = 6.000 bytes
 - No more than 5 maximum length guaranteed isochronous transactions
 - No more than 3 for a given end-point
 - Use multiple end-points on the same device to increase performance
 - Up to 6 isochronous transactions in a frame if no other transaction (i.e. bulk) is pending

Universal Serial Bus (USB)

- USB Protocol Isochronous Transaction

- Maximum limits in Full Speed Isochronous Transactions
 - Best performance obtained with
 - 20 transactions of 64 bytes
 - 10 transactions of 128 bytes
 - 5 transactions of 256 bytes
 - Shorter transactions suffer of too much overhead
 - Longer transactions do not fill the frame time optimally

Table 5-4. Full-speed Isochronous Transaction Limits

Protocol Overhead (9 bytes)		(2 SYNC bytes, 2 PID bytes, 2 Endpoint + CRC bytes, 2 CRC bytes, and a 1-byte interpacket delay)			
Data Payload	Max Bandwidth(bytes/second)	Frame Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/Frame Useful Data
1	150000	1%	150	0	150
2	272000	1%	136	4	272
4	460000	1%	115	5	460
8	704000	1%	88	4	704
16	960000	2%	60	0	960
32	1152000	3%	36	24	1152
64	1280000	5%	20	40	1280
128	1280000	9%	10	130	1280
256	1280000	18%	5	175	1280
512	1024000	35%	2	458	1024
1023	1023000	69%	1	468	1023
Max	1500000				1500

Universal Serial Bus (USB)

- USB Protocol Isochronous Transaction
 - Maximum limits in High Speed Isochronous Transactions
 - Best performance obtained with
 - 7 transactions of 1024 bytes
 - Shorter transactions suffer of too much overhead
 - Longer transactions do not fill the frame time optimally

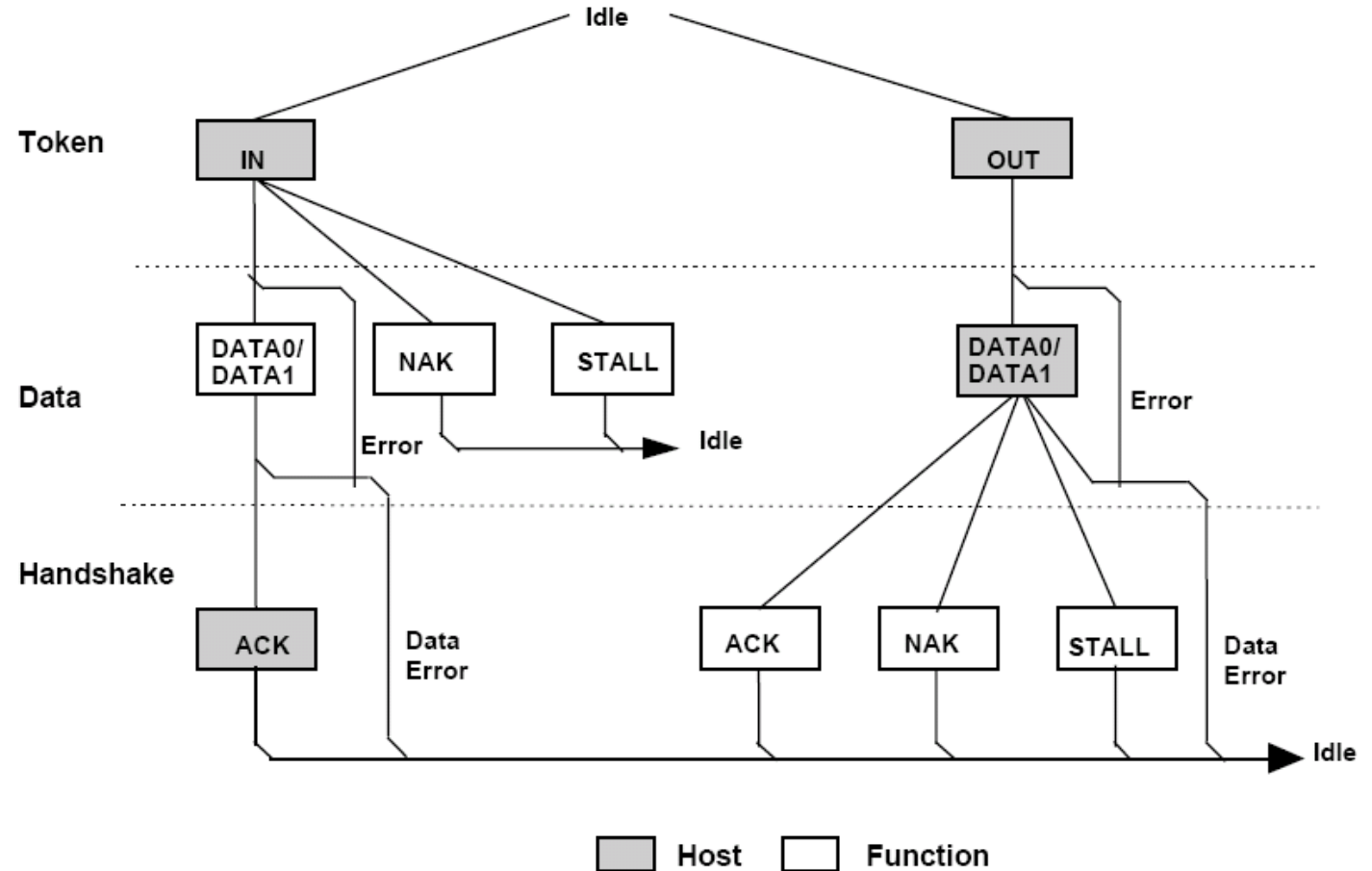
Table 5-5. High-speed Isochronous Transaction Limits

Protocol Overhead		(Based on 480Mb/s and 8 bit interpacket gap, 88 bit min bus turnaround, 32 bit sync, 8 bit EOP: (2x4 SYNC bytes, 2 PID bytes, 2 EP/ADDR+addr+CRC5, 2 CRC16, and a 2x(1+11)) byte interpacket delay (EOP, etc.))			
Data Payload	Max Bandwidth (bytes/second)	Microframe Bandwidth per Transfer	Max Transfers	Bytes Remaining	Bytes/ MicroFrame Useful Data
1	1536000	1%	192	12	192
2	2992000	1%	187	20	374
4	5696000	1%	178	24	712
8	10432000	1%	163	2	1304
16	17664000	1%	138	48	2208
32	27392000	1%	107	10	3424
64	37376000	1%	73	54	4672
128	46080000	2%	45	30	5760
256	51200000	4%	25	150	6400
512	53248000	7%	13	350	6656
1024	57344000	14%	7	66	7168
2048	49152000	28%	3	1242	6144
3072	49152000	41%	2	1280	6144
Max	60000000				7500

Universal Serial Bus (USB)

- USB Protocol Interrupt Transaction

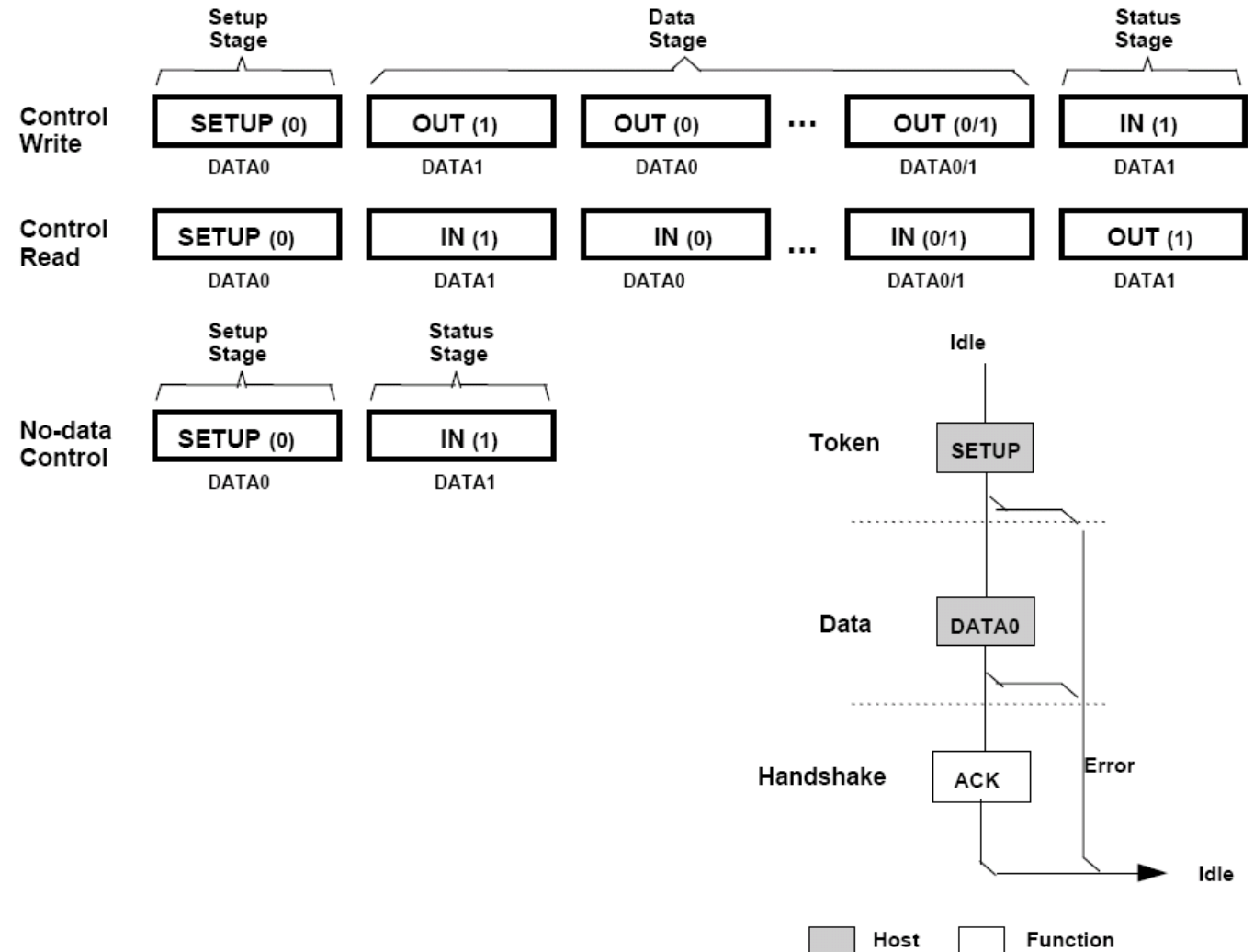
- Guaranteed as isochronous transactions
 - Available in low speed as well
- Error checking as in bulk transactions
- Does not implement a real interrupt
 - USB is a polled bus



Universal Serial Bus (USB)

- USB Protocol Control Transaction

- Fixed structure
 - Defined by the USB standard
 - SETUP (see figure)
 - Data IN or OUT
 - Optional
 - Similar to bulk
- Status



Universal Serial Bus (USB)

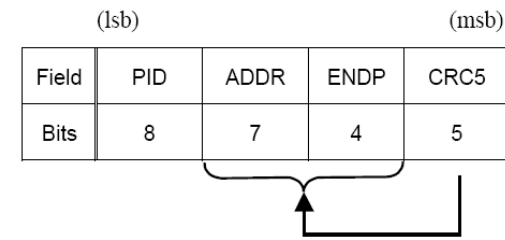
- USB Protocol Example
 - Transfer a 6656 bytes (6.5 Kbytes) file to a device
 - Bulk transactions
 - No guaranteed performance
 - Error checking
 - Video streaming from a camera
 - VGA, color (8-bit per color, RGB), uncompressed, 30 fps
 - Number of pixels per video frame: $640 \cdot 480 = 307200$ pixels/video frame
 - Number of bytes per video frame: $307200 \cdot 3 = 921600$ bytes/video frame
 - Number of bits per video frame: $921600 \cdot 8 = 7372800$ bits/video frame
 - Number of bits per second: $7372800 \cdot 30 = 221184000$ bps $\simeq 211$ Mbps
 - Number of USB frames per second: $1/0.000125 = 8000$ USBframes/s
 - Average number of required useful bits per USB frame: $221184000/8000 = 27648$ bits/USBframe
 - Average number of required useful bytes per USB frame: $27648/8 = 3456$ bytes/USBframe

Universal Serial Bus (USB)

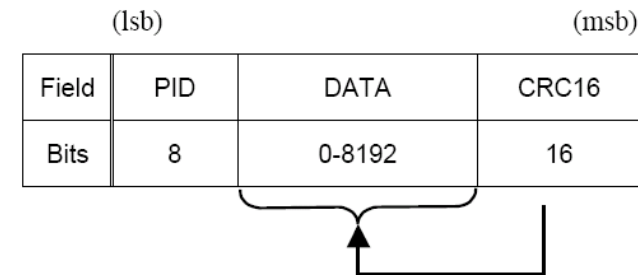
- USB Protocol Example
 - Video streaming (continued)
 - Average number of required useful bytes per USB frame: $27648/8 = 3456$ bytes/USBframe
 - Isochronous transactions
 - No error checking
 - Possible solution
 - 27 isochronous transactions of 128 data bytes each in a frame
 - Total number of data bytes: $27 \cdot 128 = 3456$ bytes
 - Other working solutions
 - 54 iso transactions of 64 data bytes
 - 13 iso transactions of 256 data bytes and 1 iso transaction of 128 bytes
 - 6 iso transactions of 512 data bytes and 1 iso transaction of 384 bytes
 - Other non working solutions
 - 108 iso transactions of 32 data bytes (does not fit in a frame)

Universal Serial Bus (USB)

- USB Protocol Example
 - Overhead in isochronous transaction
 - 4 bytes SYNC before the IN token
 - 1 byte PID for the IN token
 - 2 bytes for ADDR – ENDP – CRC5
 - 1 byte for EOP after the IN token
 - 11 bytes turnaround time
 - 4 bytes SYNC before the data packet
 - 1 byte PID for the data packet
 - (128 bytes of useful data)
 - 2 bytes CRC16
 - 1 byte for EOP after the data packet
 - 11 bytes turnaround time
 - → 38 bytes



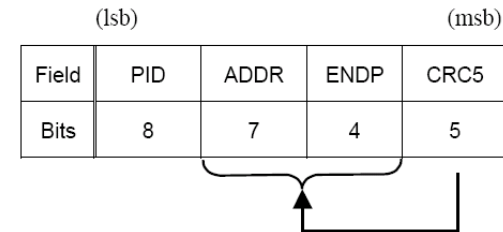
IN token
PID = 1001_2



Data packet
PID DATA0 = 0011_2
PID DATA1 = 1011_2

Universal Serial Bus (USB)

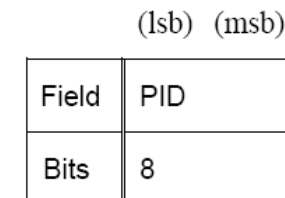
- USB Protocol Example
 - Total for an Isochronous transaction
 - $38 + 128 = 166$ bytes
 - Overhead in bulk transaction
 - Similar to isochronous transaction
 - OUT token instead of IN token
 - 512 bytes of useful payload in data packet
 - 4 bytes SYNC before the ACK token
 - 1 byte PID for the ACK token
 - 1 byte for EOP after the ACK token
 - 11 bytes turnaround time
 - $\rightarrow 38 + 17 = 55$ bytes
 - Total for a Bulk transaction
 - $55 + 512 = 567$ bytes



OUT token
PID = 0001_2



Data packet
PID DATA0 = 0011_2
PID DATA1 = 1011_2



ACK token
PID = 0010_2

Universal Serial Bus (USB)

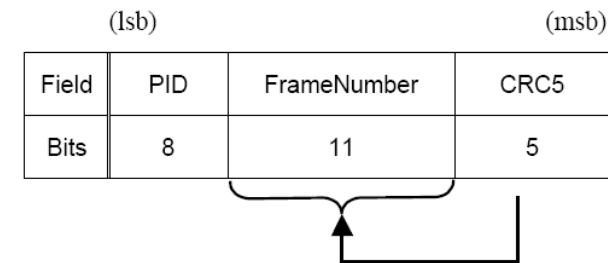
- USB Protocol Example

- Start of Frame

- 4 bytes for SYNC before SOF token
 - 1 byte for PID for the SOF token
 - 2 bytes for FRAMENUMBER – CRC5
 - 1 byte for EOP after the SOF token
 - 11 bytes turnaround time
 - → 19 bytes

- Space available in a USB frame

- $480 \cdot 10^6 / 8000 = 60000 \text{ bits/USBframe} = 7500 \text{ bytes/USBframe}$
 - Guaranteed space for Isochronous transactions (80% of total USB frame)
 - $7500 \cdot 0.8 = 6000 \text{ bytes}$
 - 27 Isochronous transactions occupy
 - $27 \cdot 166 = 4482 \text{ bytes}$



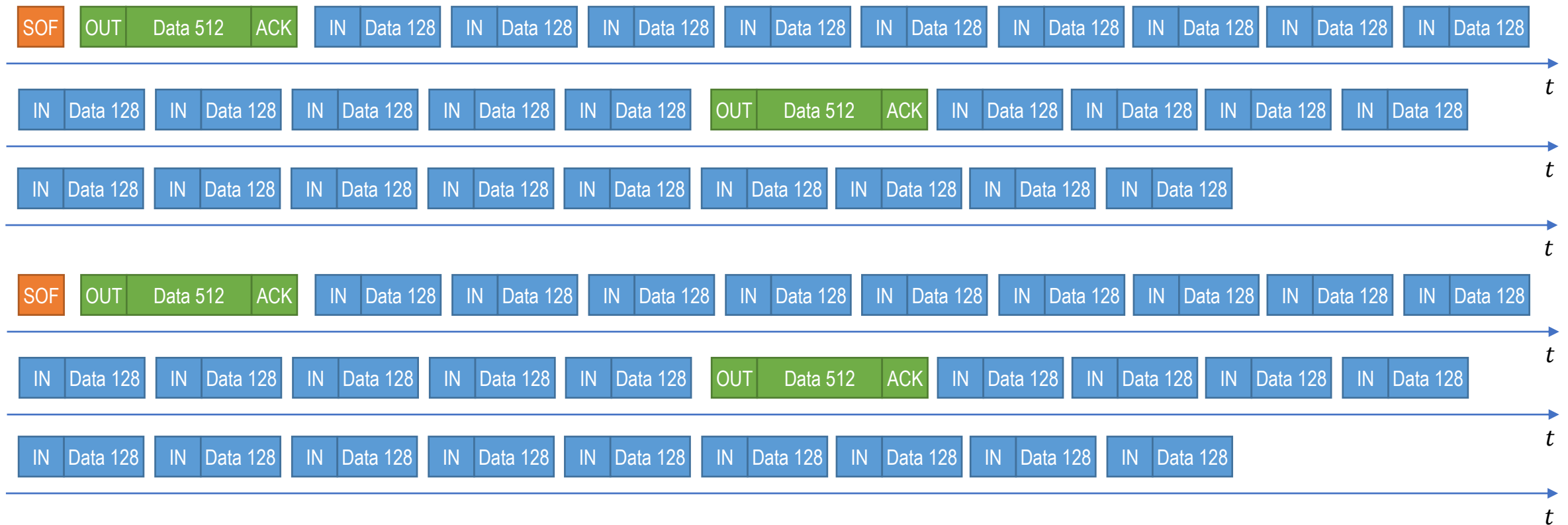
SOF token
PID = 0101₂

Universal Serial Bus (USB)

- USB Protocol Example
 - Number of bytes left for Bulk transactions
 - $6000 - 4482 - 19 = 1499$
 - Total number of 512 bytes Bulk transactions in a frame
 - $1499/512 = 2.92$
 - Only 2 Bulk transactions fit in a frame
 - There are some unused bytes
 - $\text{Unused} = 1499 - 2 \cdot 512 = 475$ bytes
 - The file transfer requires 13 Bulk transactions
 - It takes 6.5 USB frames
 - Total time: $6.5 \cdot 125\mu s = 812.5\mu s$
 - Transfer rate: $\frac{6656}{812.5\mu s} = 8192000 \text{ bytes/s} \approx 8 \text{ Mbyte/s}$

Universal Serial Bus (USB)

- USB Protocol Example
 - Two frames timing diagram
 - Each frame: 1 SOF, 2 BULK OUT, 27 ISO IN



Summary of interconnections

Protocol	Masters/Slaves	Direction	Arbitration	Addressing	Encoding	Bit synch	Shared wires	Error detection	Noise rejection	Speed	Length
UART	1/1	Single	N/A	Point-to-point	NRZ	Start Bit	N/A	Parity	N/A	100 kbps	5/50 m
SPI	1/m	Full Duplex	N/A	SPISS	NRZ	SPICLK	Slave Select	N/A	N/A	2/10 Mbps	20 cm
I ² C	n/m	Single	OpenDrain Lowest Addr	7-bit address	NRZ	SCLK	OpenDrain	ACK	N/A	400 kbps	20 cm
CAN	n	Single	OpenDrain Lowest Id	11-bit address	NRZ DS	Embedded Clock (bit stuffing)	OpenDrain	CRC+ACK	Differential Signaling	10 kbps 1 Mbps	2 km
USB	1/m	Single	N/A	7-bit address 4-bit endpoint	NRZI DS	Embedded Clock (bit stuffing)	Addressing Protocol	CRC+ACK	Differential Signaling Shielded	480 Mbps 5/10 Gbps	5/10 m